

本节内容

插入排序

插入排序



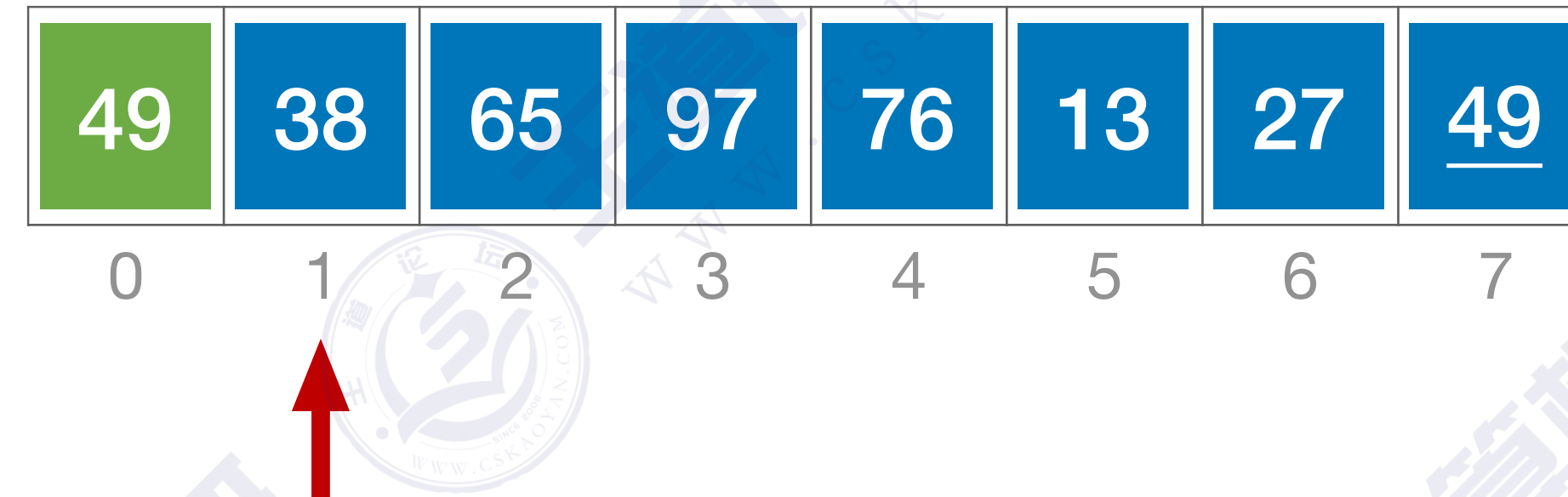
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。

49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7

插入排序



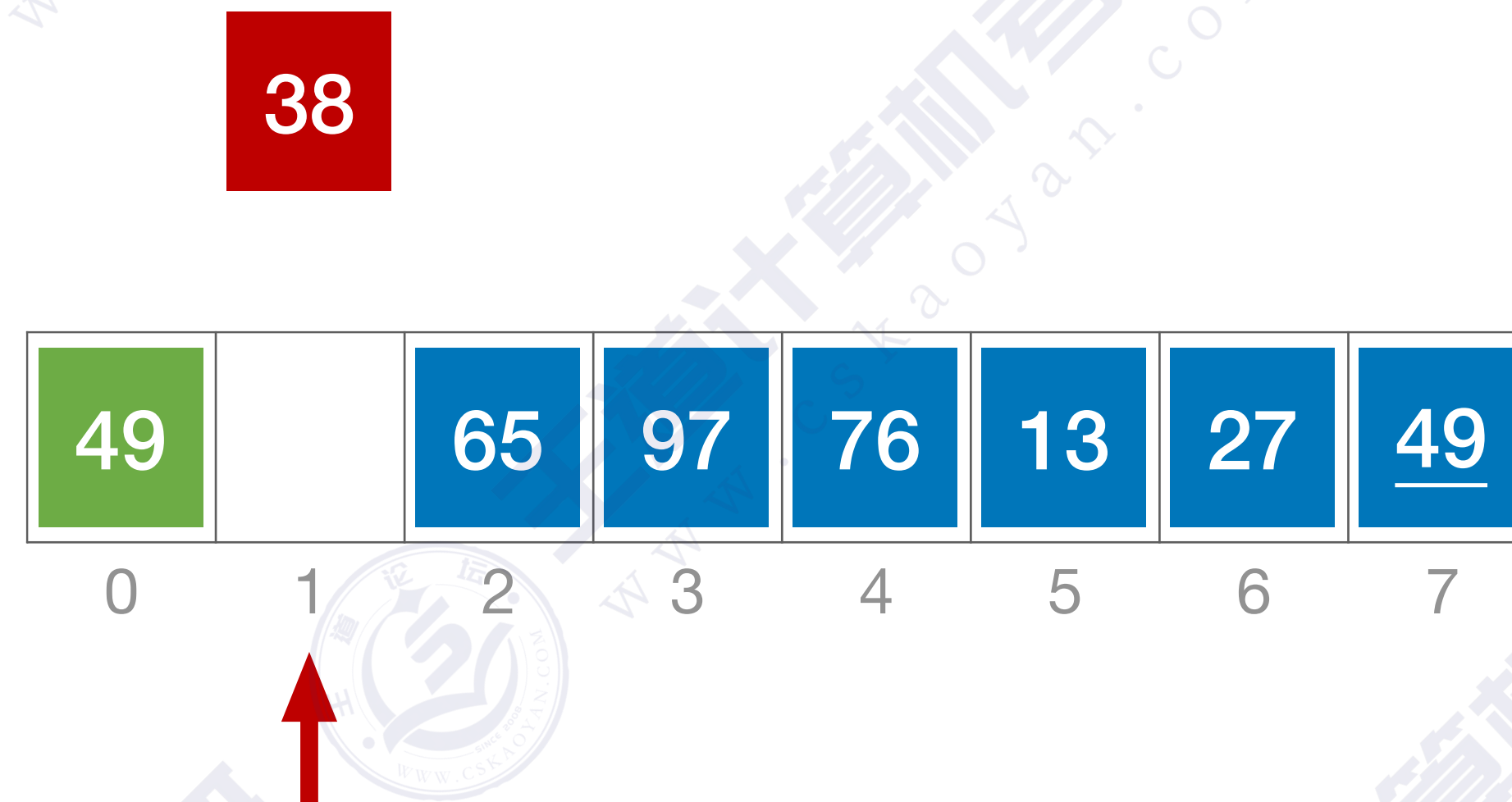
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



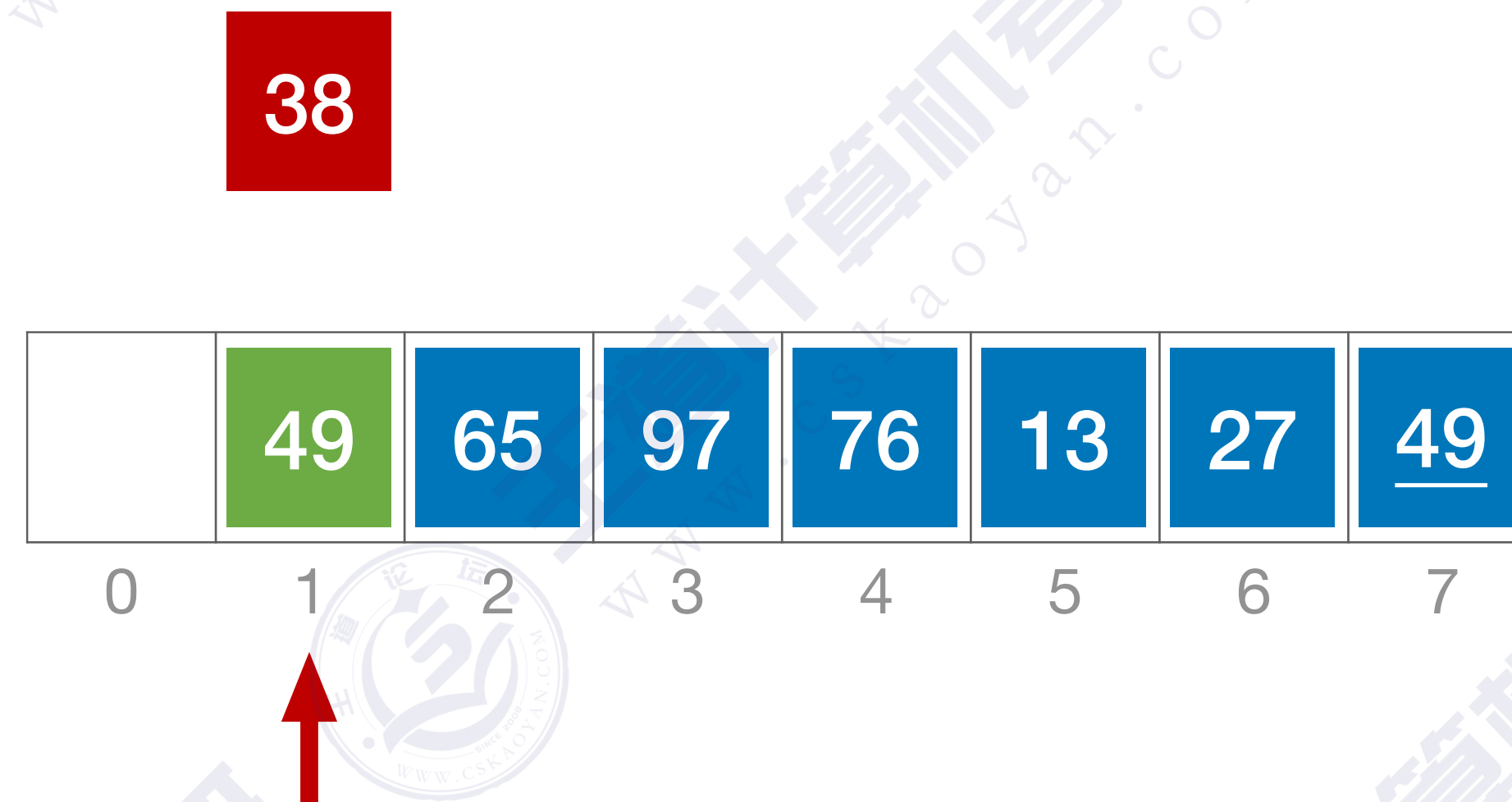
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



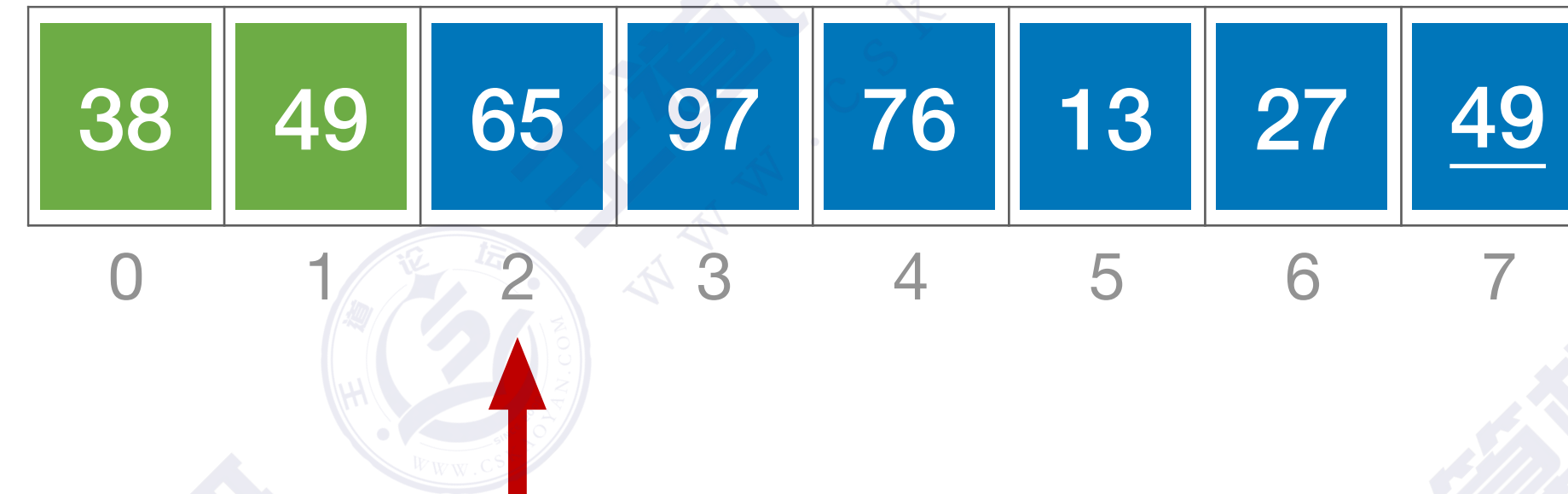
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



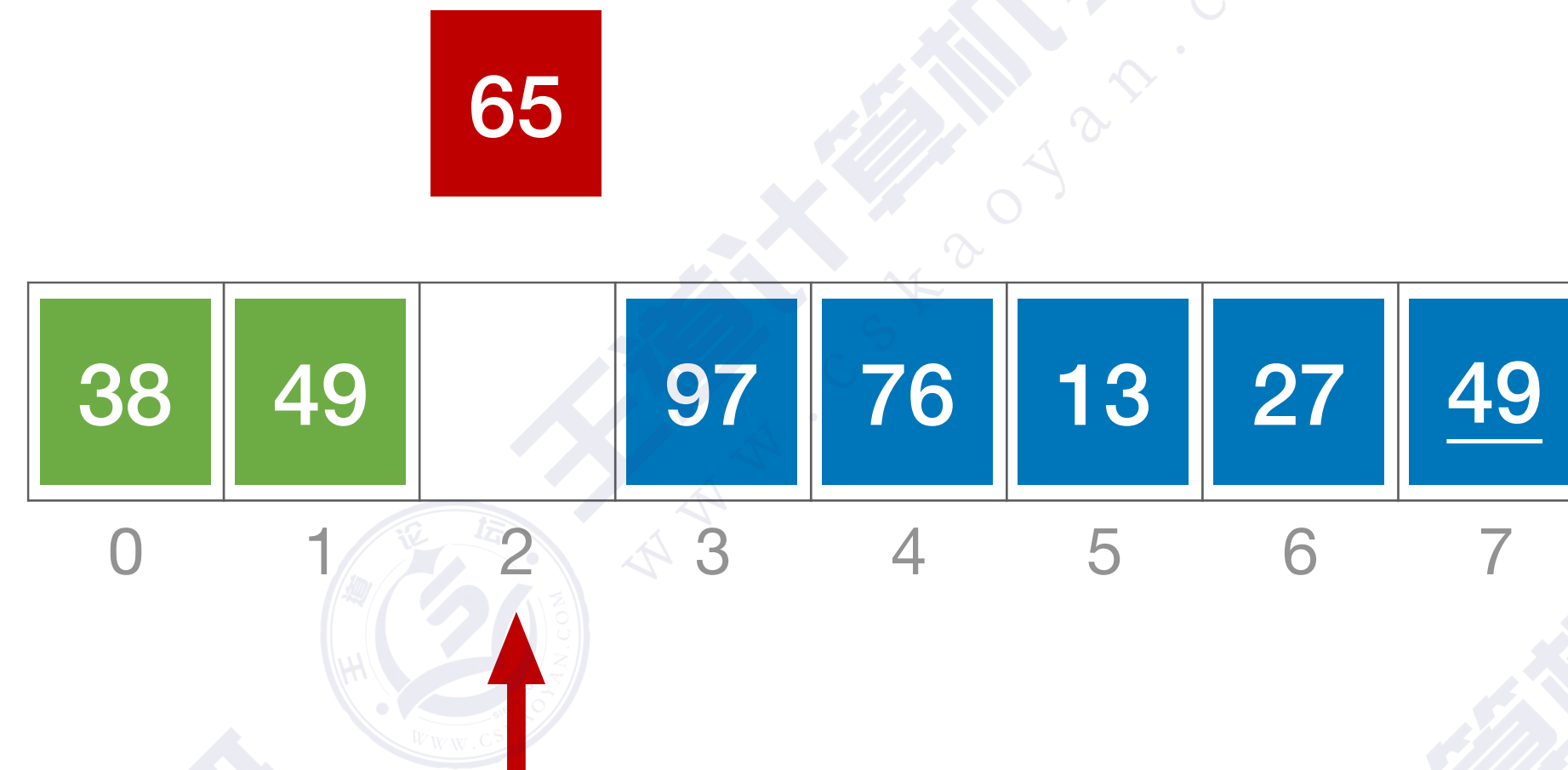
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



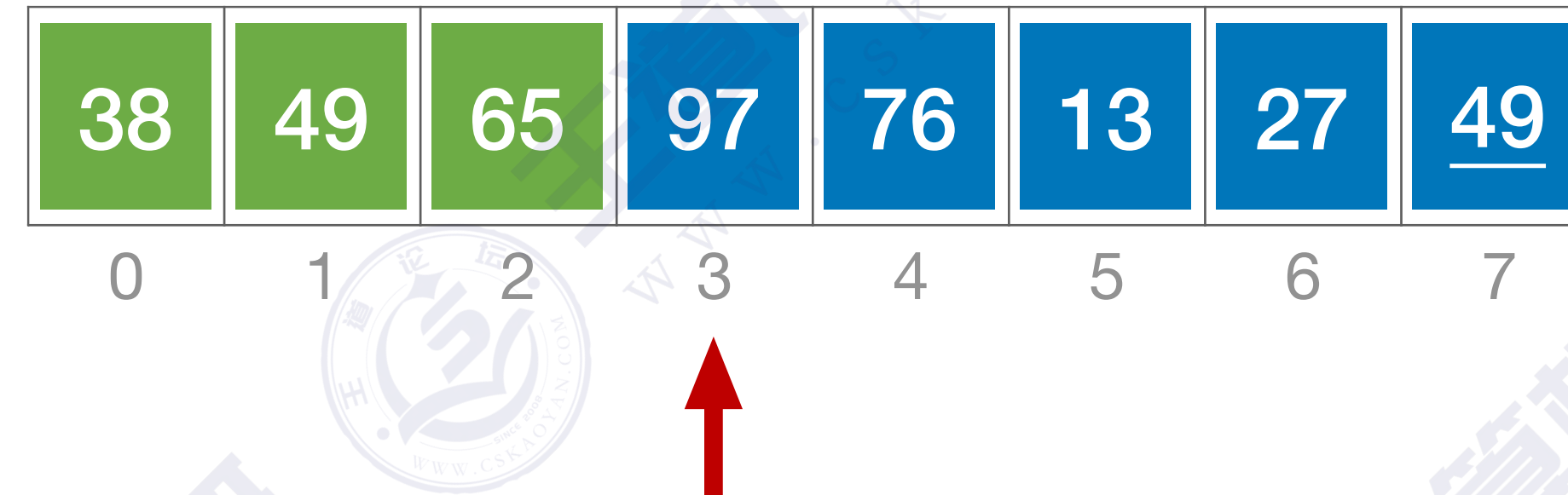
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



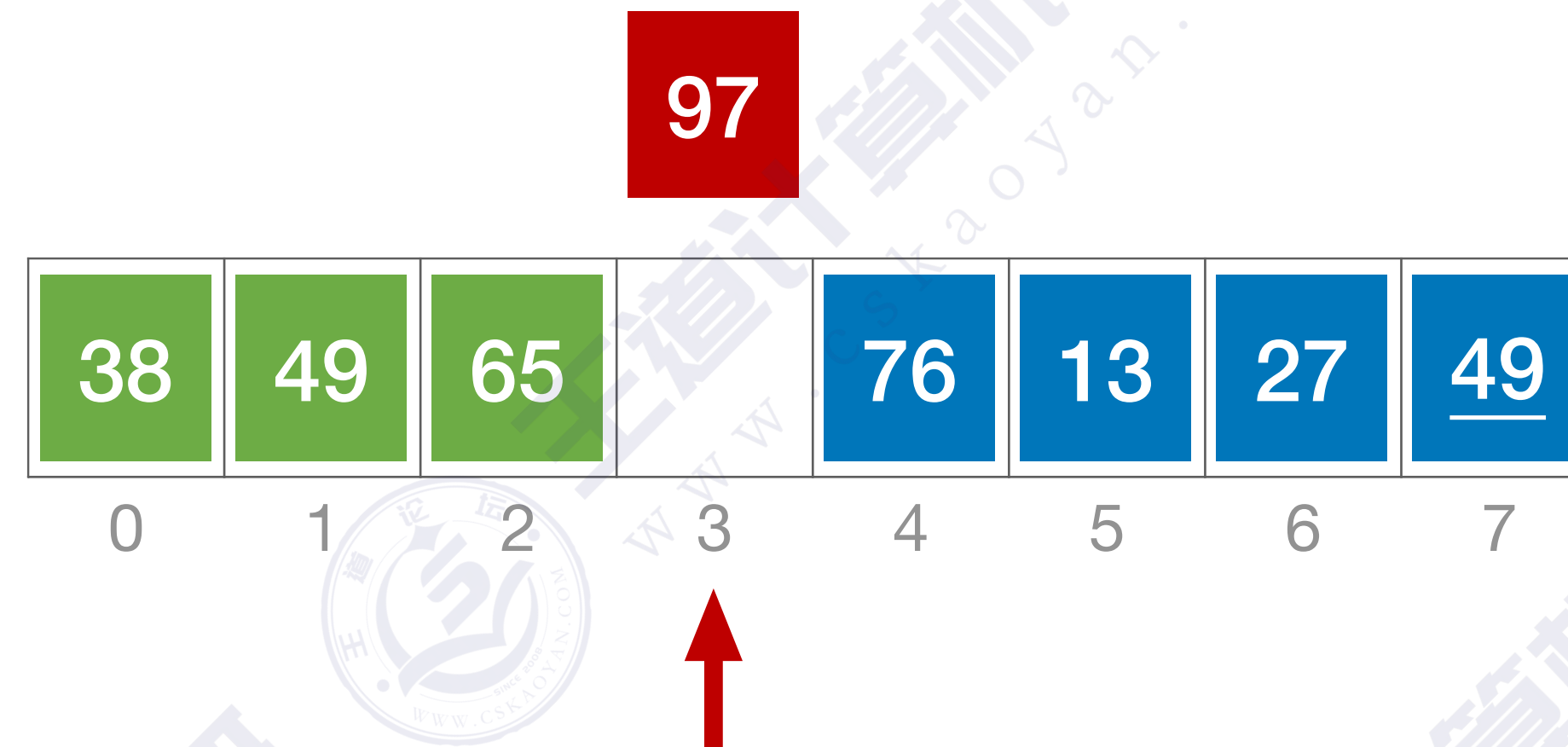
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



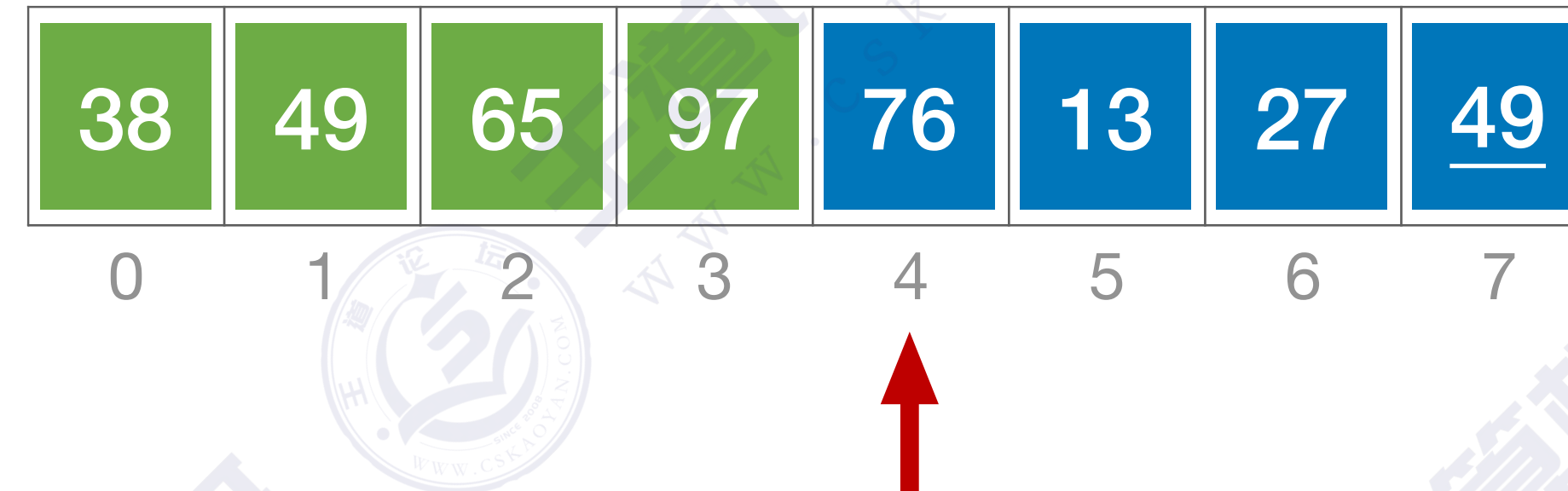
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



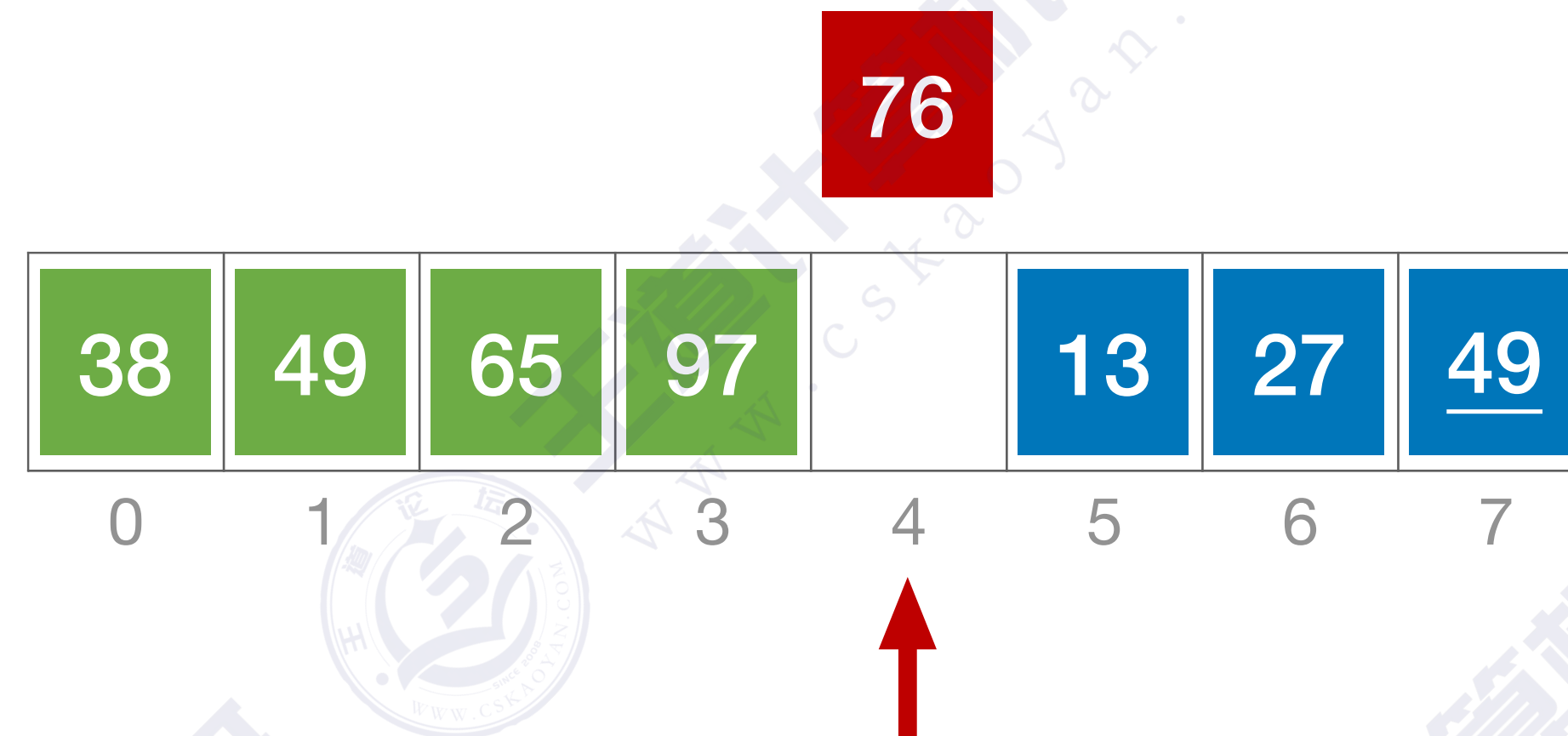
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



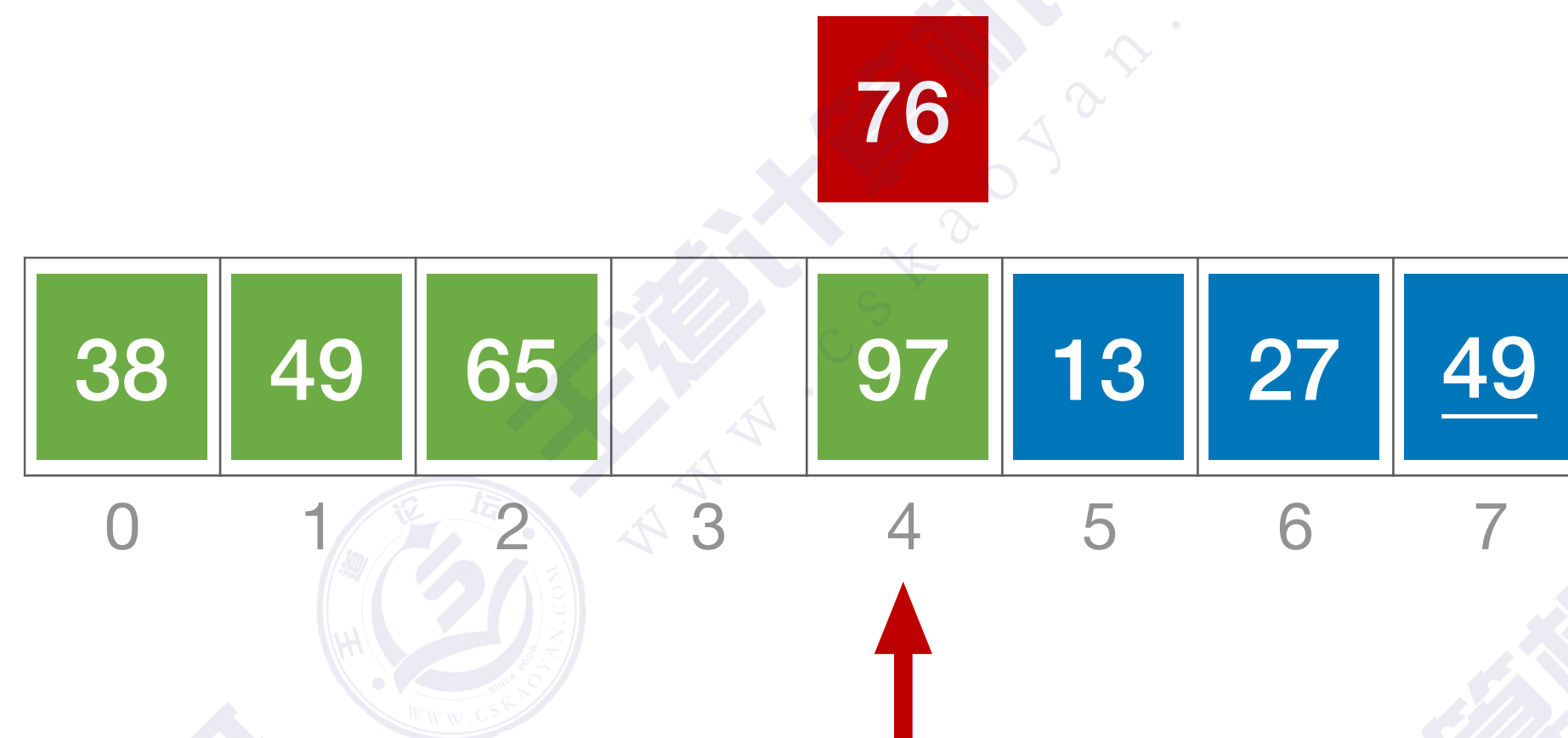
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



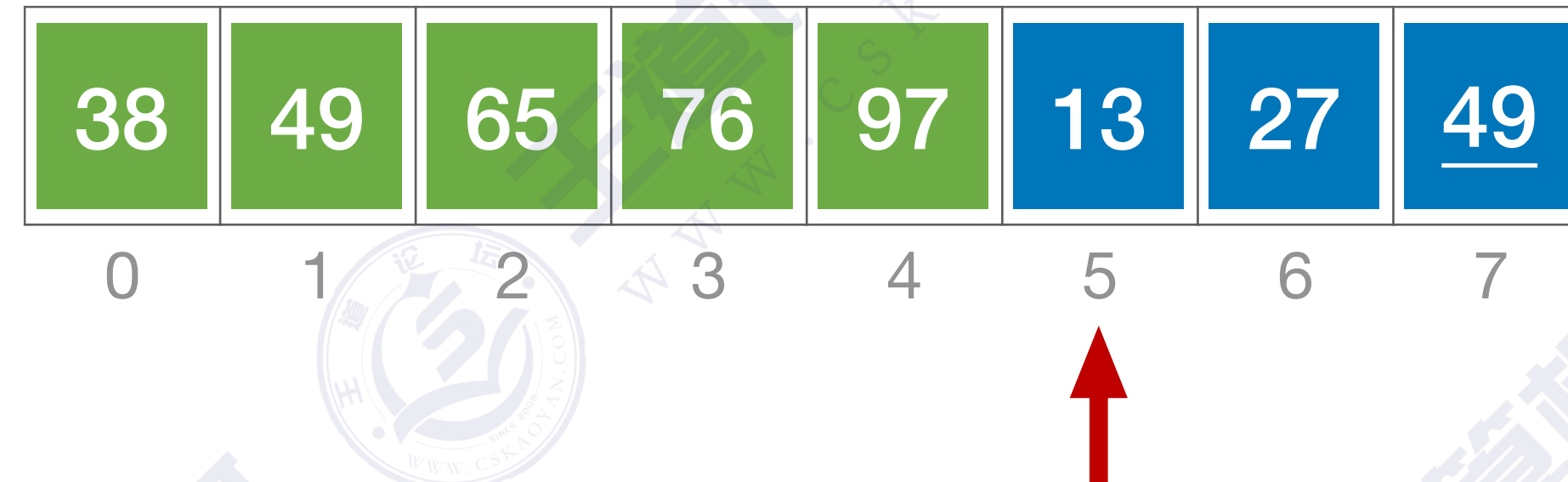
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



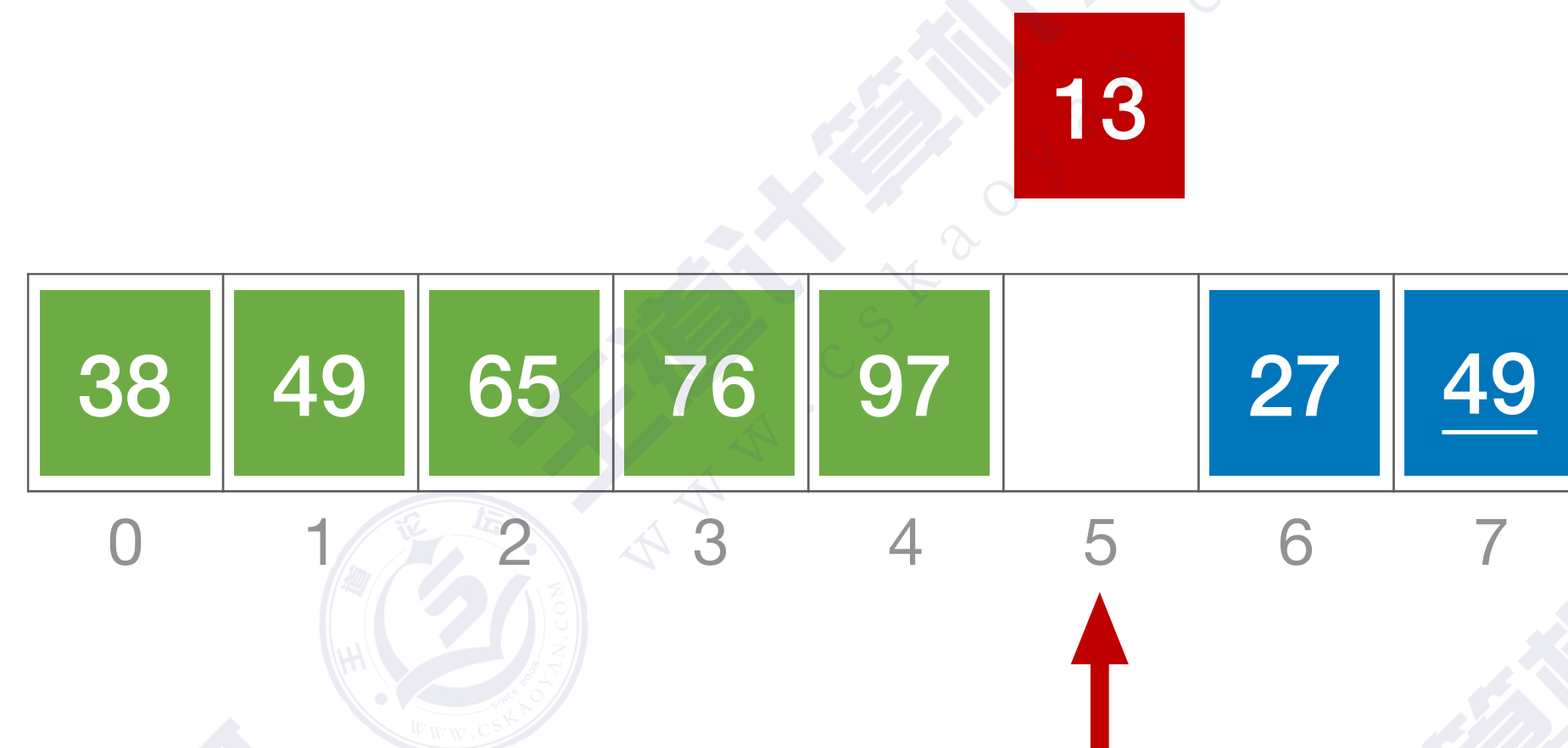
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



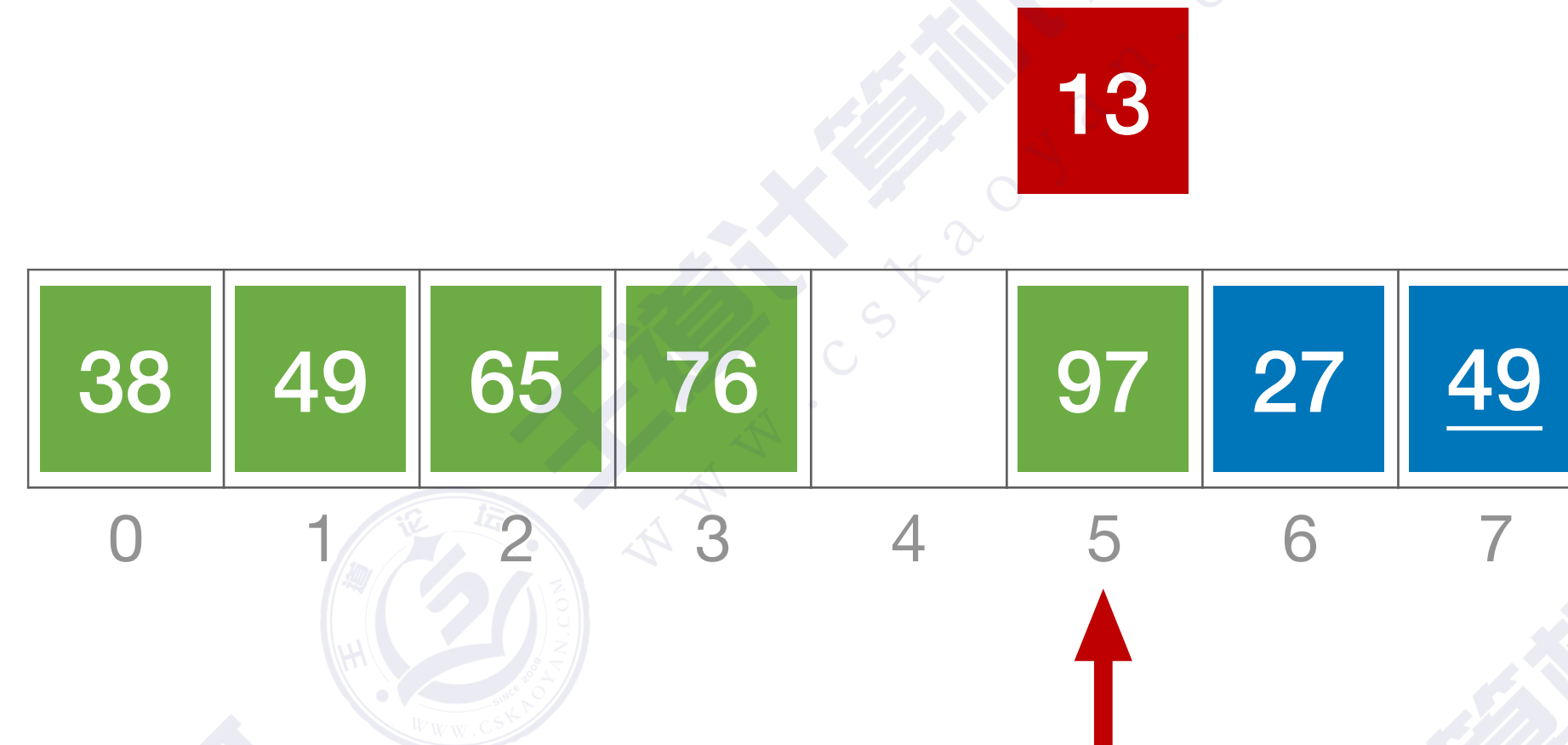
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



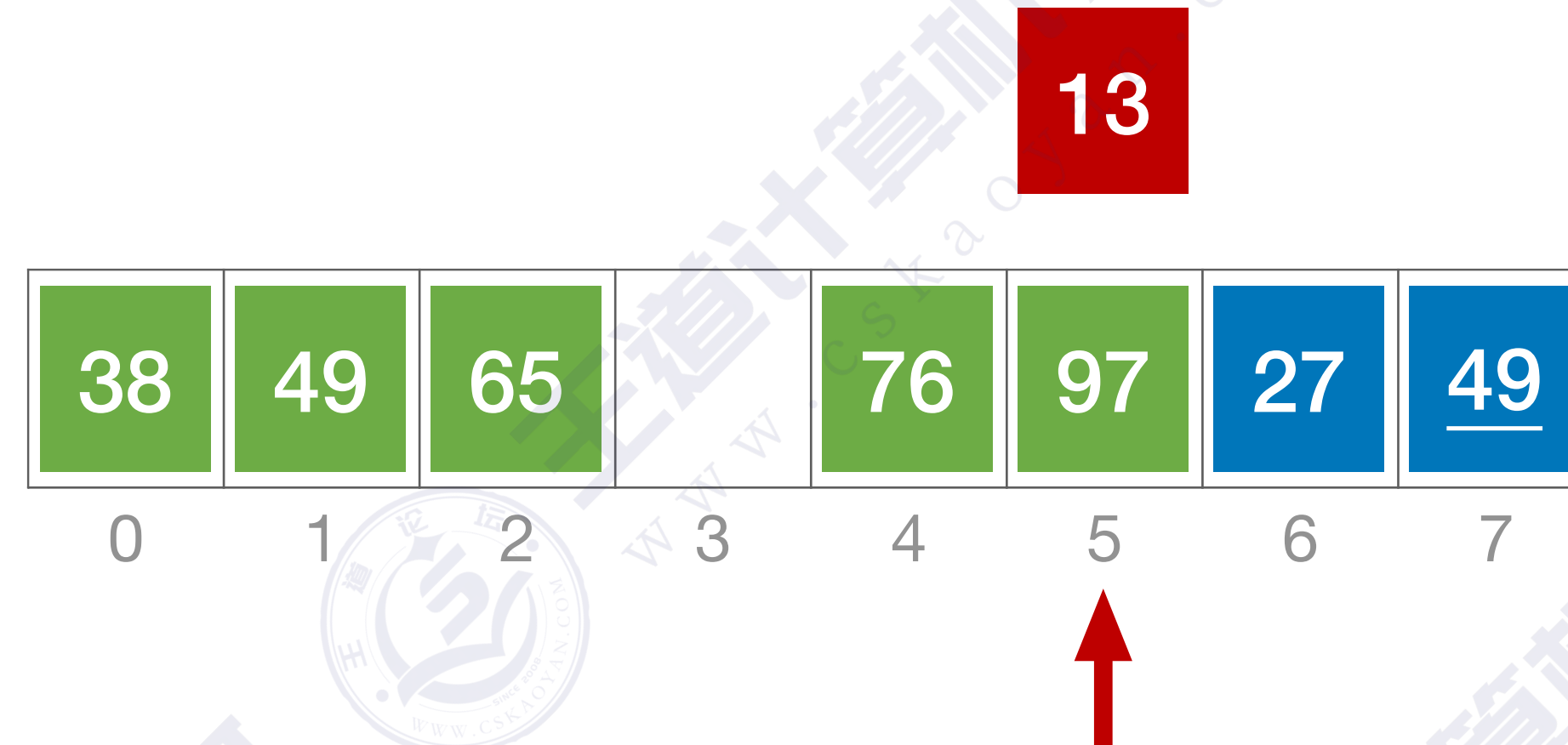
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



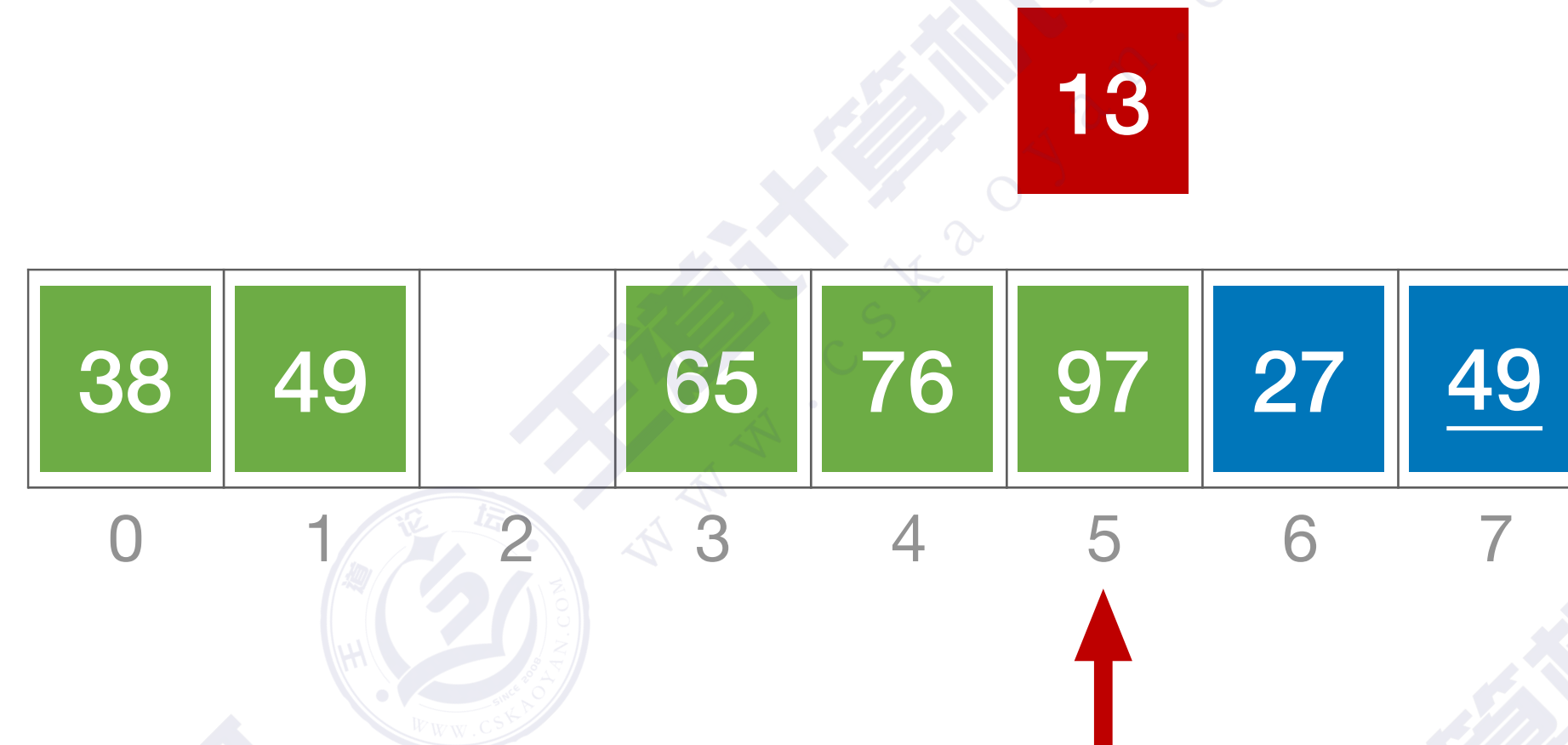
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



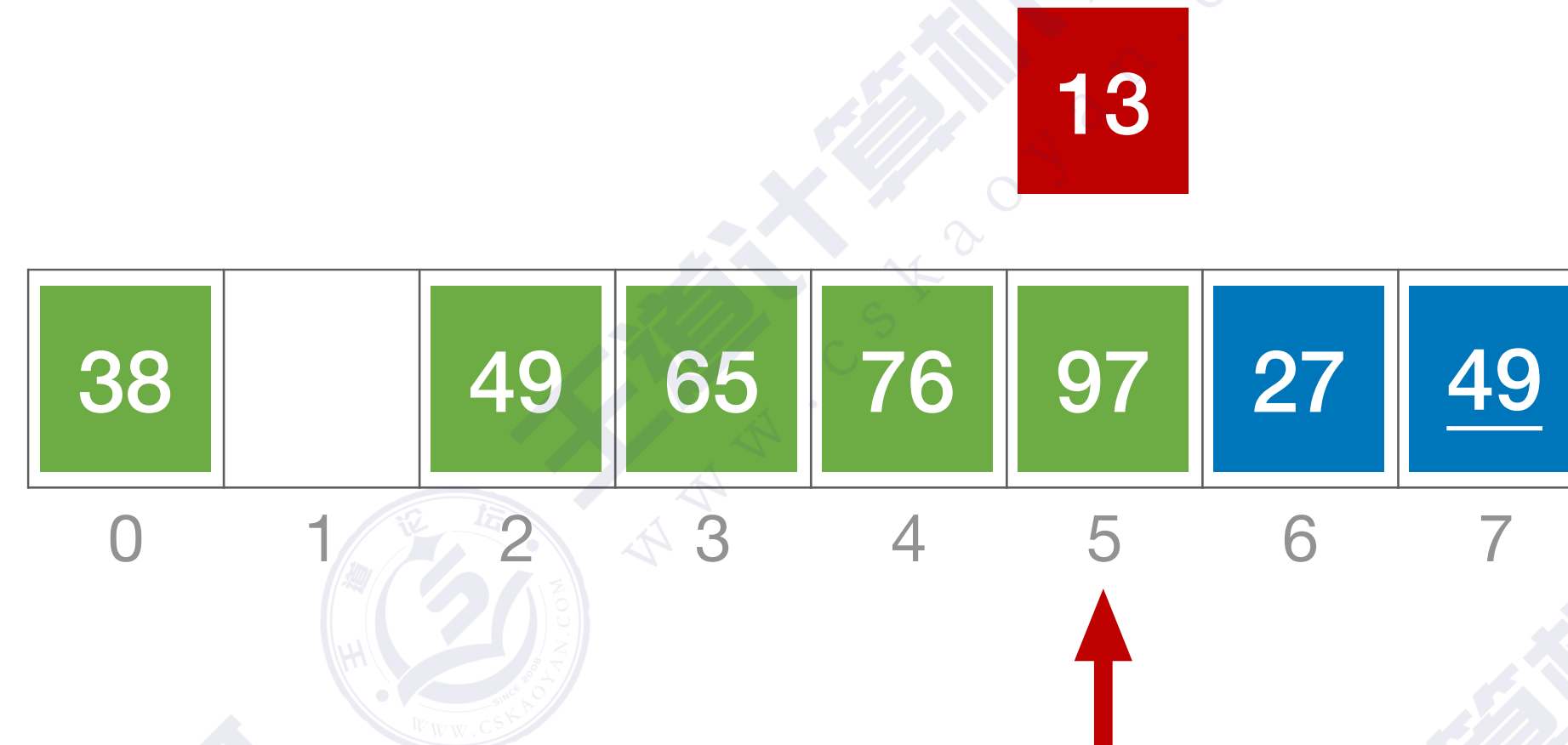
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



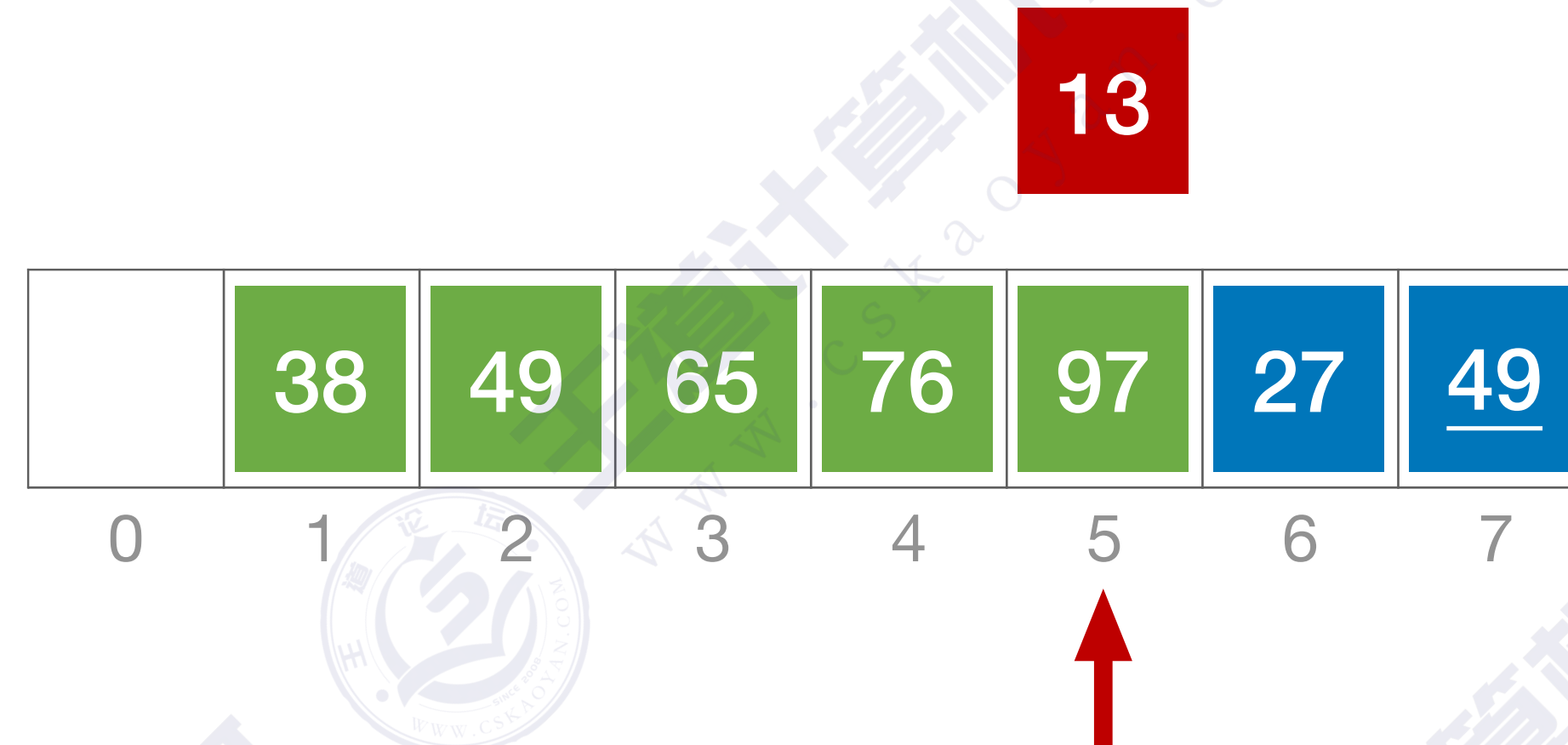
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



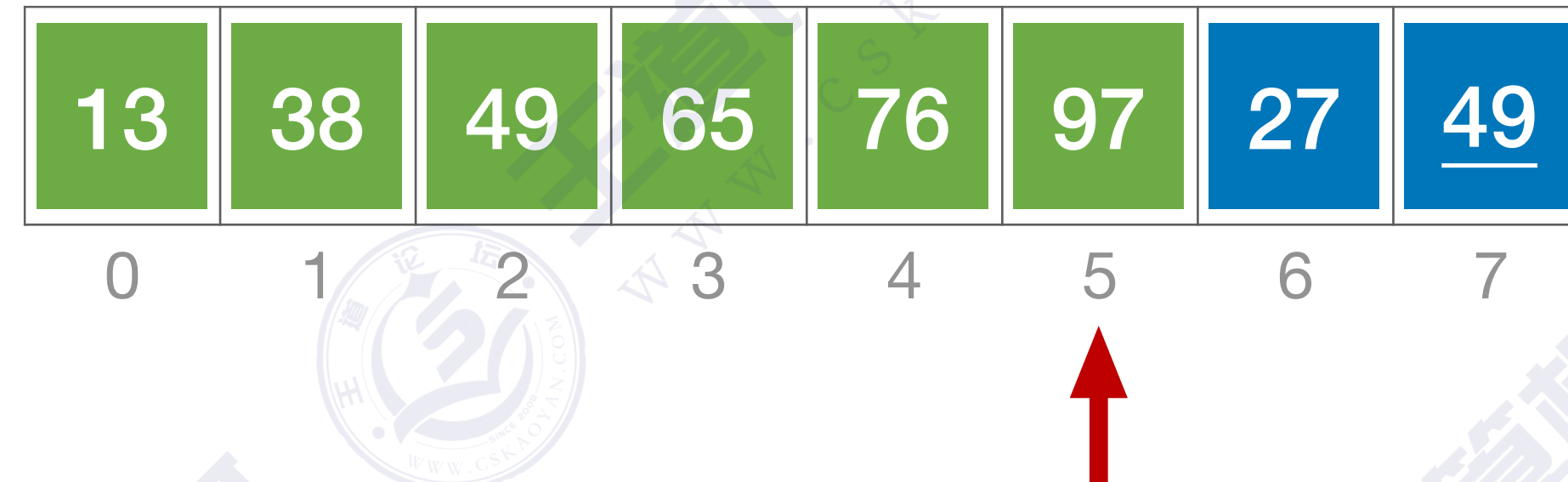
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



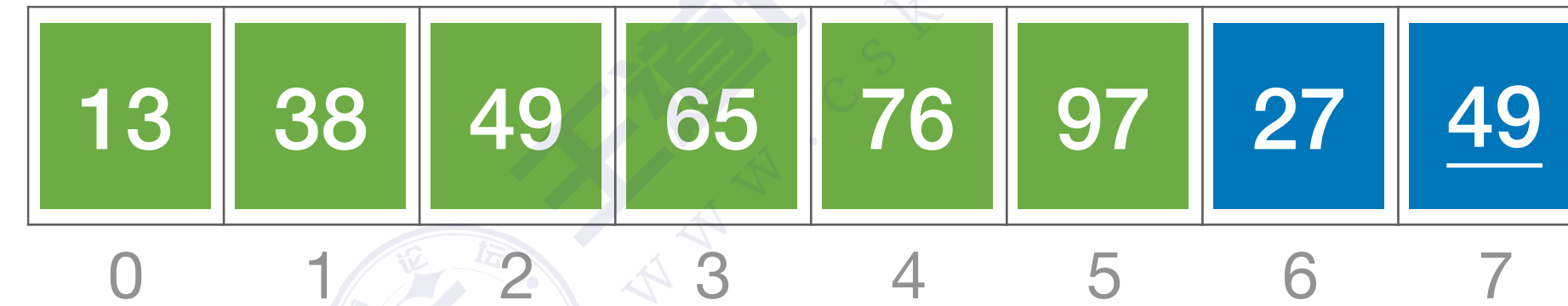
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



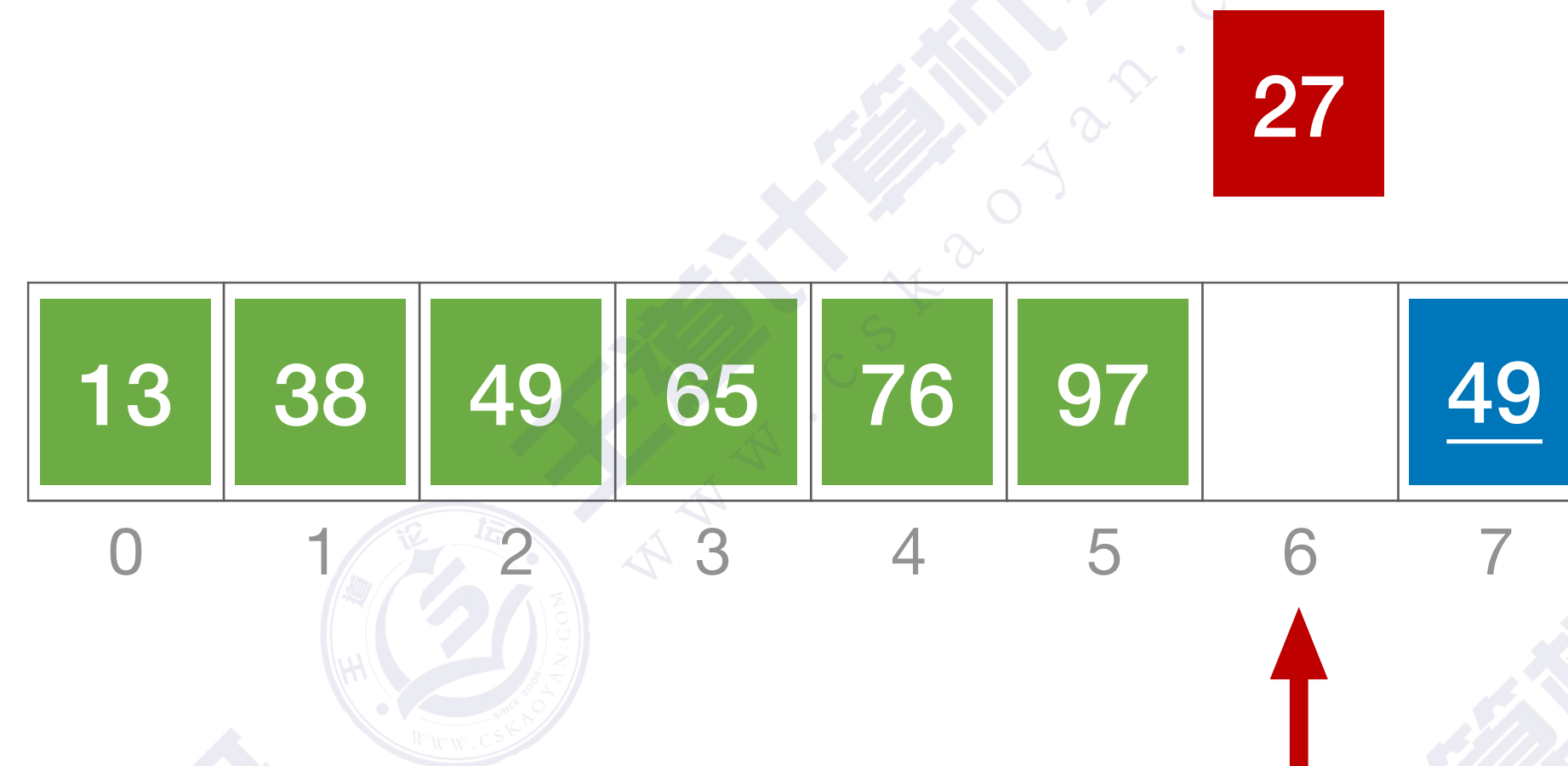
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



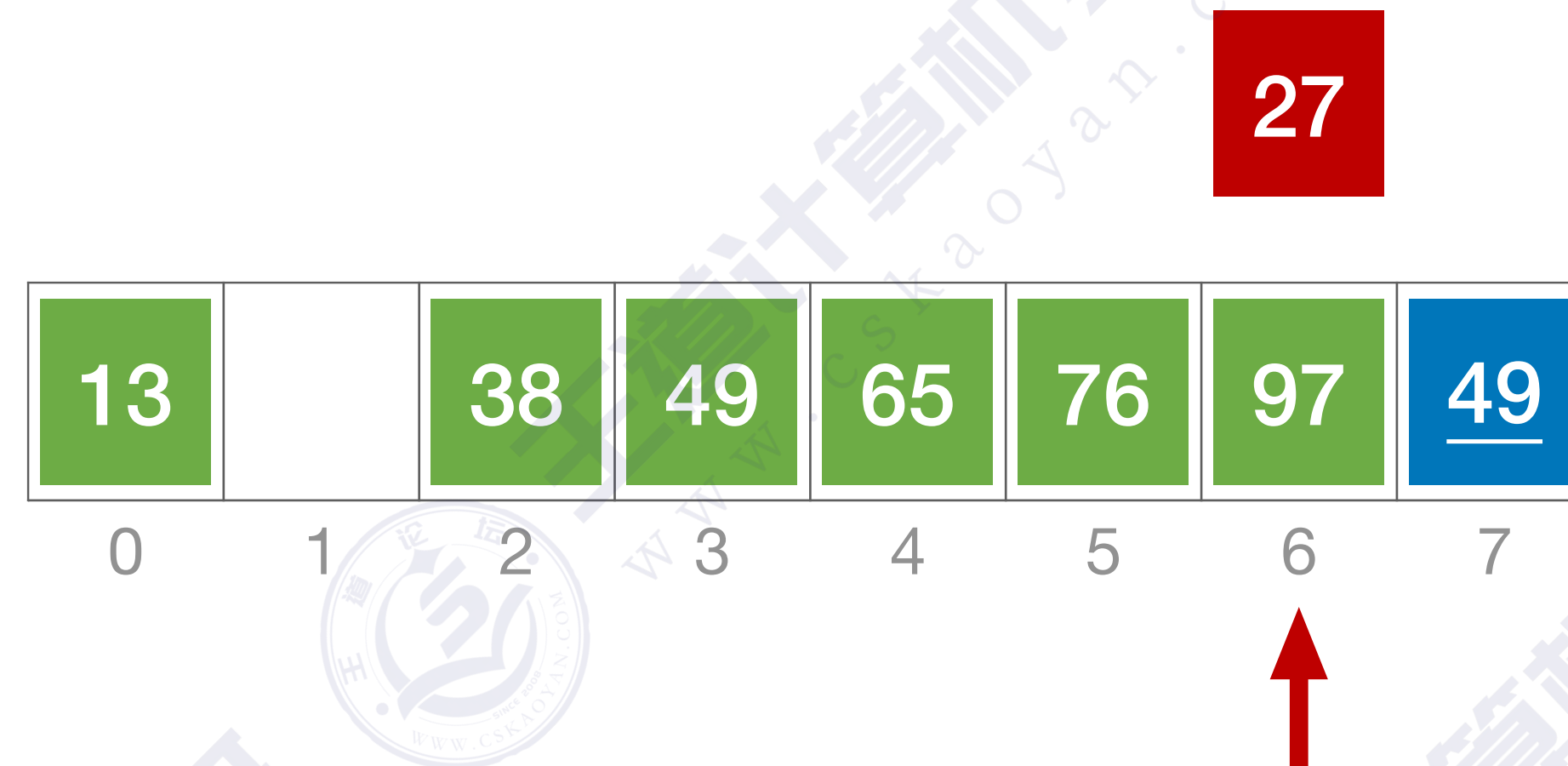
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



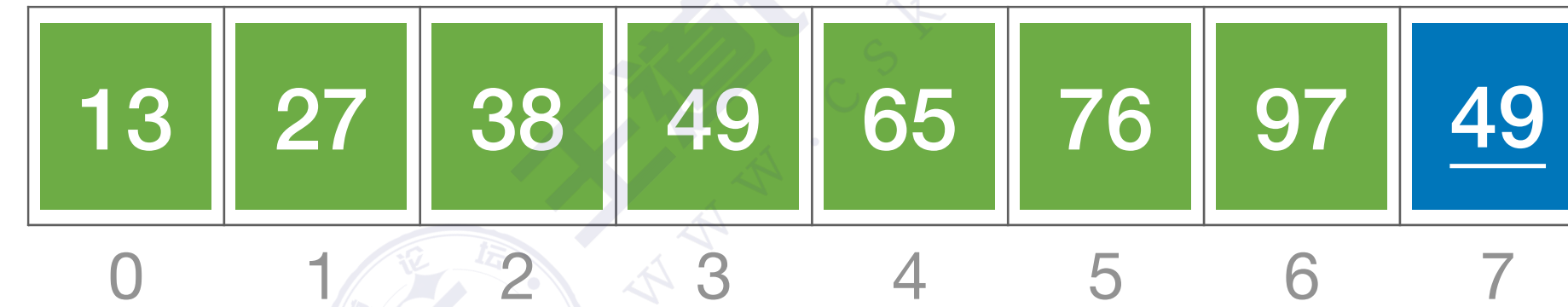
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



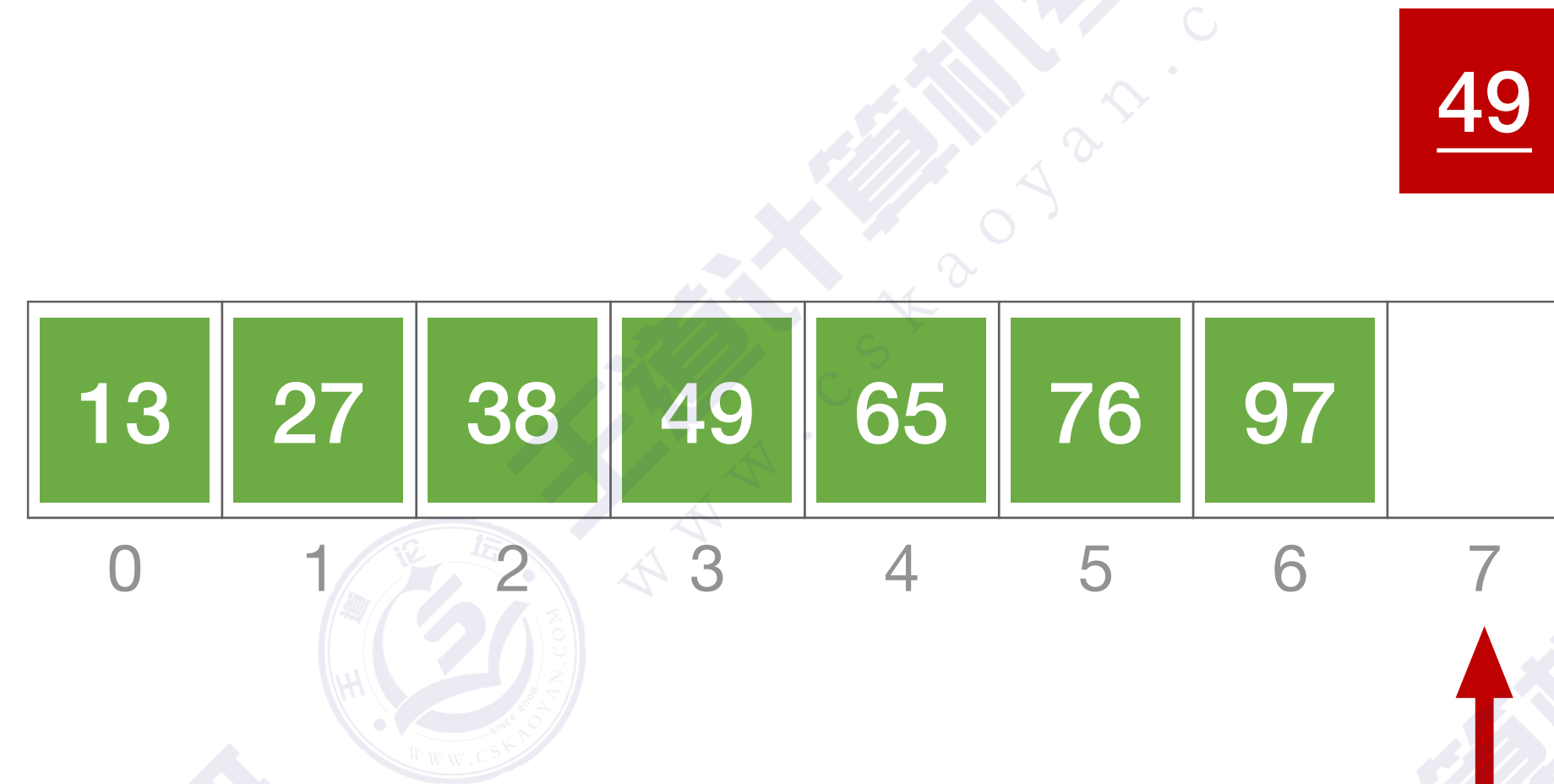
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



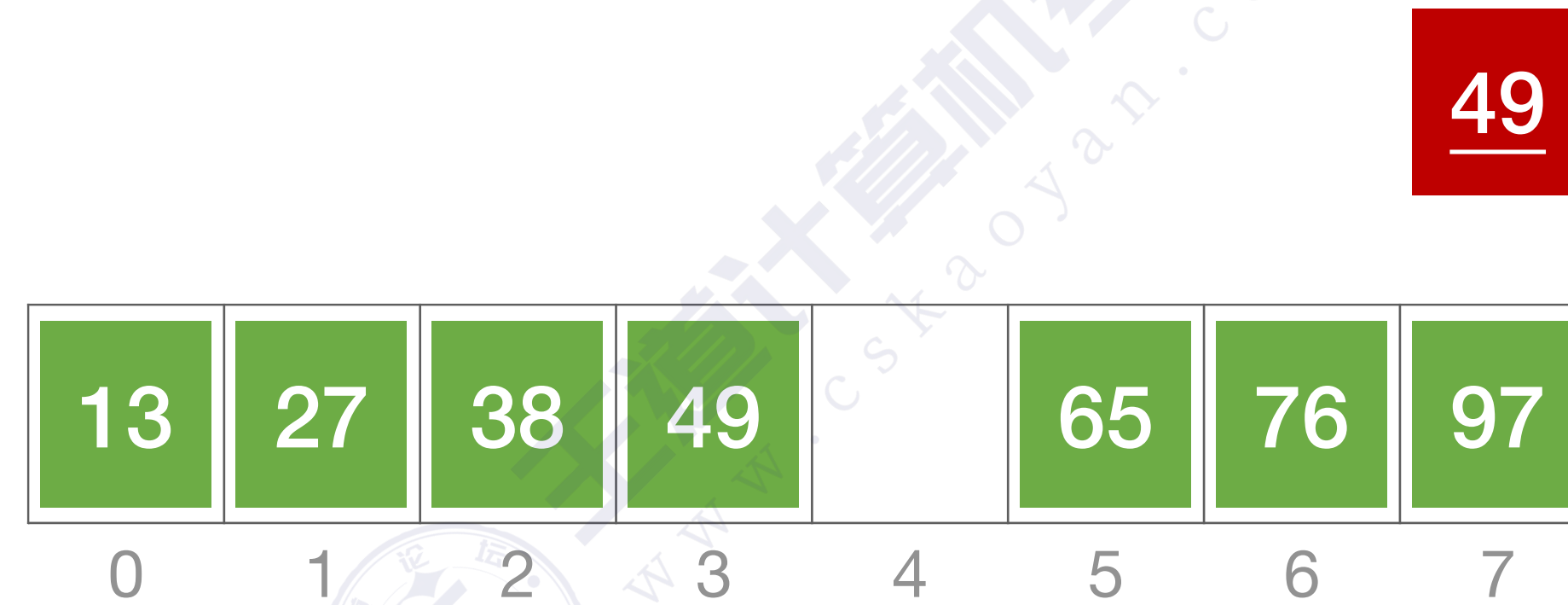
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



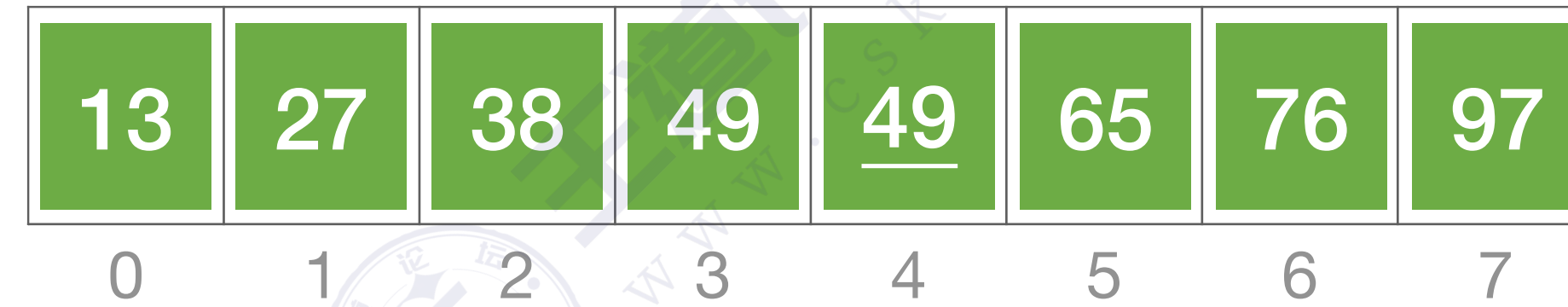
算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



插入排序



算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



算法实现

//直接插入排序

```
void InsertSort(int A[],int n){
```

```
    int i,j,temp;
```

```
    for(i=1;i<n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            temp=A[i];
```

```
            for(j=i-1;j>=0 && A[j]>temp;--j) //检查所有前面已排好序的元素
```

```
                A[j+1]=A[j]; //所有大于temp的元素都向后挪位
```

```
            A[j+1]=temp; //复制到插入位置
```

```
        }
```

```
    }
```

49	38	65	97	76	13	27	49
----	----	----	----	----	----	----	----

0

1

2

3

4

5

6

7

↑
i

算法实现

//直接插入排序

```
void InsertSort(int A[],int n){
```

```
    int i,j,temp;
```

```
    for(i=1;i<n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            temp=A[i];
```

```
            for(j=i-1;j>=0 && A[j]>temp;--j) //检查所有前面已排好序的元素
```

```
                A[j+1]=A[j]; //所有大于temp的元素都向后挪位
```

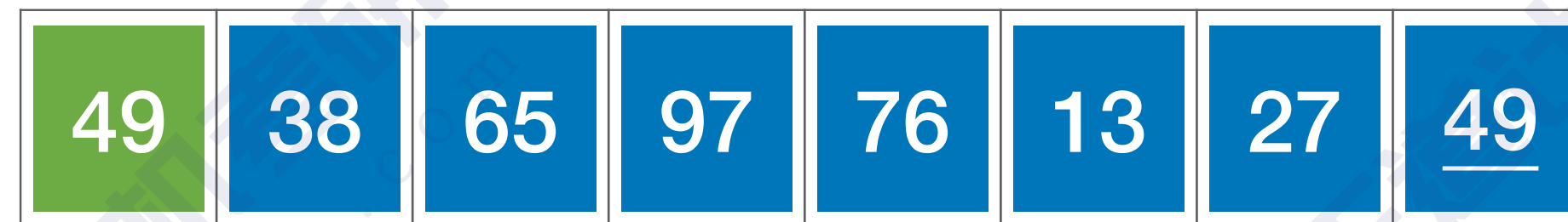
```
            A[j+1]=temp; //复制到插入位置
```

```
        }
```

```
    }
```

temp

38



0

1

2

3

4

5

6

7

↑
i

算法实现

//直接插入排序

```
void InsertSort(int A[],int n){
```

```
    int i,j,temp;
```

```
    for(i=1;i<n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            temp=A[i];
```

```
            for(j=i-1;j>=0 && A[j]>temp;--j) //检查所有前面已排好序的元素
```

```
                A[j+1]=A[j]; //所有大于temp的元素都向后挪位
```

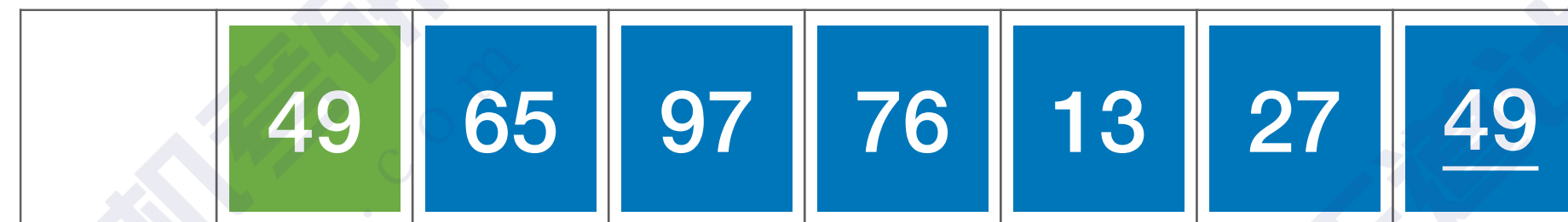
```
            A[j+1]=temp; //复制 to 插入位置
```

```
        }
```

```
    }
```

temp

38



0

1

2

3

4

5

6

7

↑
i

算法实现

//直接插入排序

```
void InsertSort(int A[],int n){
```

```
    int i,j,temp;
```

```
    for(i=1;i<n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            temp=A[i];
```

```
            for(j=i-1;j>=0 && A[j]>temp;--j)    //检查所有前面已排序的元素
```

```
                A[j+1]=A[j];    //所有大于temp的元素都向后挪位
```

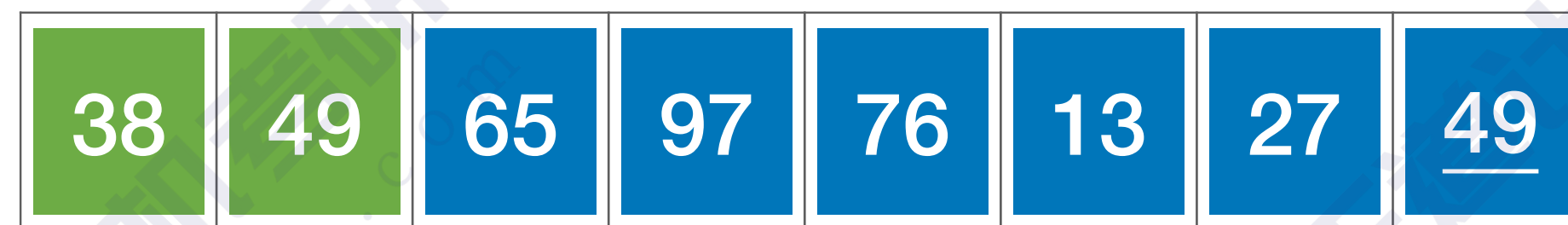
```
            A[j+1]=temp;    //复制 to 插入位置
```

```
        }
```

```
    }
```

temp

38



0

1

2

3

4

5

6

7

↑
i

算法实现（带哨兵）

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++){
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];        //向后挪位
```

```
            A[j+1]=A[0];        //复制到插入位置
```

```
        }
```

```
    }
```

//依次将 $A[2] \sim A[n]$ 插入到前面已排序序列

//若 $A[i]$ 关键码小于其前驱，将 $A[i]$ 插入有序表

//复制为哨兵， $A[0]$ 不存放元素

//向后挪位

//复制到插入位置



	49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7	8

算法实现（带哨兵）

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++){
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];        //向后挪位
```

```
            A[j+1]=A[0];        //复制到插入位置
```

```
        }
```

```
    }
```

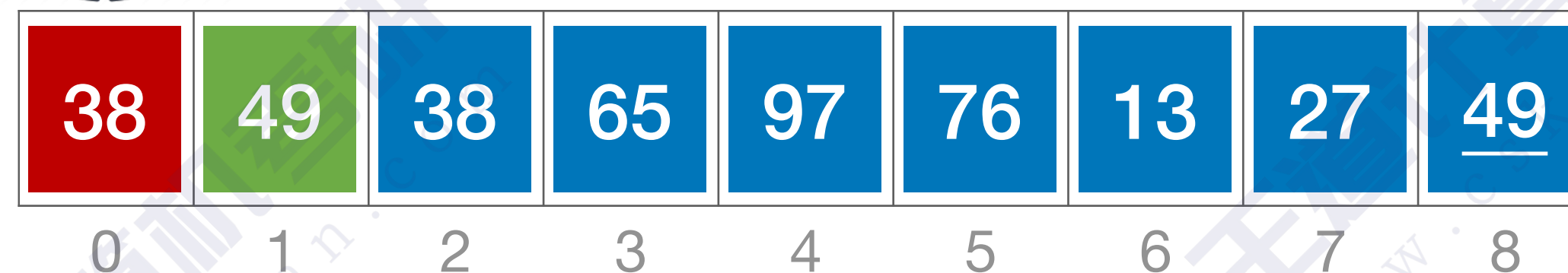
//依次将 $A[2] \sim A[n]$ 插入到前面已排序序列

//若 $A[i]$ 关键码小于其前驱，将 $A[i]$ 插入有序表

//复制为哨兵， $A[0]$ 不存放元素

//向后挪位

//复制到插入位置



↑
i

算法实现（带哨兵）

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++){
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];        //向后挪位
```

```
            A[j+1]=A[0];        //复制到插入位置
```

```
        }
```

```
    }
```

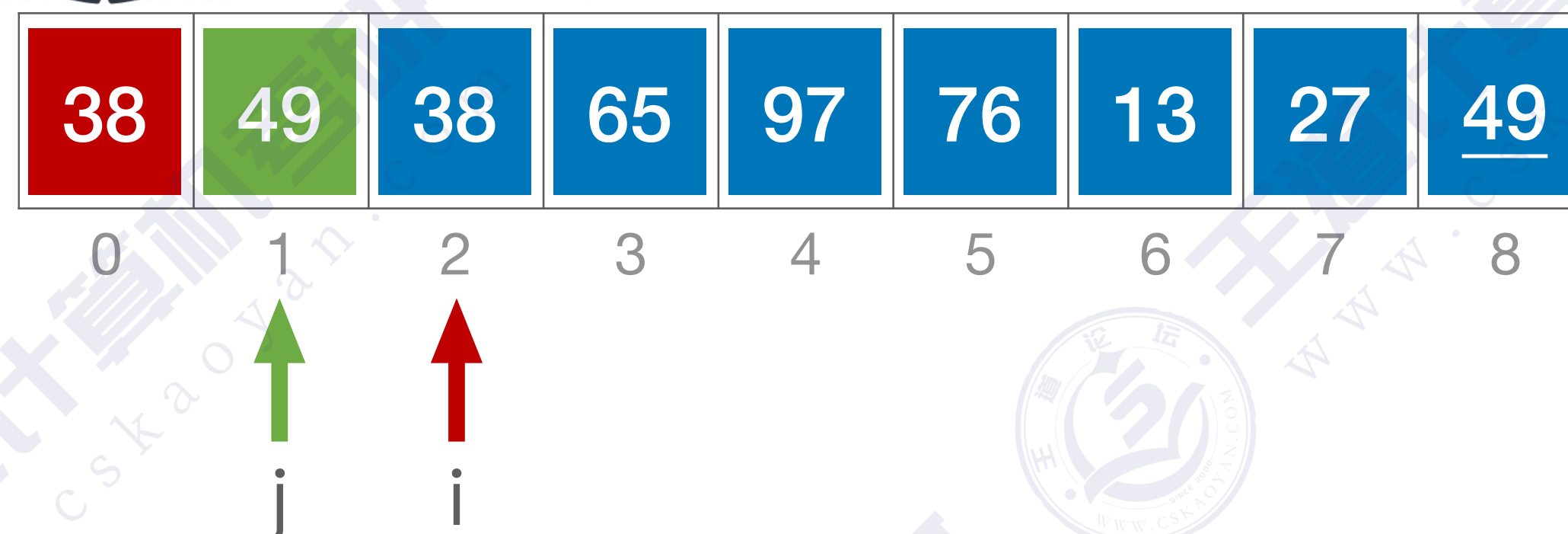
//依次将 $A[2] \sim A[n]$ 插入到前面已排序序列

//若 $A[i]$ 关键码小于其前驱，将 $A[i]$ 插入有序表

//复制为哨兵， $A[0]$ 不存放元素

//向后挪位

//复制到插入位置



算法实现（带哨兵）

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++){
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];        //向后挪位
```

```
            A[j+1]=A[0];        //复制到插入位置
```

```
        }
```

```
    }
```

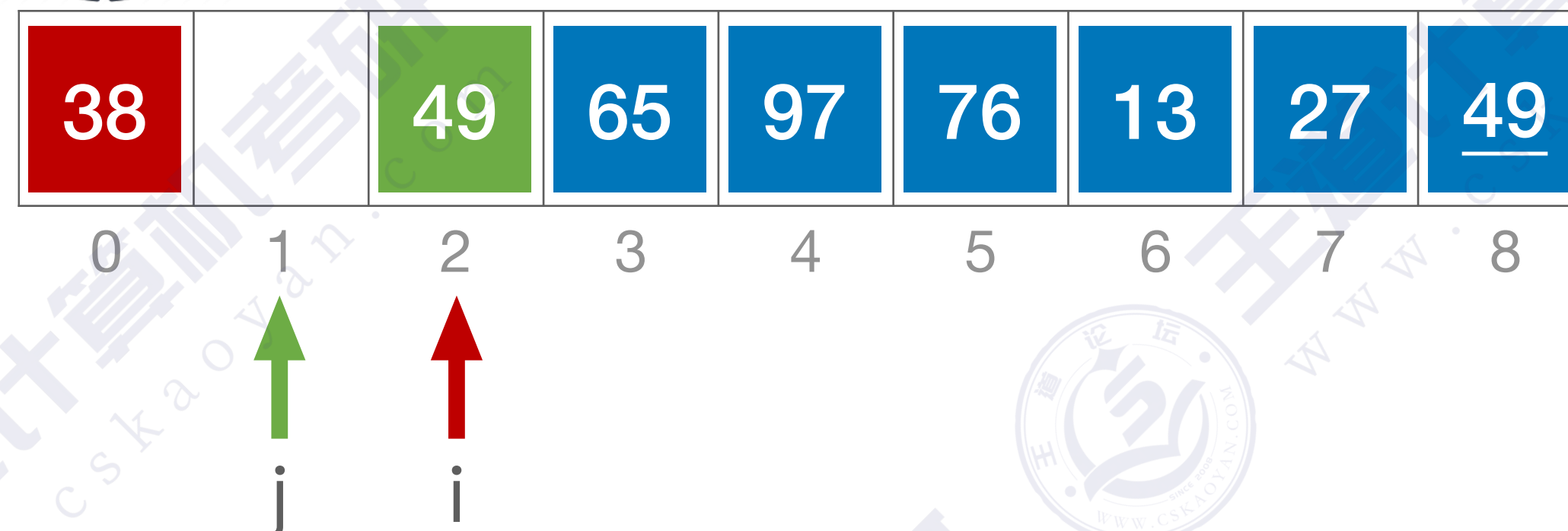
//依次将 $A[2] \sim A[n]$ 插入到前面已排序序列

//若 $A[i]$ 关键码小于其前驱，将 $A[i]$ 插入有序表

//复制为哨兵， $A[0]$ 不存放元素

//向后挪位

//复制到插入位置



算法实现（带哨兵）

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++){
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];        //向后挪位
```

```
            A[j+1]=A[0];        //复制到插入位置
```

```
        }
```

```
    }
```

//依次将A[2]~A[n]插入到前面已排序序列

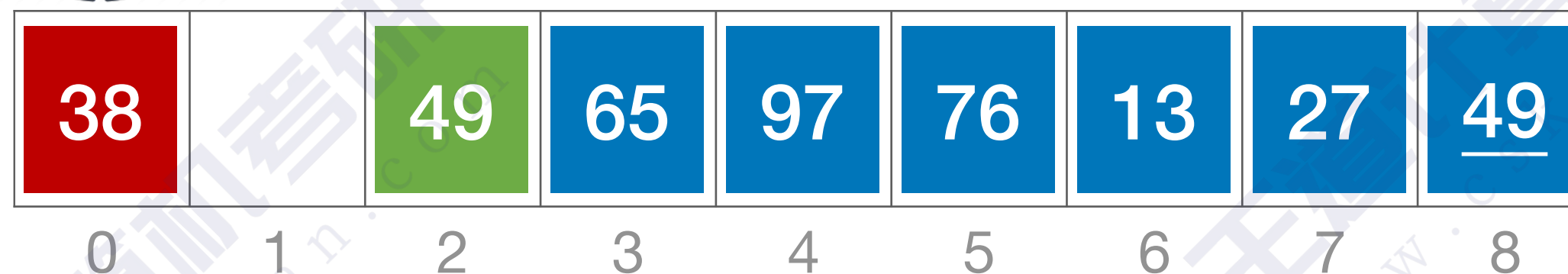
//若A[i]关键码小于其前驱，将A[i]插入有序表

//复制为哨兵，A[0]不存放元素

//从后往前查找待插入位置

//向后挪位

//复制到插入位置



↑
j

↑
i

算法实现（带哨兵）

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++){
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];        //向后挪位
```

```
            A[j+1]=A[0];        //复制到插入位置
```

```
        }
```

```
    }
```

//依次将A[2]~A[n]插入到前面已排序序列

//若A[i]关键码小于其前驱，将A[i]插入有序表

//复制为哨兵，A[0]不存放元素

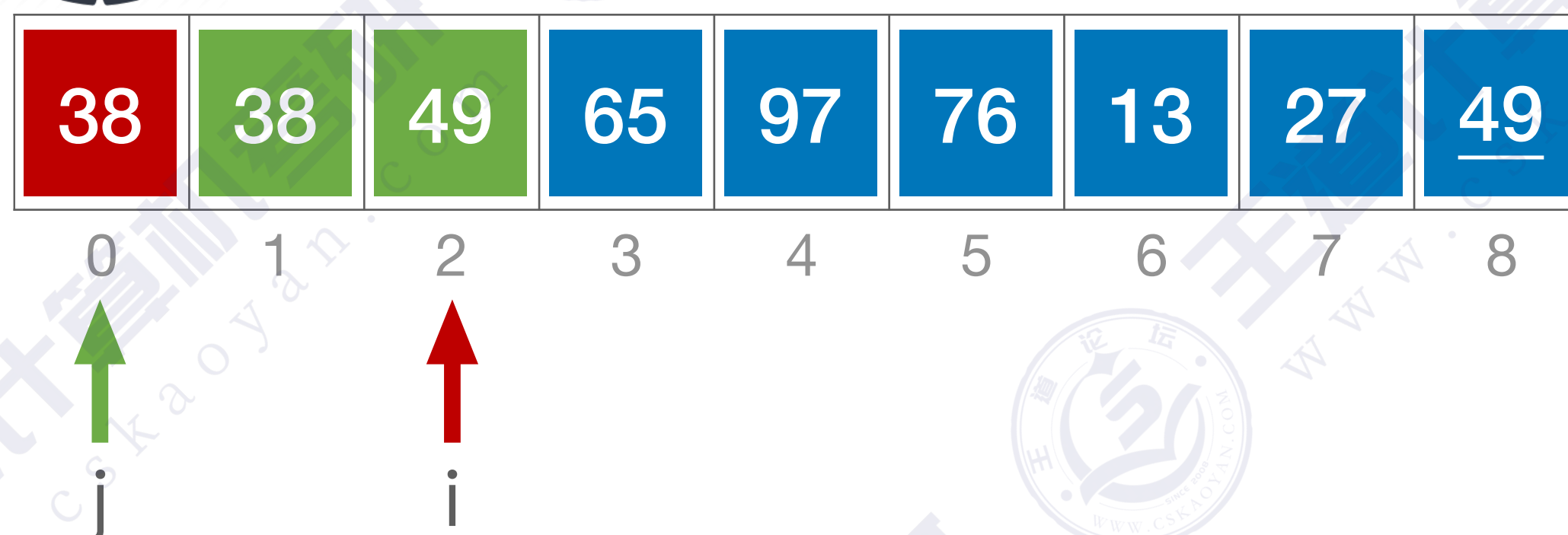
//从后往前查找待插入位置

//向后挪位

//复制到插入位置



优点：不用每轮循环都判断 $j \geq 0$



算法效率分析

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];
```

```
            A[j+1]=A[0];
```

```
        }
```

```
    }
```

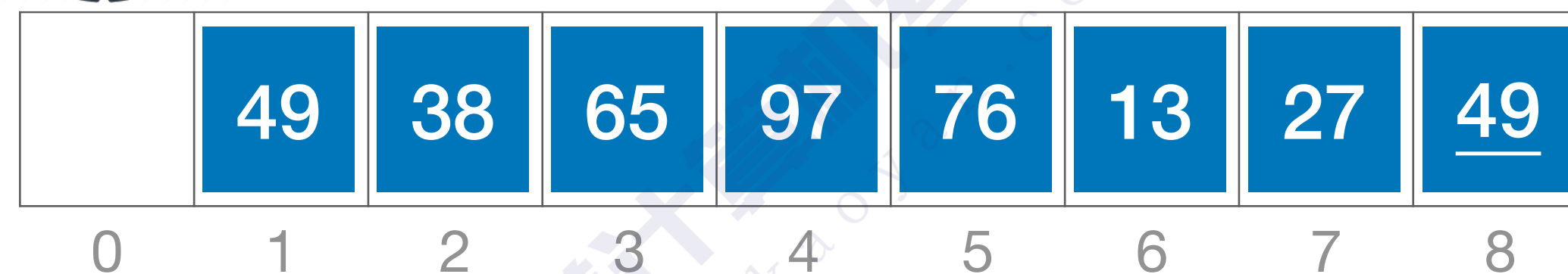
//依次将A[2]~A[n]插入到前面已排序序列

//若A[i]关键码小于其前驱，将A[i]插入有序表

//复制为哨兵，A[0]不存放元素

//向后挪位

//复制到插入位置



空间复杂度：O(1)

时间复杂度：主要来自对比关键字、移动元素
若有 n 个元素，则需要 n-1 趟处理

算法效率分析

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];
```

```
            A[j+1]=A[0];
```

```
        }
```

```
    }
```

//依次将A[2]~A[n]插入到前面已排序序列

//若A[i]关键码小于其前驱，将A[i]插入有序表

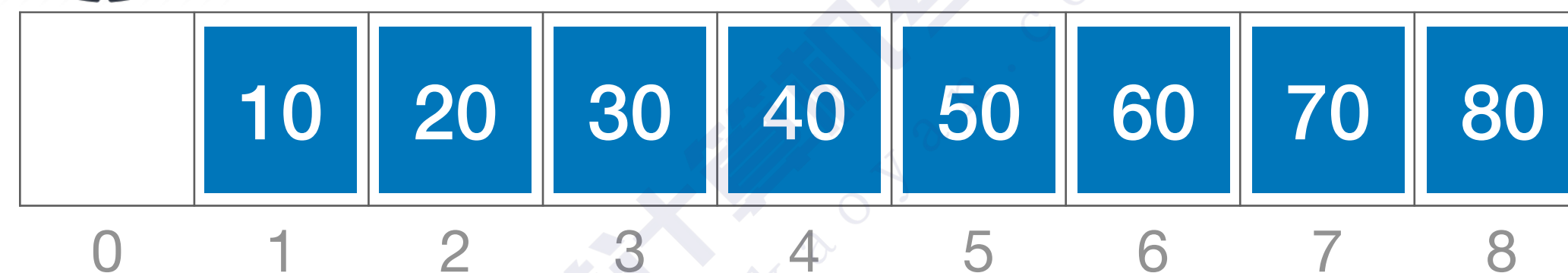
//复制为哨兵，A[0]不存放元素

//向后挪位

//复制到插入位置



最好情况：原本就有序



最好情况：

共n-1趟处理，每一趟只需要对比关键字1次，不用移动元素

最好时间复杂度—— $O(n)$

算法效率分析

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];
```

```
            A[j+1]=A[0];
```

```
        }
```

```
    }
```

//依次将 $A[2] \sim A[n]$ 插入到前面已排序序列

//若 $A[i]$ 关键码小于其前驱，将 $A[i]$ 插入有序表

//复制为哨兵， $A[0]$ 不存放元素

//向后挪位

//复制到插入位置

最坏情况：

第1趟：对比关键字2次，移动元素3次

第2趟：对比关键字3次，移动元素4次

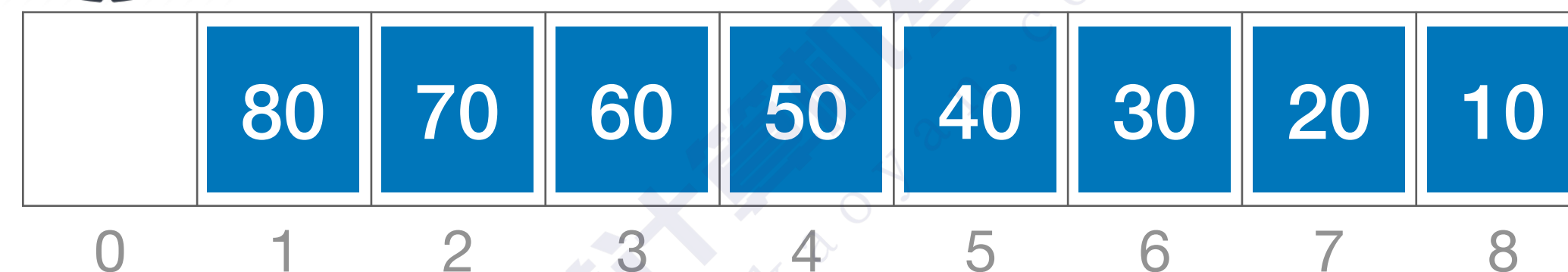
...

第 i 趟：对比关键字 $i+1$ 次，移动元素 $i+2$ 次

...



最坏情况：原本为逆序



算法效率分析

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];
```

```
            A[j+1]=A[0];
```

```
        }
```

```
    }
```

//依次将A[2]~A[n]插入到前面已排序序列

//若A[i]关键码小于其前驱，将A[i]插入有序表

//复制为哨兵，A[0]不存放元素

//向后挪位

//复制到插入位置

最坏情况：

第1趟：对比关键字2次，移动元素3次

第2趟：对比关键字3次，移动元素4次

第 i 趟：对比关键字 i+1次，移动元素 i+2 次

...

第n-1趟：对比关键字 n 次，移动元素 n+1 次

最坏情况：原本为逆序



最坏时间复杂度—— $O(n^2)$

算法效率分析

//直接插入排序（带哨兵）

```
void InsertSort(int A[],int n){
```

```
    int i,j;
```

```
    for(i=2;i<=n;i++)
```

```
        if(A[i]<A[i-1]){
```

```
            A[0]=A[i];
```

```
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
```

```
                A[j+1]=A[j];
```

```
            A[j+1]=A[0];
```

```
        }
```

```
    }
```

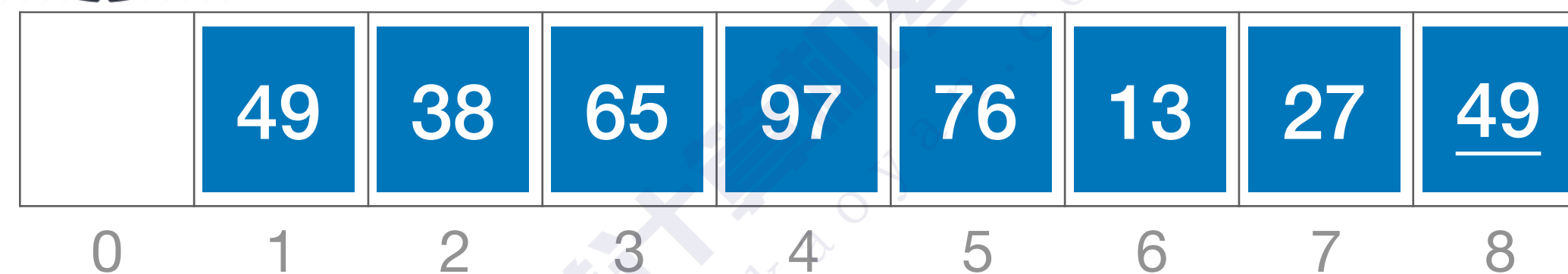
//依次将A[2]~A[n]插入到前面已排序序列

//若A[i]关键码小于其前驱，将A[i]插入有序表

//复制为哨兵，A[0]不存放元素

//向后挪位

//复制到插入位置



空间复杂度：O(1)

最好时间复杂度（全部有序）：O(n)

最坏时间复杂度（全部逆序）：O(n²)

平均时间复杂度：O(n²)

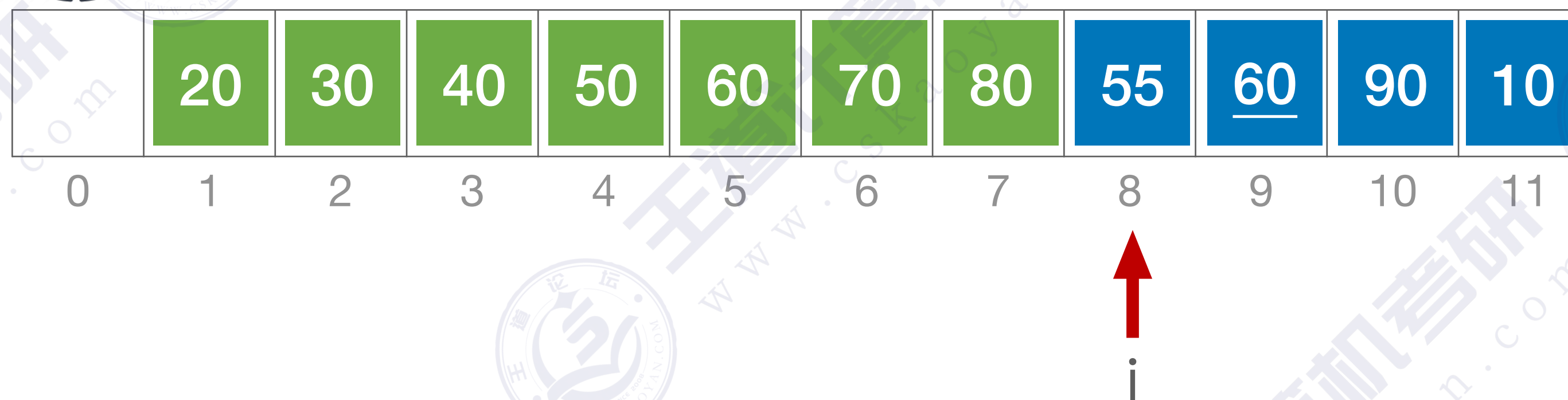
算法稳定性：稳定 老哥稳



优化——折半插入排序



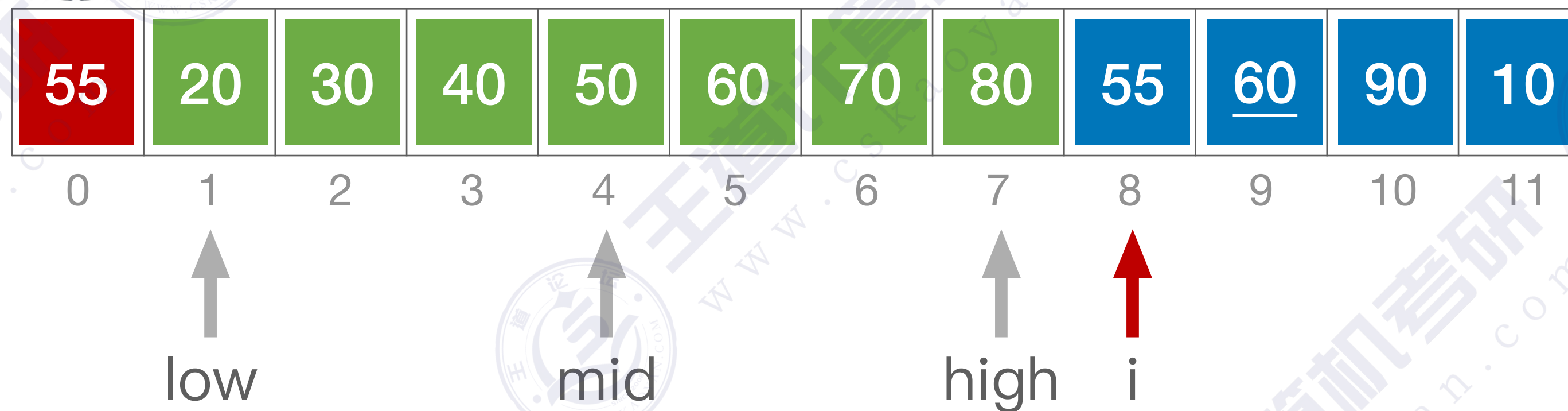
思路：先用折半查找找到应该插入的位置，再移动元素



优化——折半插入排序



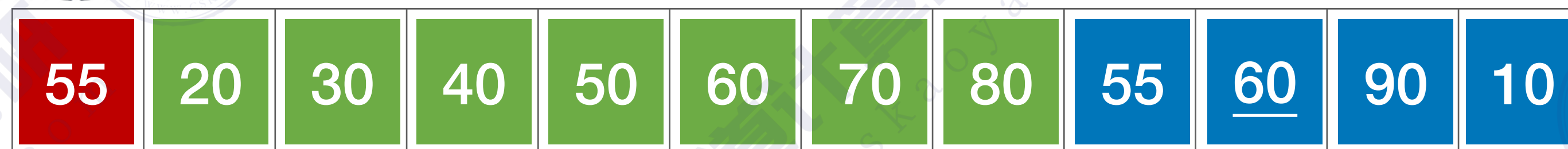
思路：先用折半查找找到应该插入的位置，再移动元素



优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



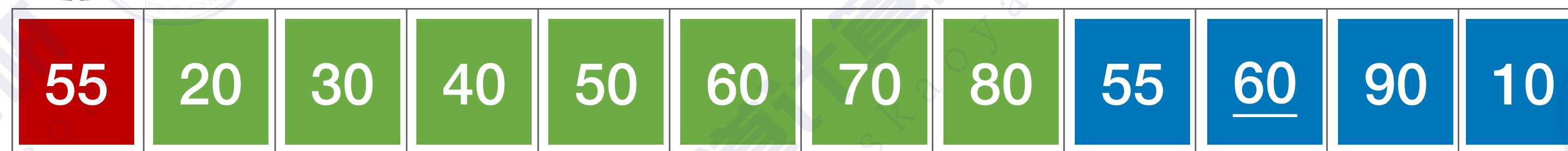
0 1 2 3 4 5 6 7 8 9 10 11

↑ ↑ ↑ ↑
low mid high i

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



0

1

2

3

4

5

6

7

8

9

10

11

low

high

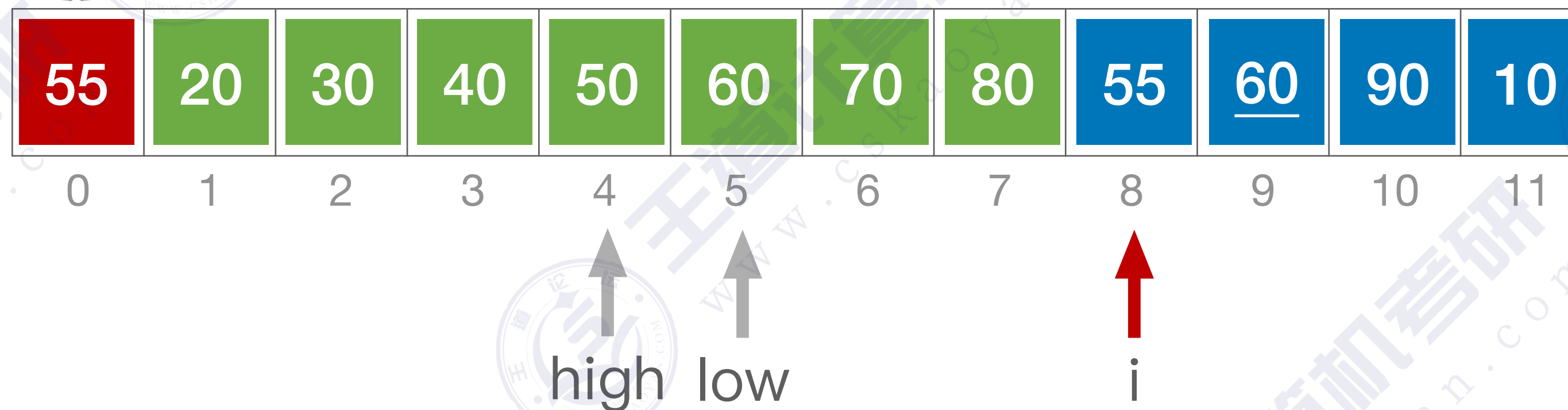
mid

i

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

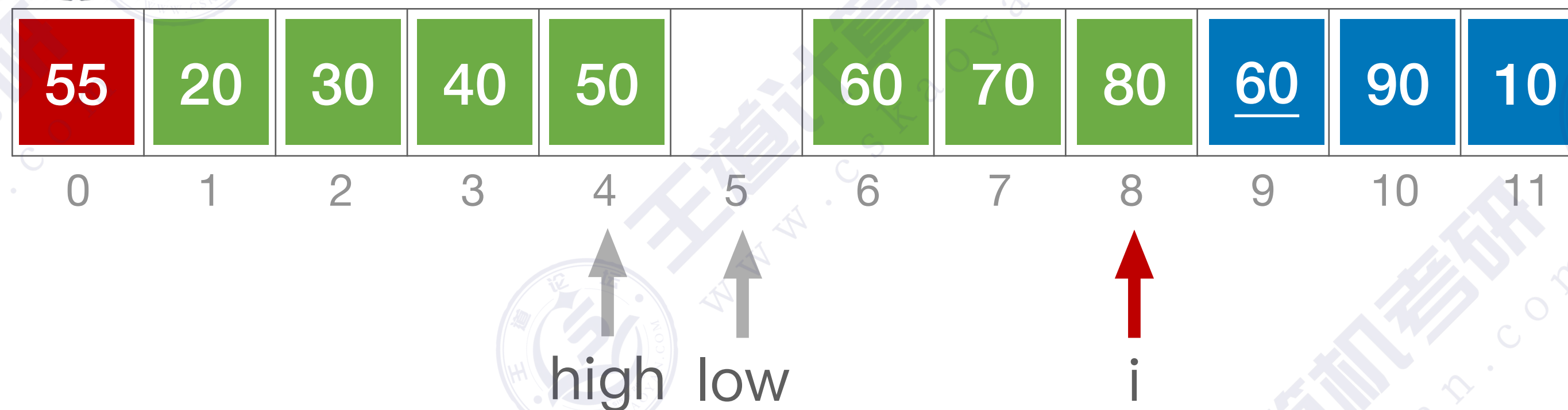


当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

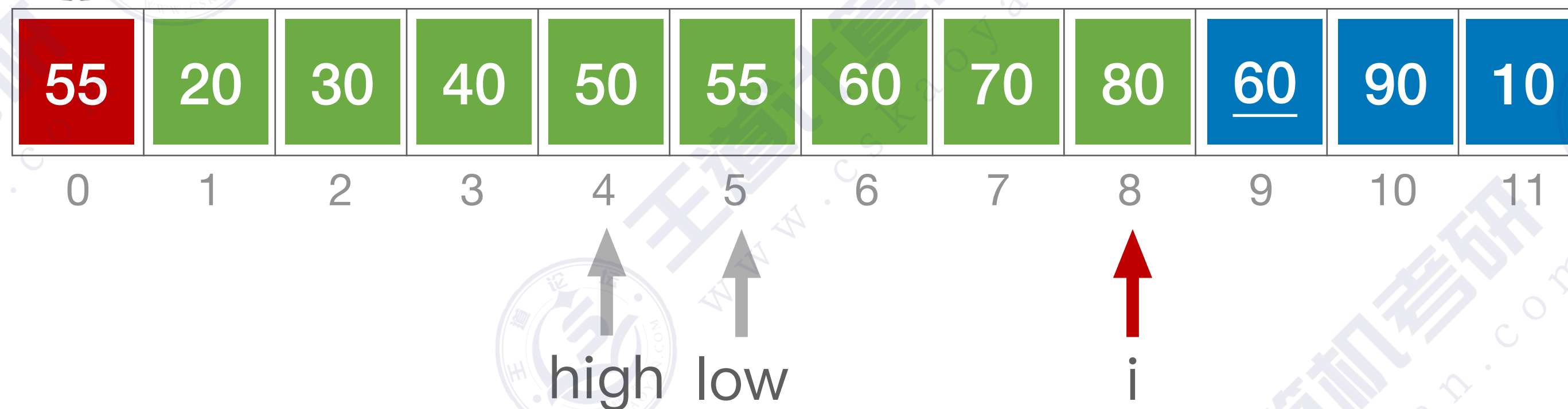


当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

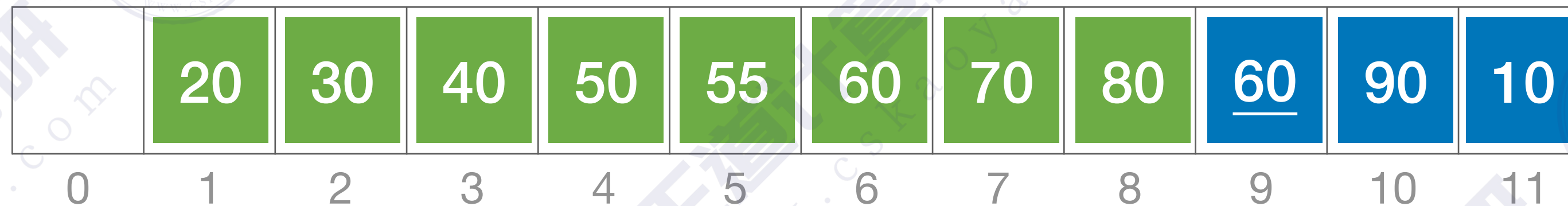


当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

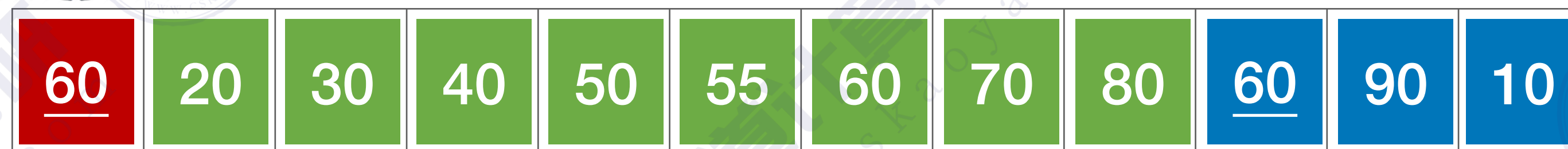


i

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



0

1

2

3

4

5

6

7

8

9

10

11

↑
low

↑
mid

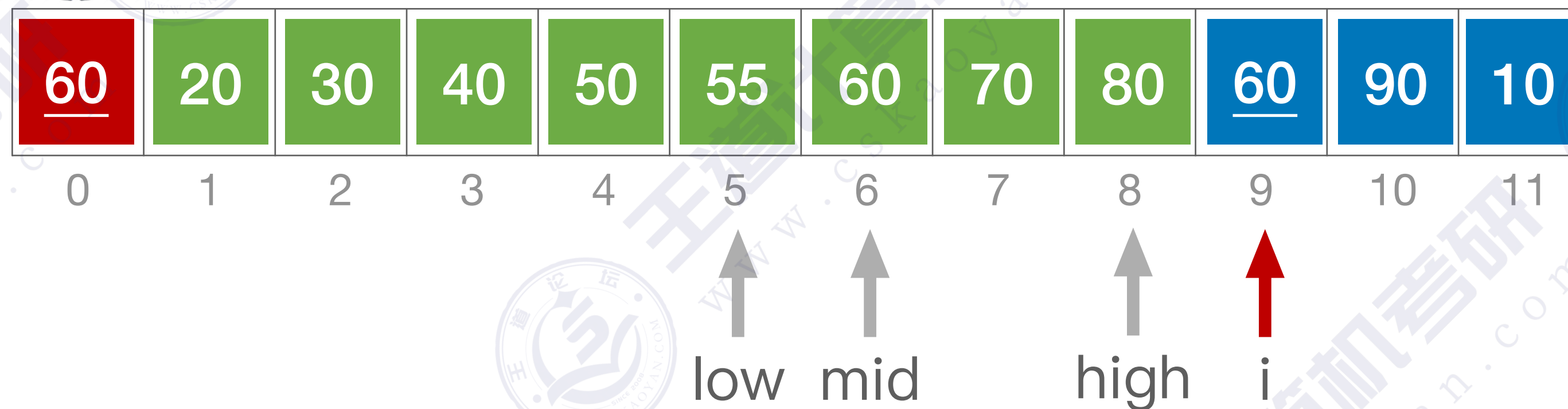
↑
high

↑
i

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

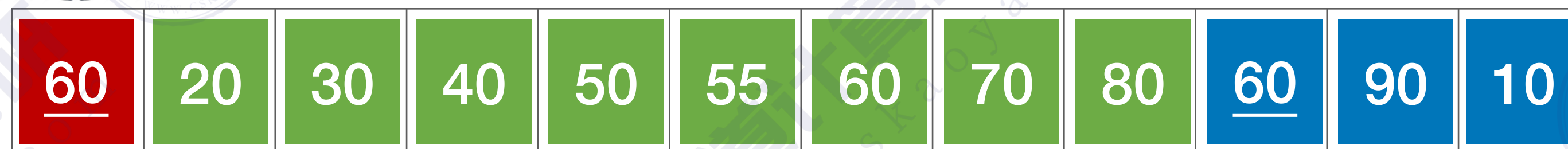


当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



0 1 2 3 4 5 6 7 8 9 10 11

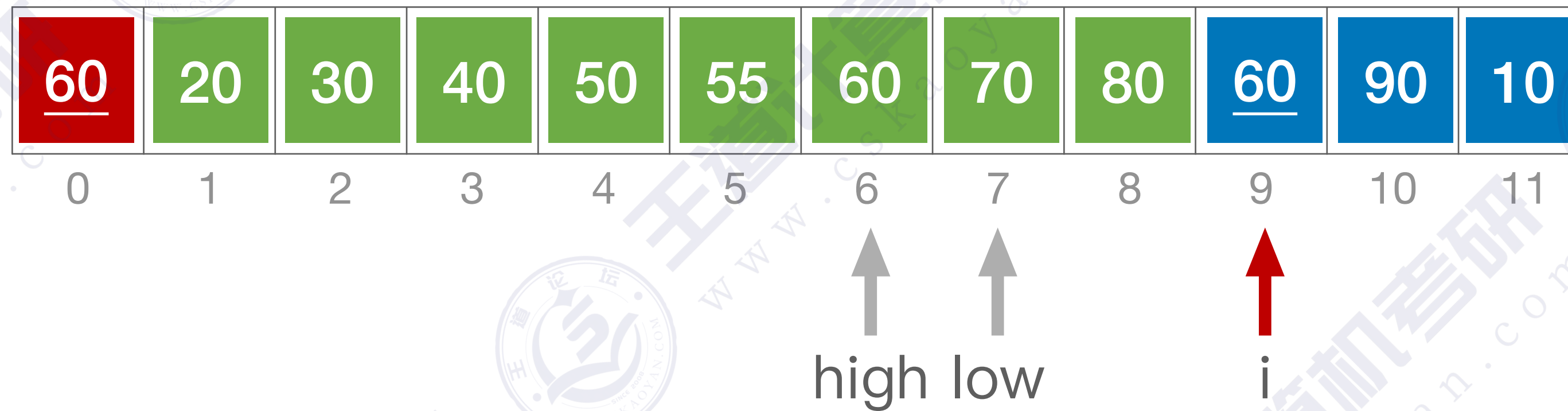
↑
low high i
↑
mid

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



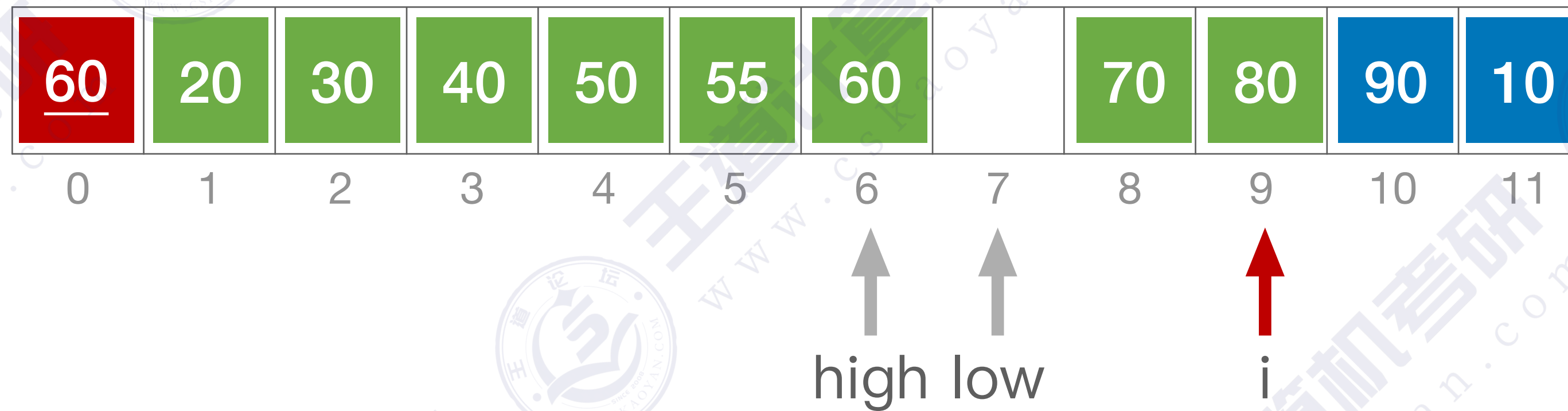
当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



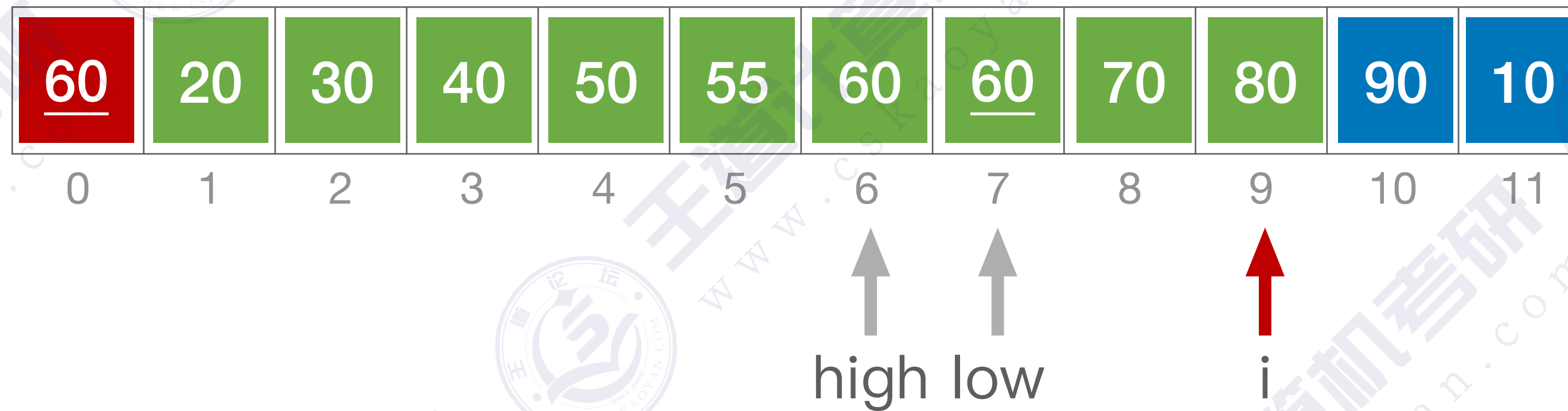
当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



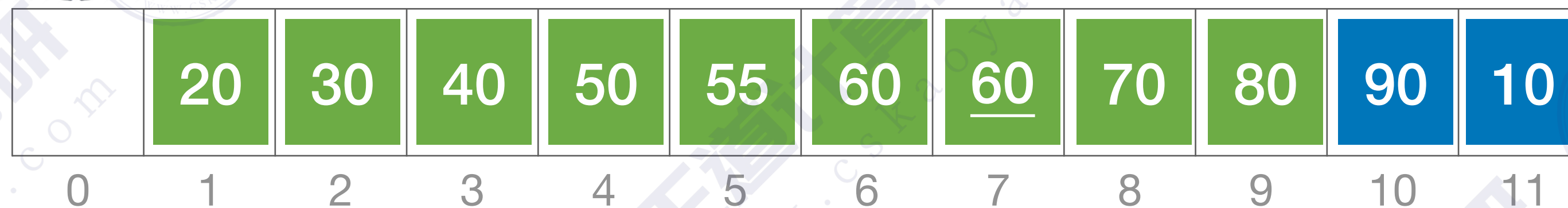
当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

优化——折半插入排序



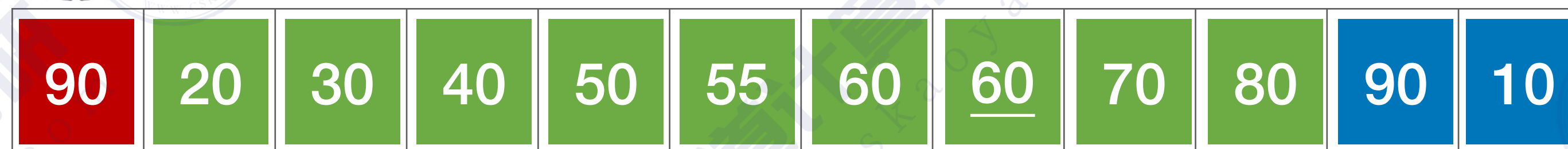
思路：先用折半查找找到应该插入的位置，再移动元素



优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



0

1

2

3

4

5

6

7

8

9

10

11

↑
low

↑
mid

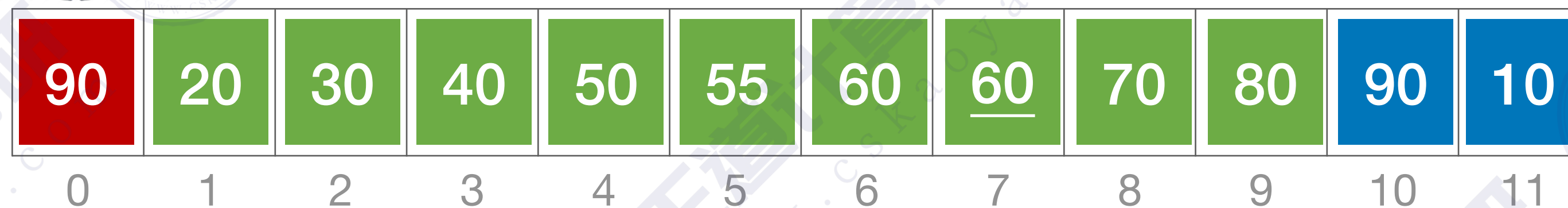
↑
high

↑
i

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



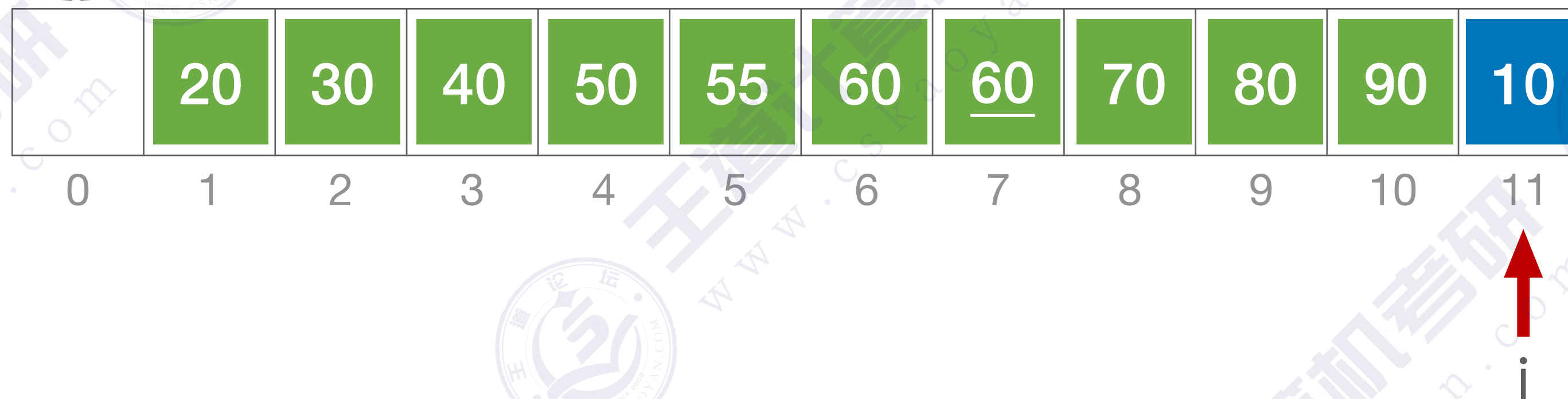
↑ high
↑ i
↑ low

当 $low > high$ 时折半查找停止，应将 ~~$[low, i-1]$~~ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



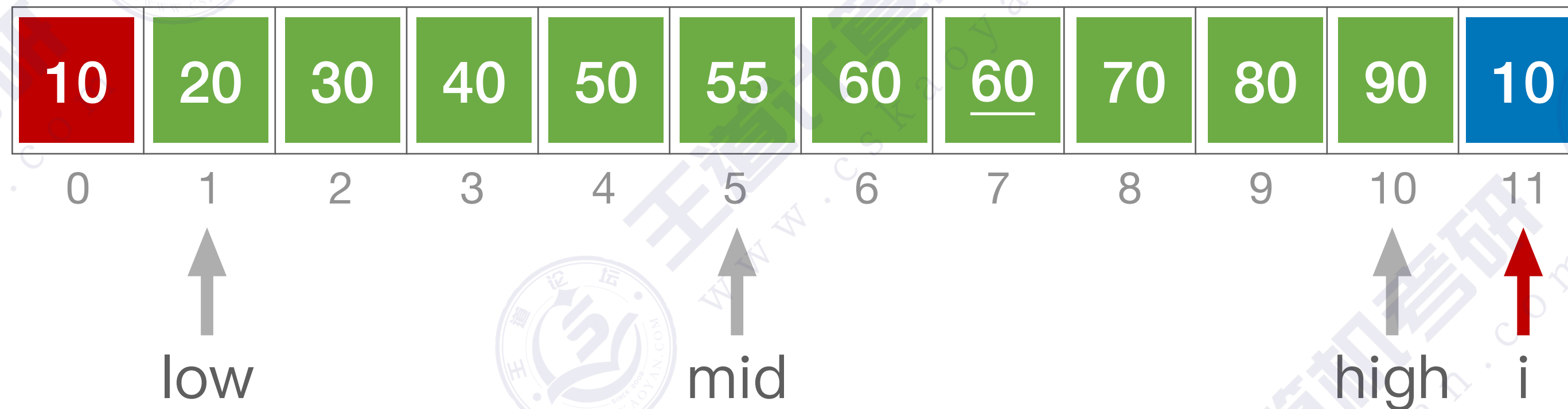
思路：先用折半查找找到应该插入的位置，再移动元素



优化——折半插入排序



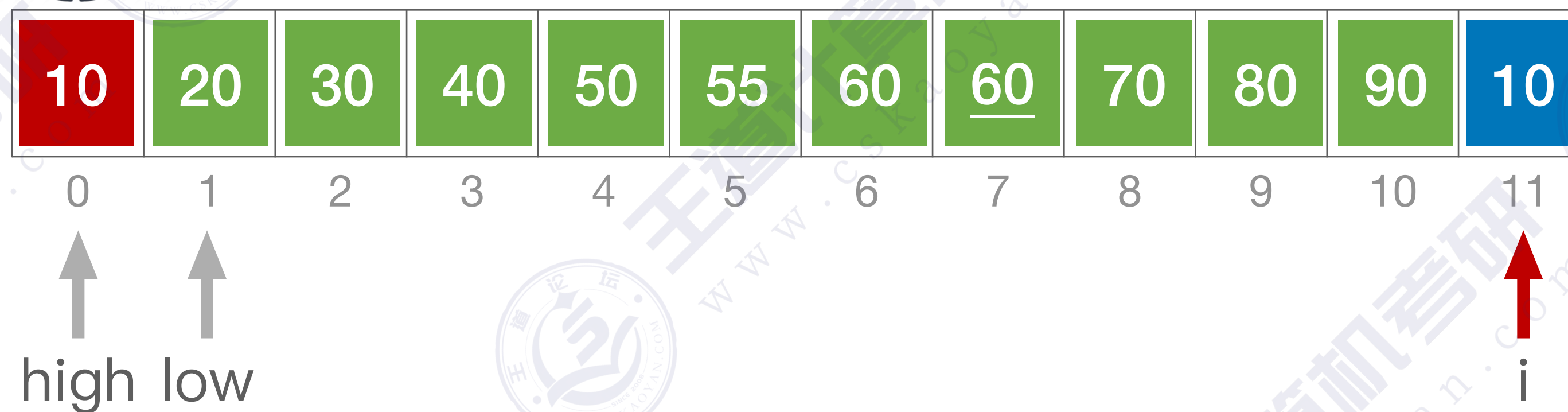
思路：先用折半查找找到应该插入的位置，再移动元素



优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

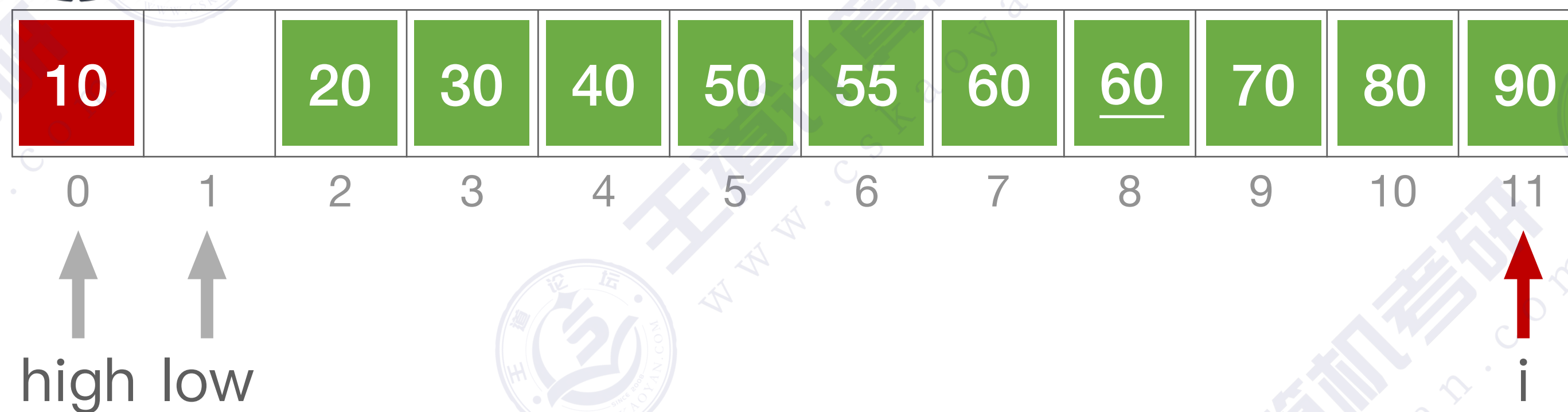


当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素

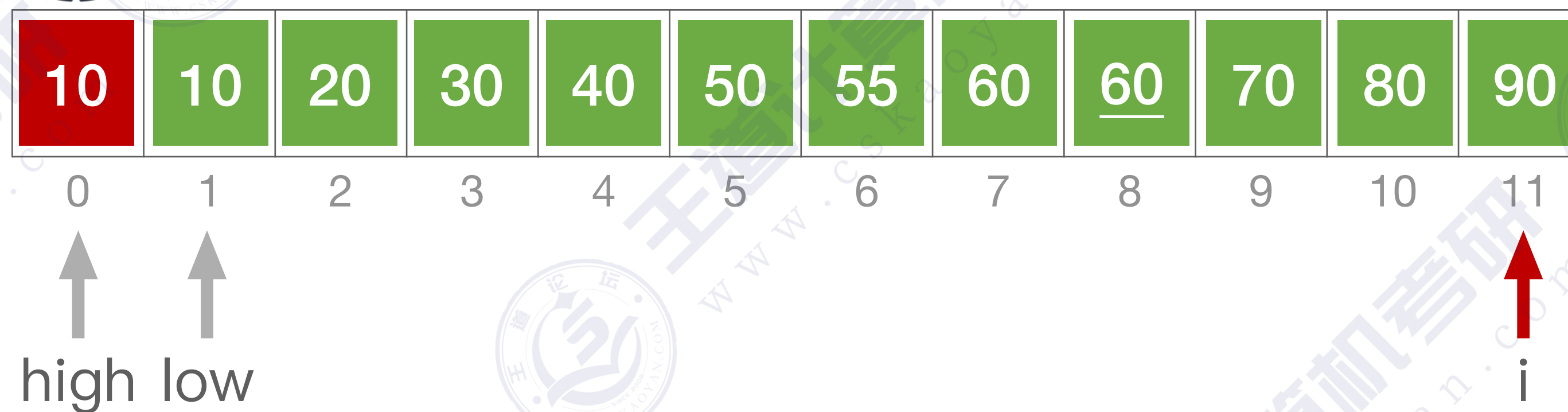


当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序



思路：先用折半查找找到应该插入的位置，再移动元素



	10	20	30	40	50	55	60	<u>60</u>	70	80	90
0	1	2	3	4	5	6	7	8	9	10	11

当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

优化——折半插入排序

//折半插入排序

```
void InsertSort(int A[], int n){
    int i, j, low, high, mid;
    for(i=2; i<=n; i++){
        A[0]=A[i];           //依次将A[2]~A[n]插入前面的已排序序列
        low=1; high=i-1;     //将A[i]暂存到A[0]
        while(low<=high){    //设置折半查找的范围
            mid=(low+high)/2; //折半查找(默认递增有序)
            if(A[mid]>A[0]) high=mid-1; //取中间点
            else low=mid+1;         //查找左半子表
        }
        for(j=i-1; j>=high+1; --j) //查找右半子表
            A[j+1]=A[j];           //统一后移元素, 空出插入位置
        A[high+1]=A[0];           //插入操作
    }
}
```

这操作也就那样吧

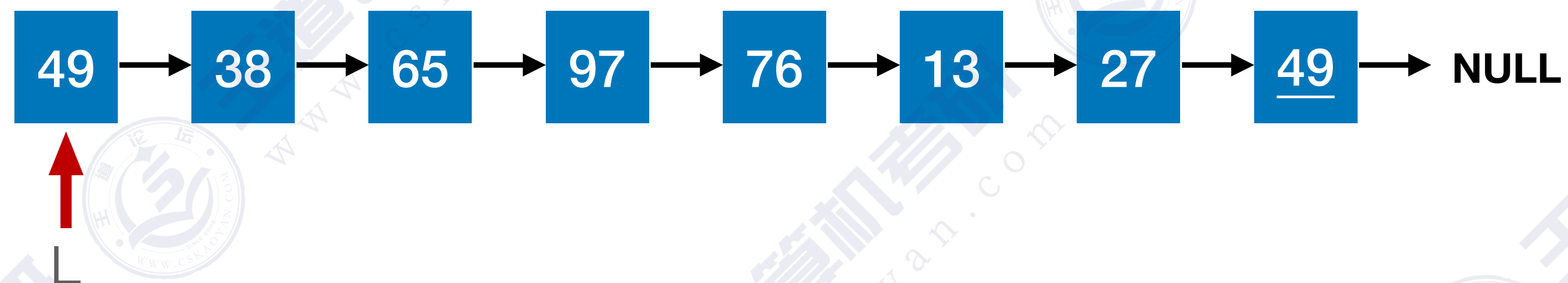


比起“直接插入排序”，比较关键字的次数减少了，但是移动元素的次数没变，整体来看**时间复杂度依然是 $O(n^2)$**

当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置

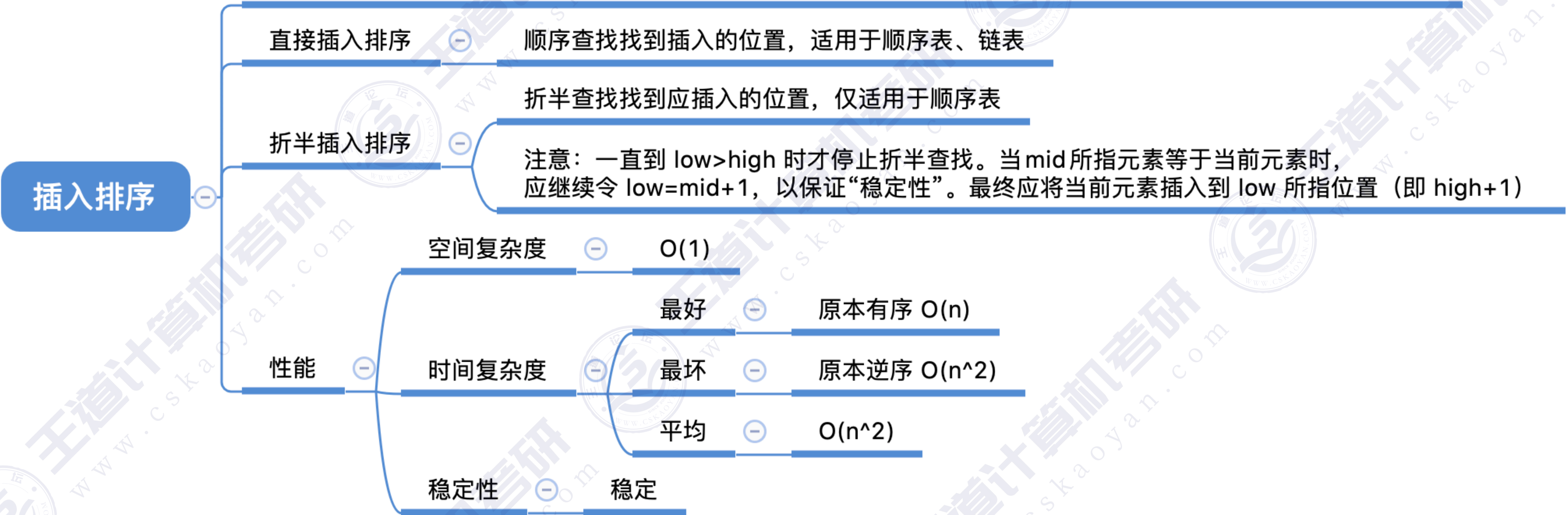
对链表进行插入排序



移动元素的次数变少了，但是关键字对比的次数依然是 $O(n^2)$ 数量级，
整体来看时间复杂度依然是 $O(n^2)$

知识回顾与重要考点

算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



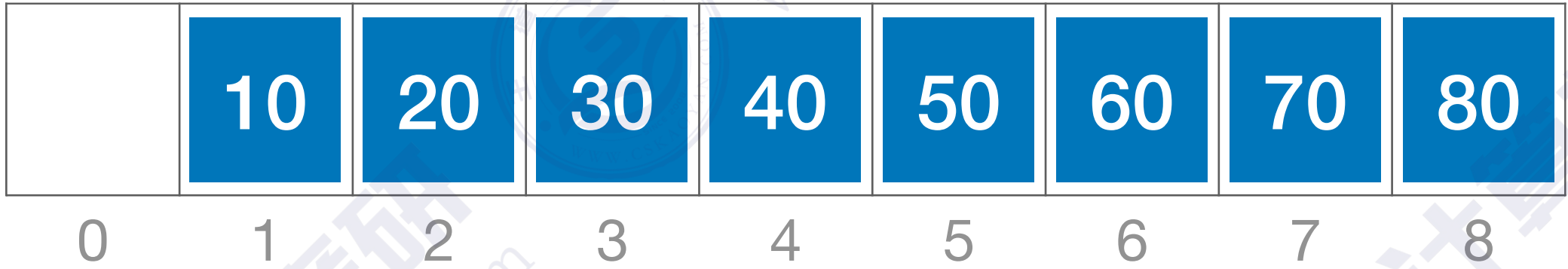
本节内容

希尔排序

希尔排序 (Shell Sort)



最好情况：原本就有序



比较好的情况：基本有序



希尔排序：先追求表中元素部分有序，再逐渐逼近全局有序

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

	49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7	8

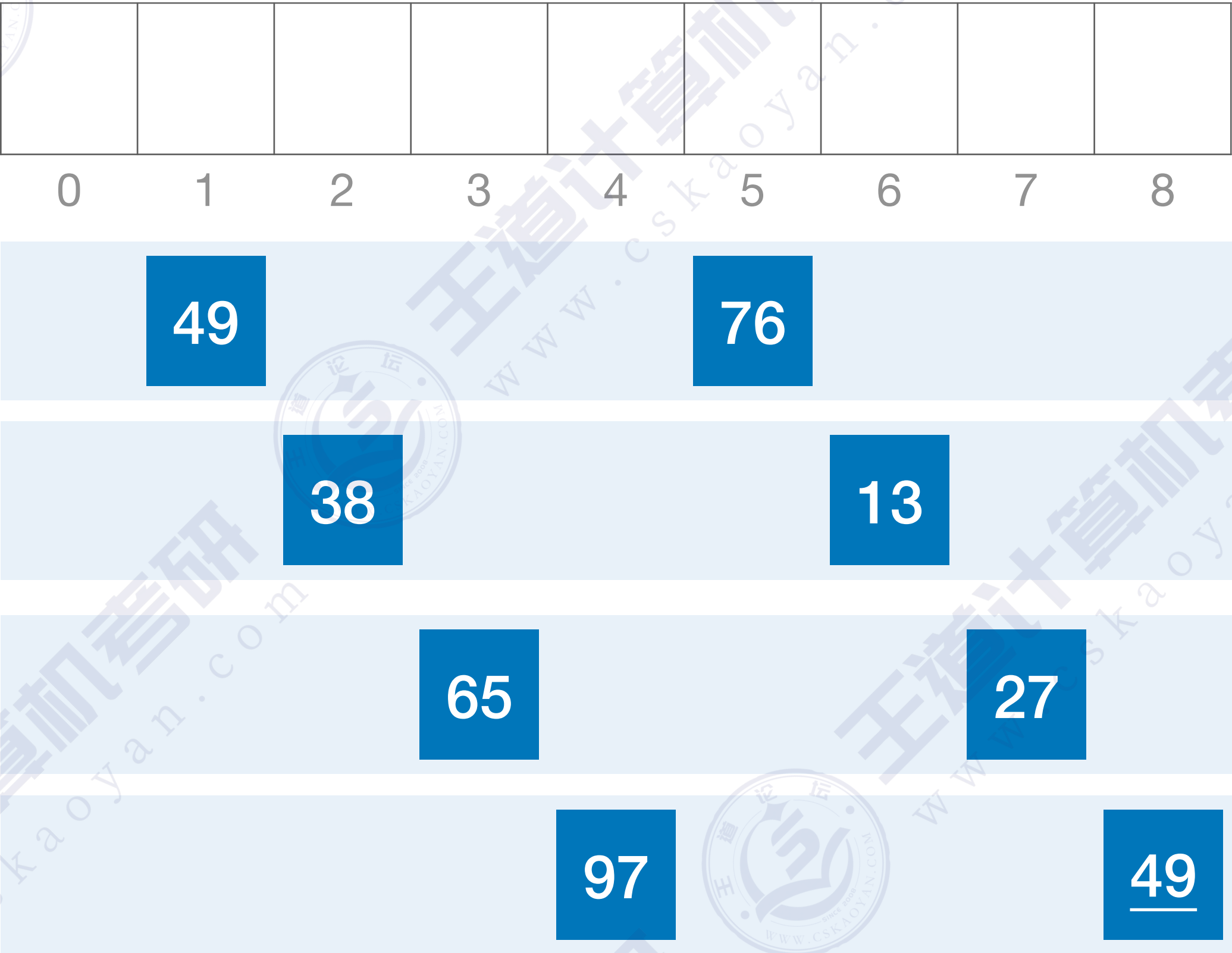
第一趟： $d_1=n/2=4$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=n/2=4$



子表1

子表2

子表3

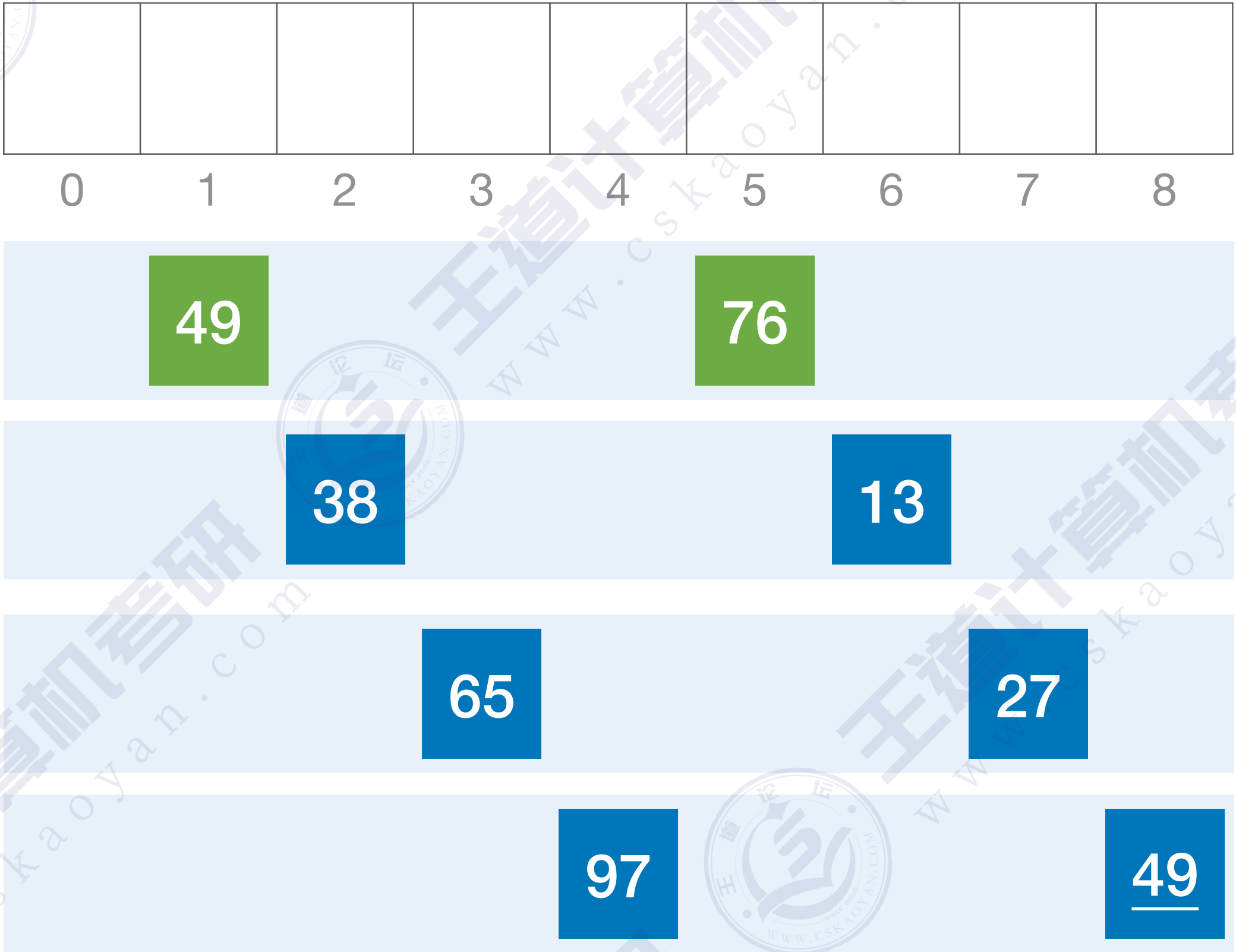
子表4

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=n/2=4$



子表1

子表2

子表3

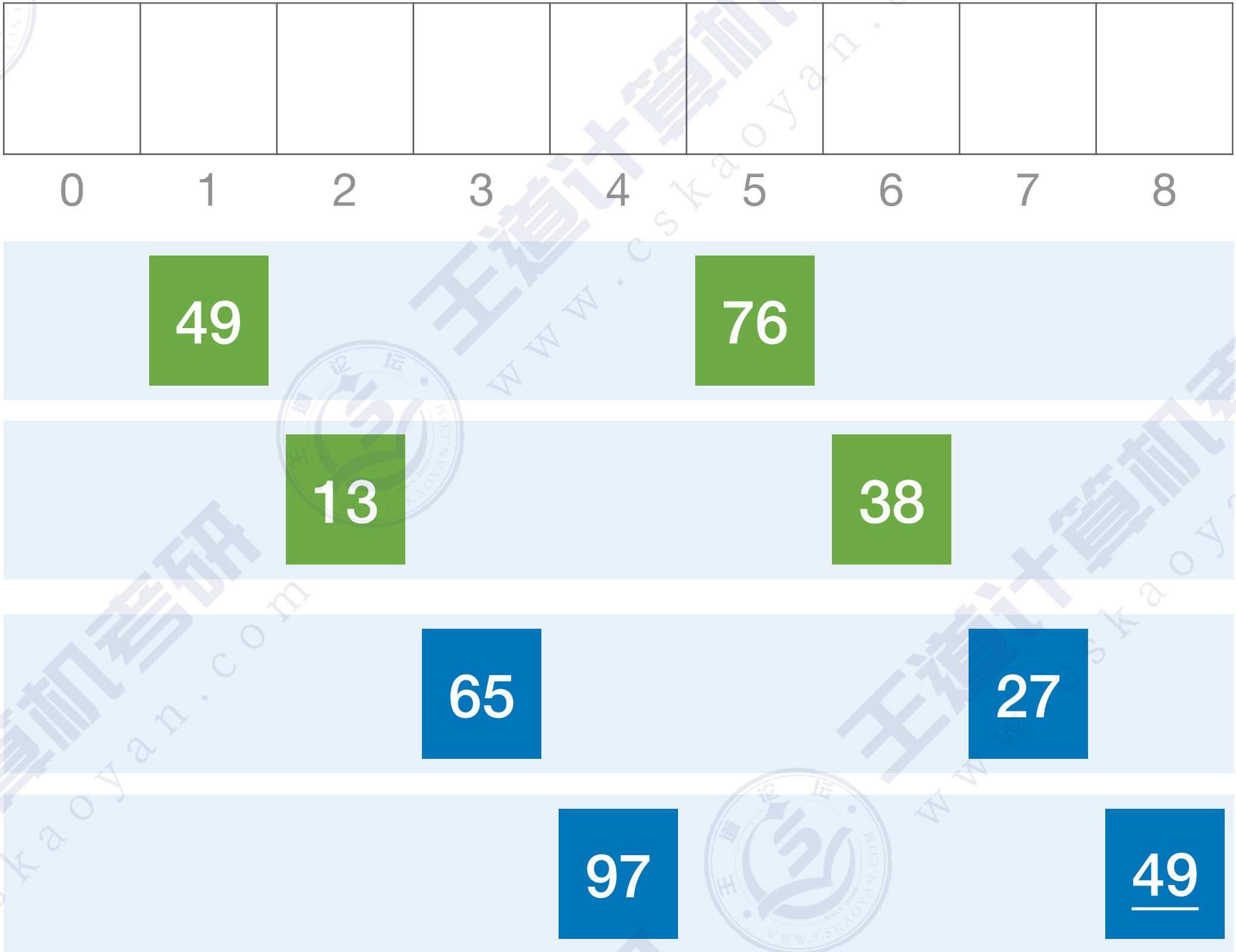
子表4

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=n/2=4$



子表1

子表2

子表3

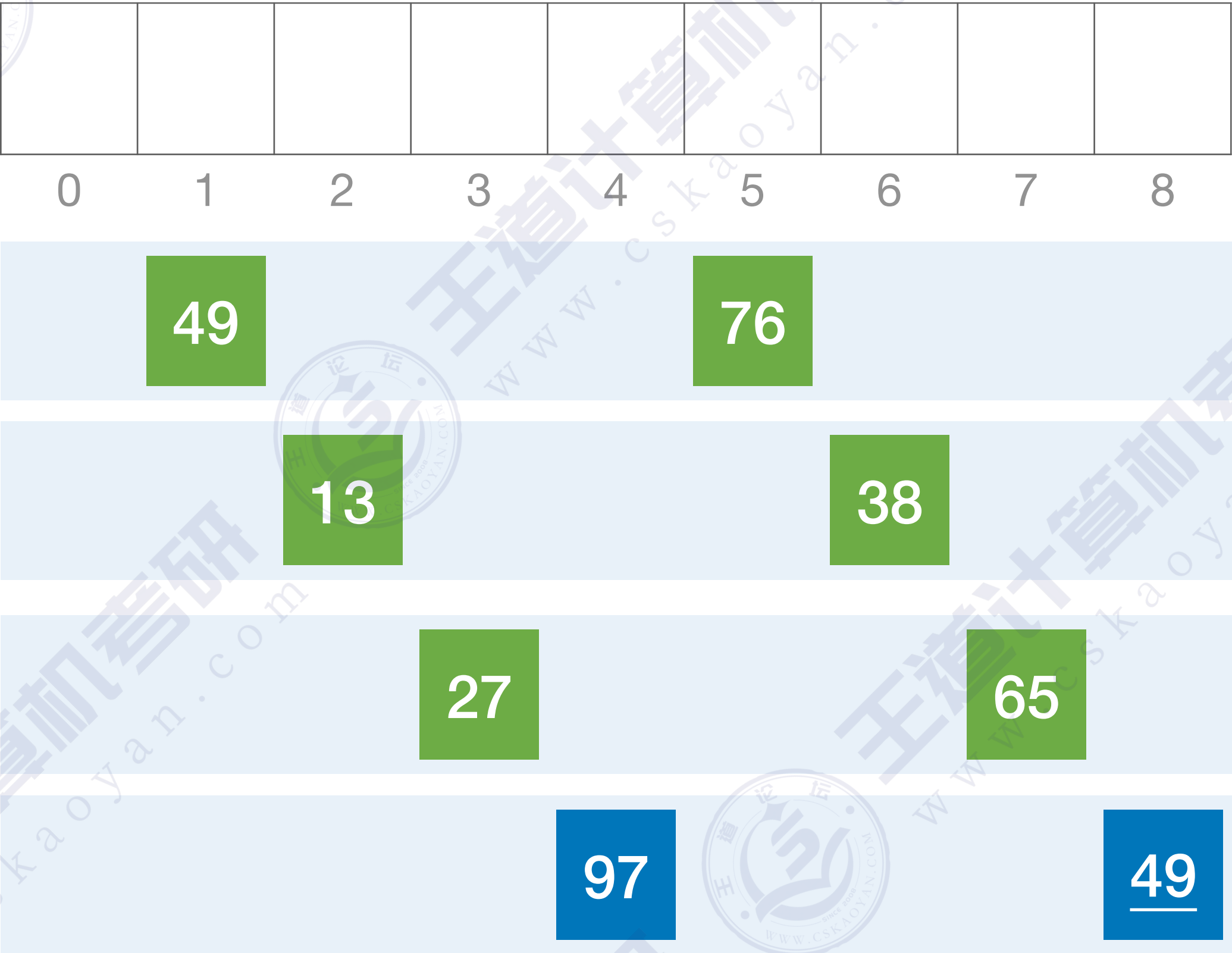
子表4

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=n/2=4$



子表1

子表2

子表3

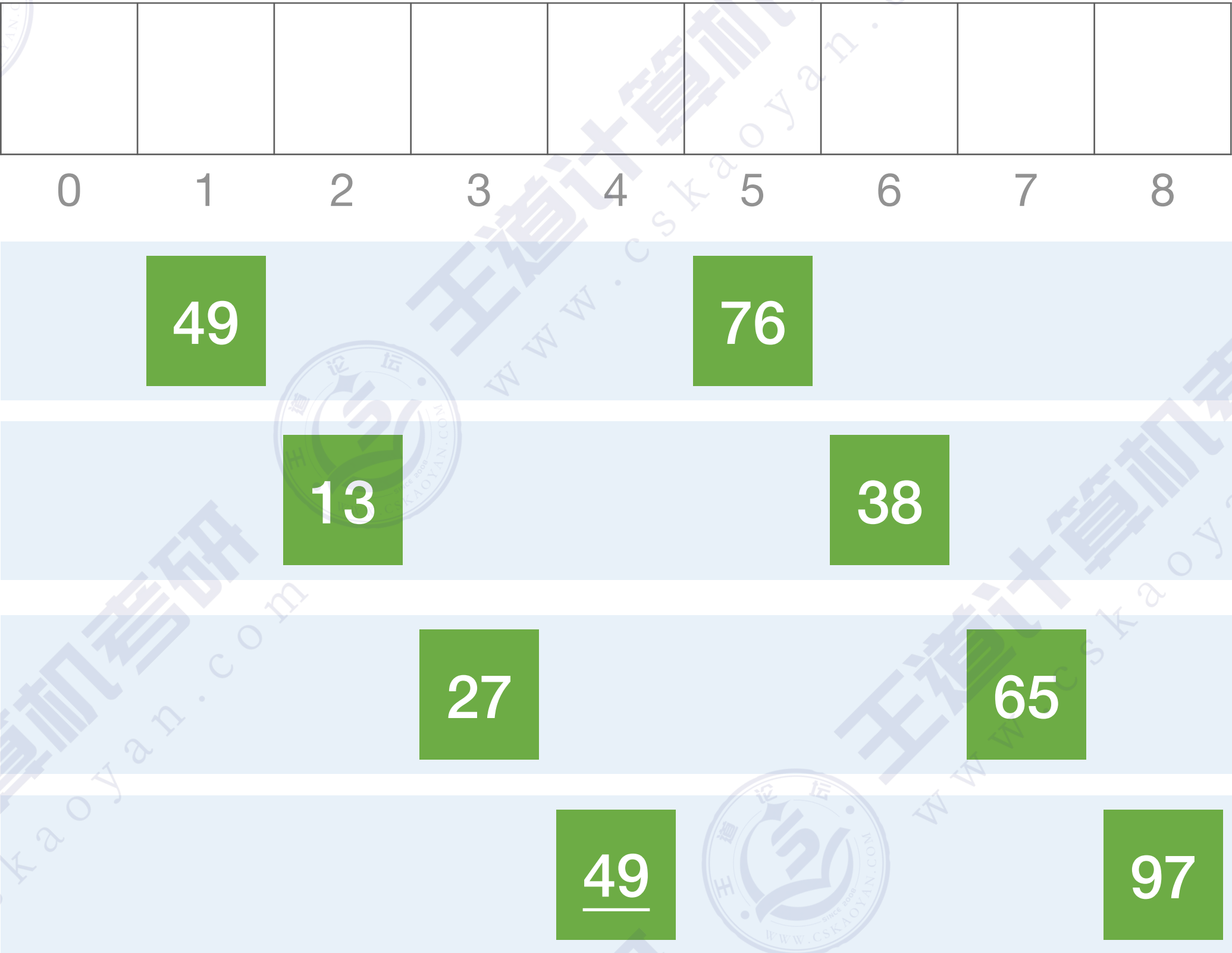
子表4

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=n/2=4$



子表1

子表2

子表3

子表4

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

	49	13	27	<u>49</u>	76	38	65	97
0	1	2	3	4	5	6	7	8

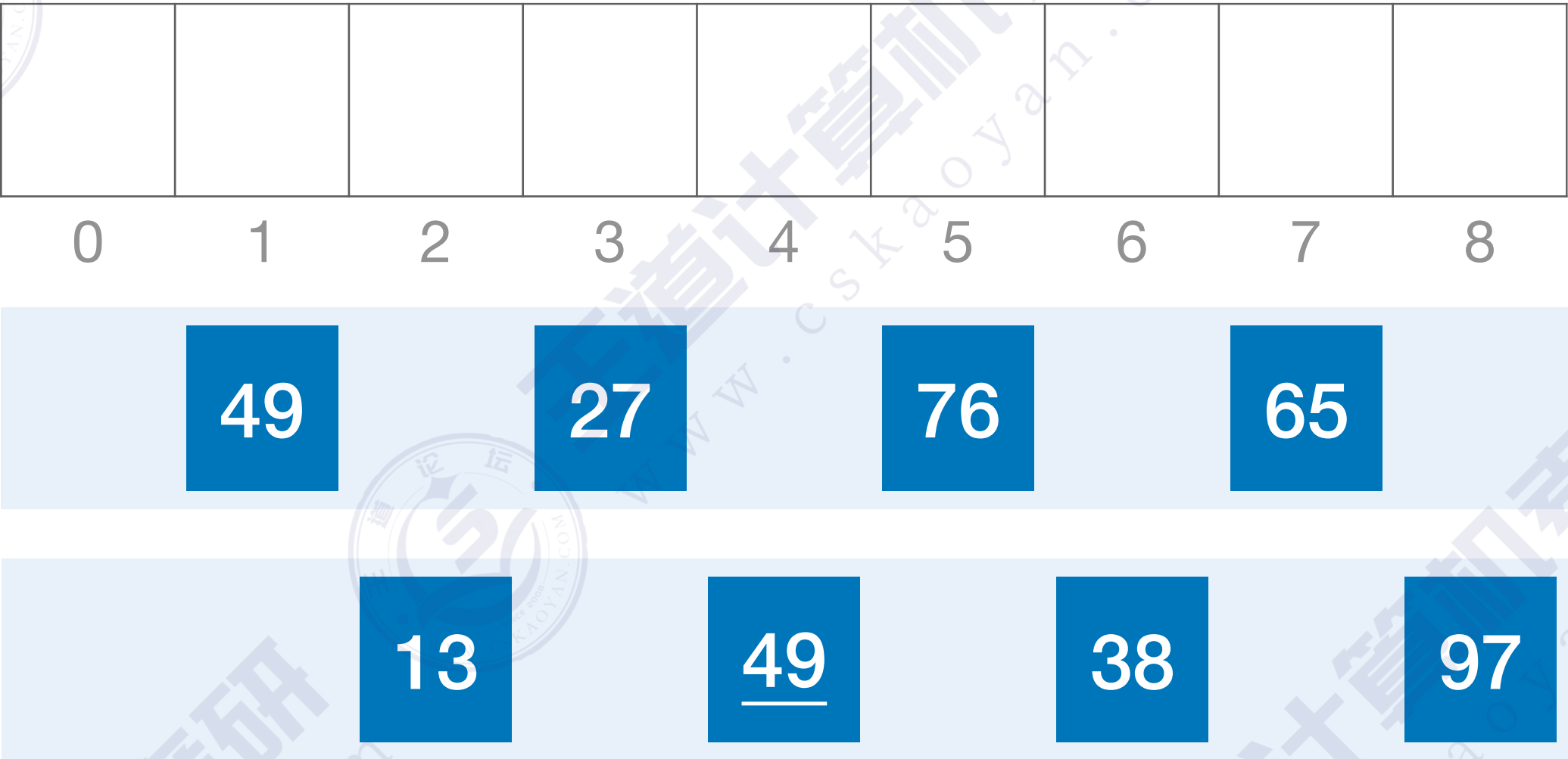
第二趟： $d_2=d_1/2=2$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第二趟： $d_2=d_1/2=2$



子表1

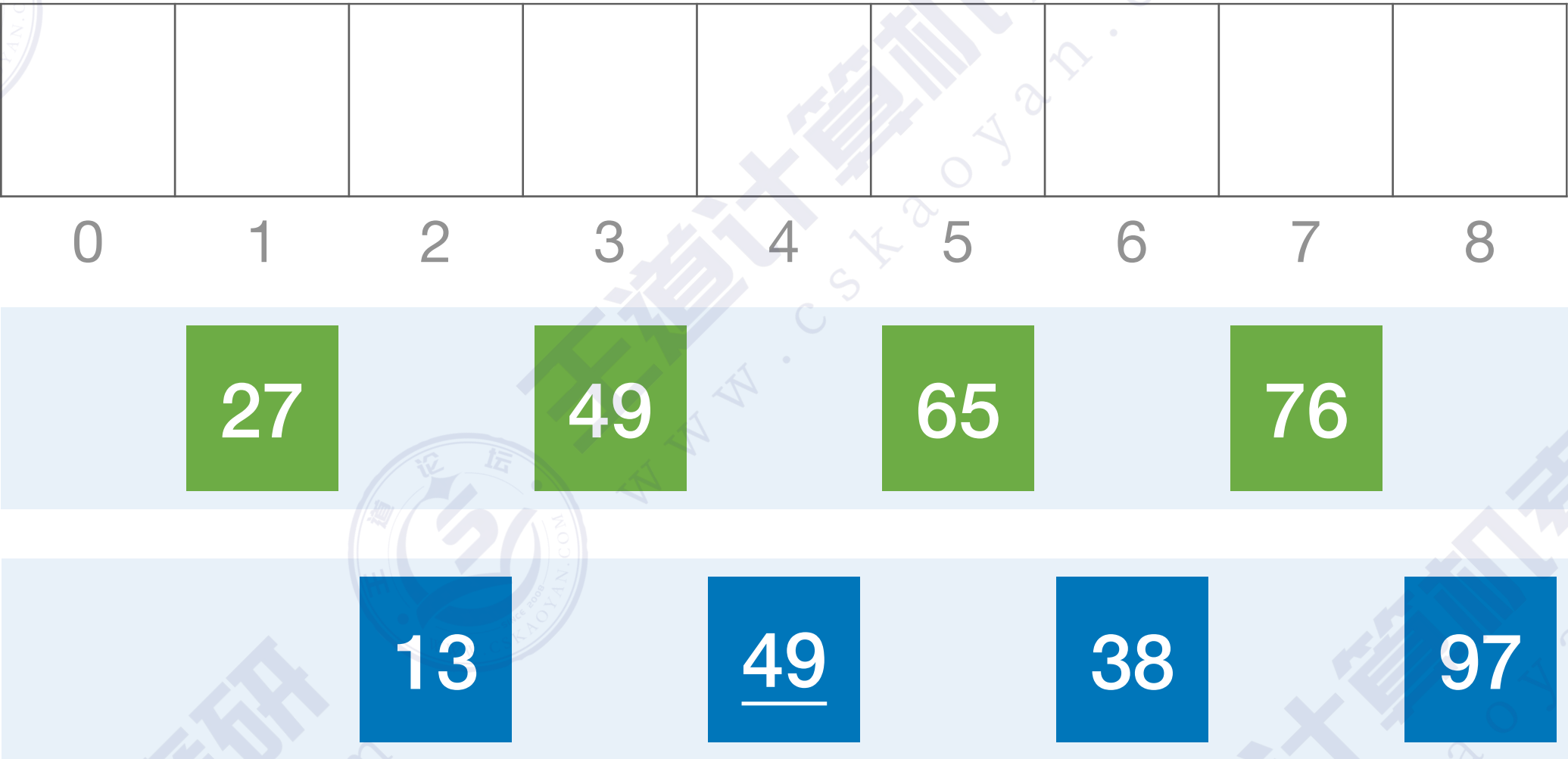
子表2

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第二趟： $d_2=d_1/2=2$



子表1

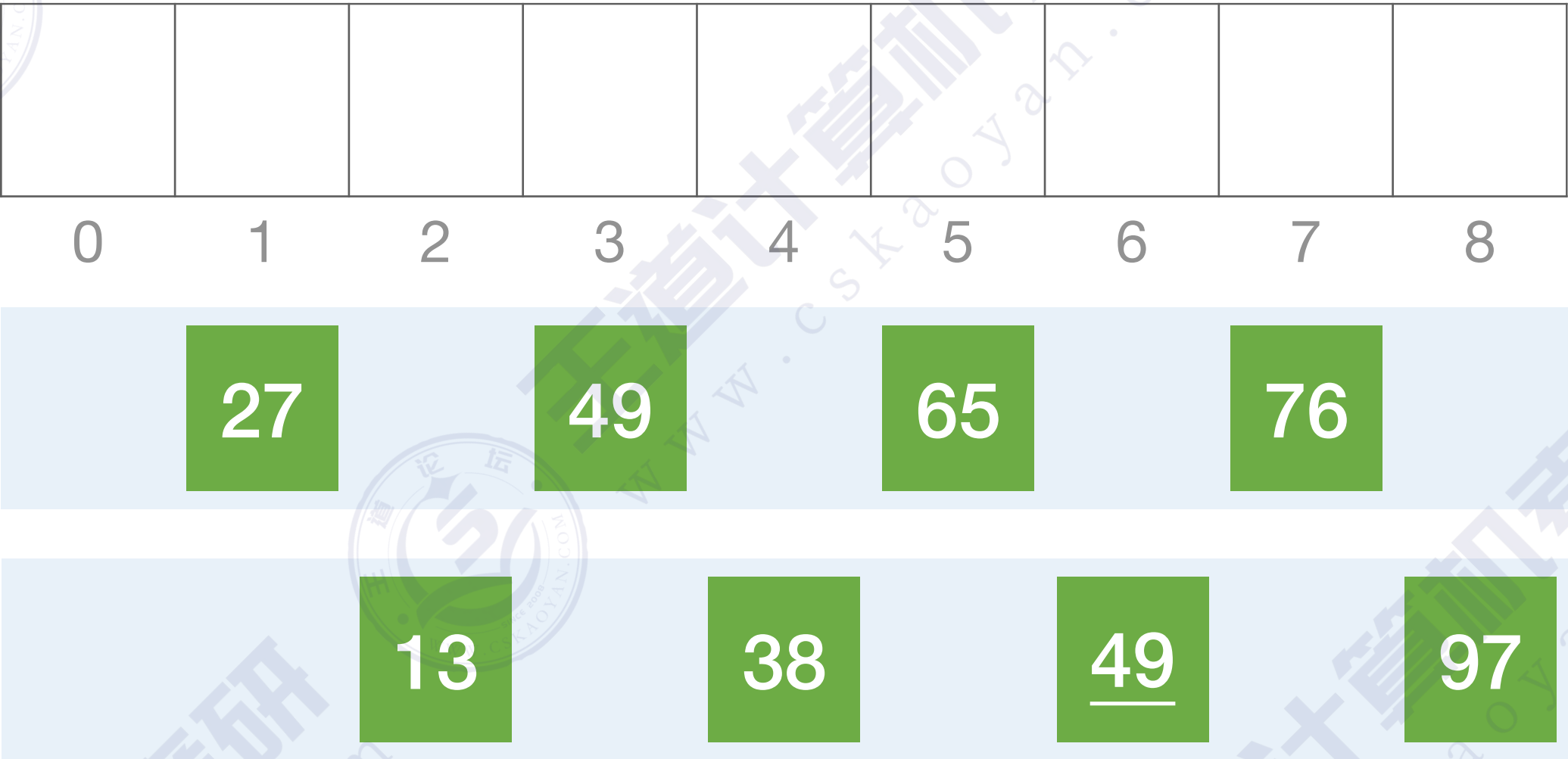
子表2

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第二趟： $d_2=d_1/2=2$



子表1

子表2

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

	27	13	49	38	65	<u>49</u>	76	97
0	1	2	3	4	5	6	7	8

第三趟： $d_3=d_2/2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

0	1	2	3	4	5	6	7	8

第三趟： $d_3=d_2/2=1$

27	13	49	38	65	<u>49</u>	76	97
----	----	----	----	----	-----------	----	----

子表1

整个表已呈现出“基本有序”，对整体再进行一次“直接插入排序”

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

0	1	2	3	4	5	6	7	8

第三趟： $d_3=d_2/2=1$

13	27	38	49	<u>49</u>	65	76	97
----	----	----	----	-----------	----	----	----

子表1

整个表已呈现出“基本有序”，对整体再进行一次“直接插入排序”

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量d，重复上述过程，直到d=1为止。

	13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7	8

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

希尔本人建议：每次将增量缩小一半

第一趟： $d_1=n/2=4$

	49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7	8

	49	13	27	<u>49</u>	76	38	65	97
0	1	2	3	4	5	6	7	8

第二趟： $d_2=d_1/2=2$

	27	13	49	38	65	<u>49</u>	76	97
0	1	2	3	4	5	6	7	8

第三趟： $d_3=d_2/2=1$

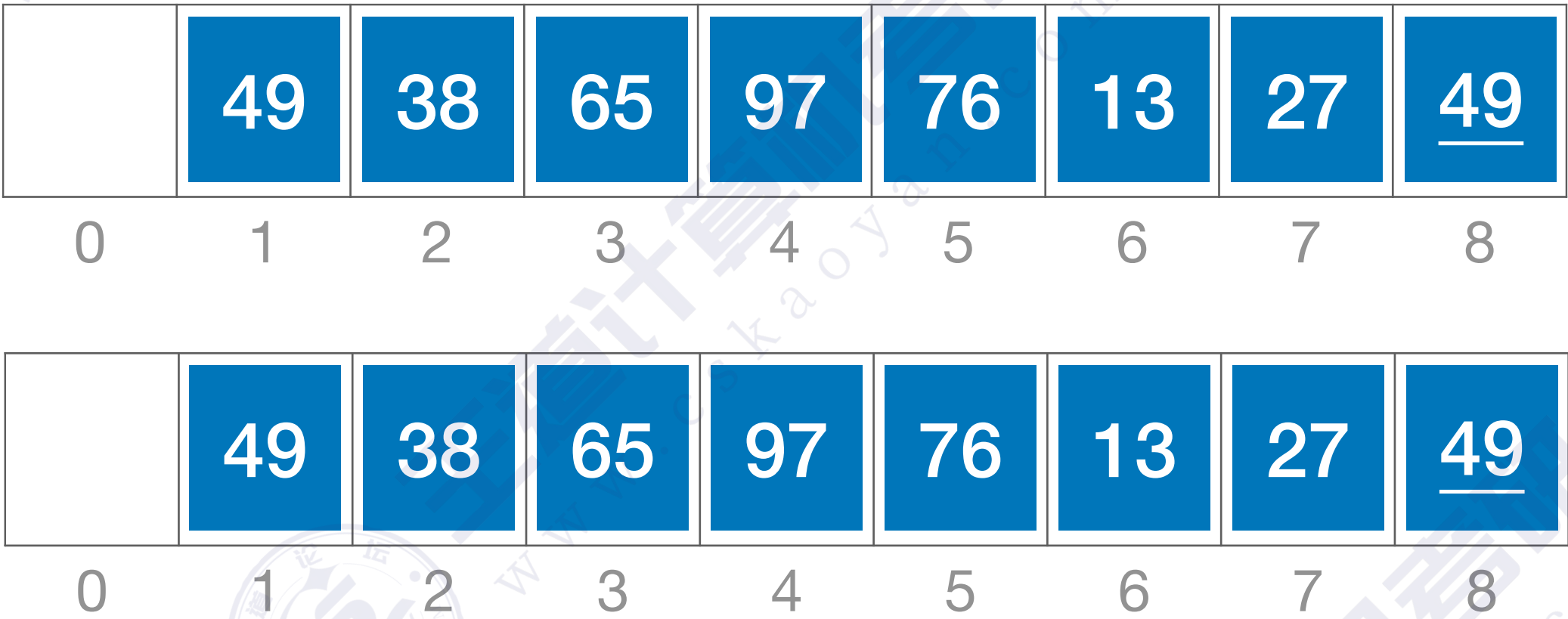
	13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7	8

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



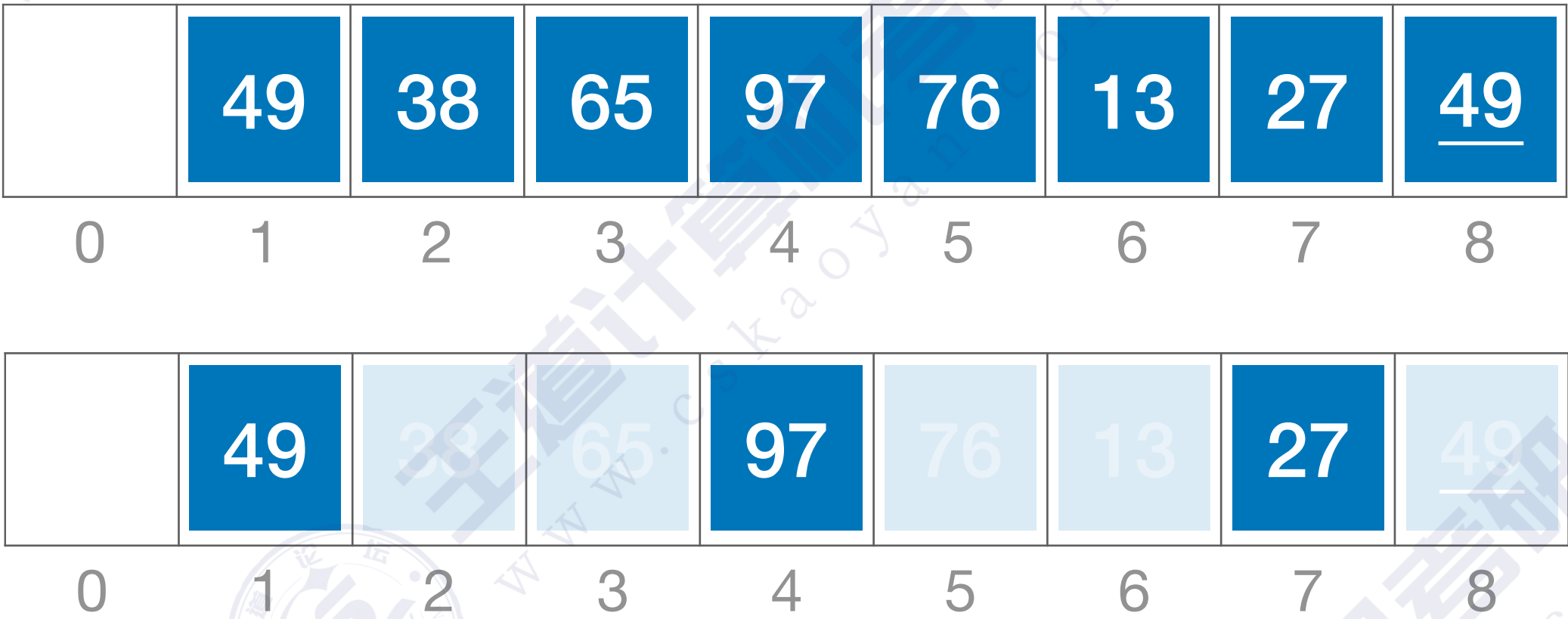
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



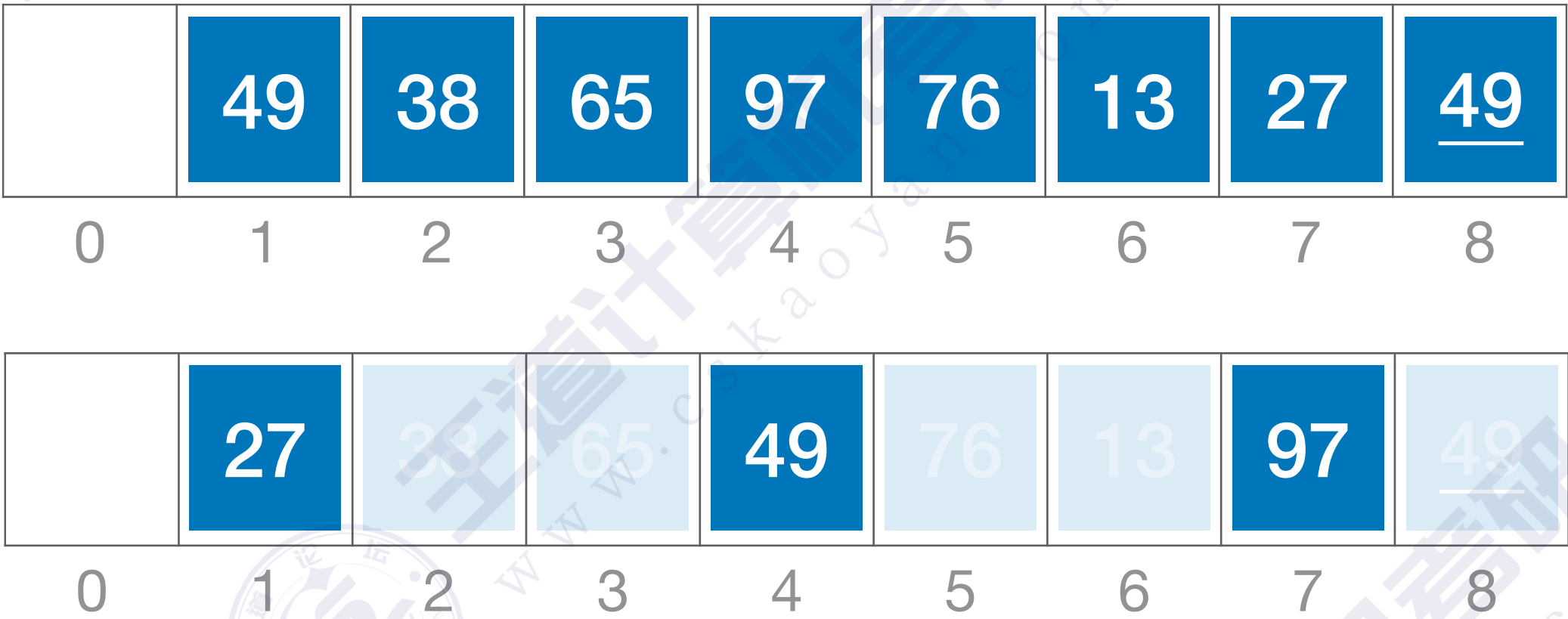
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



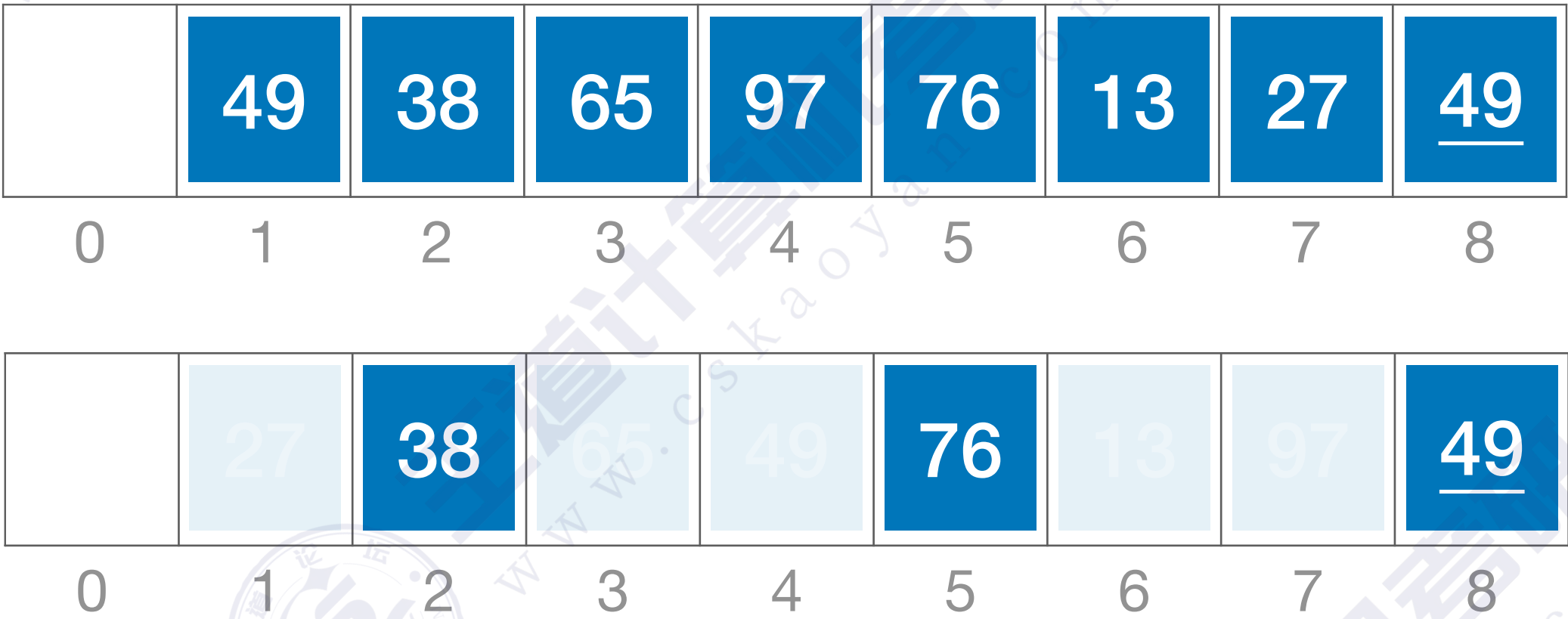
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



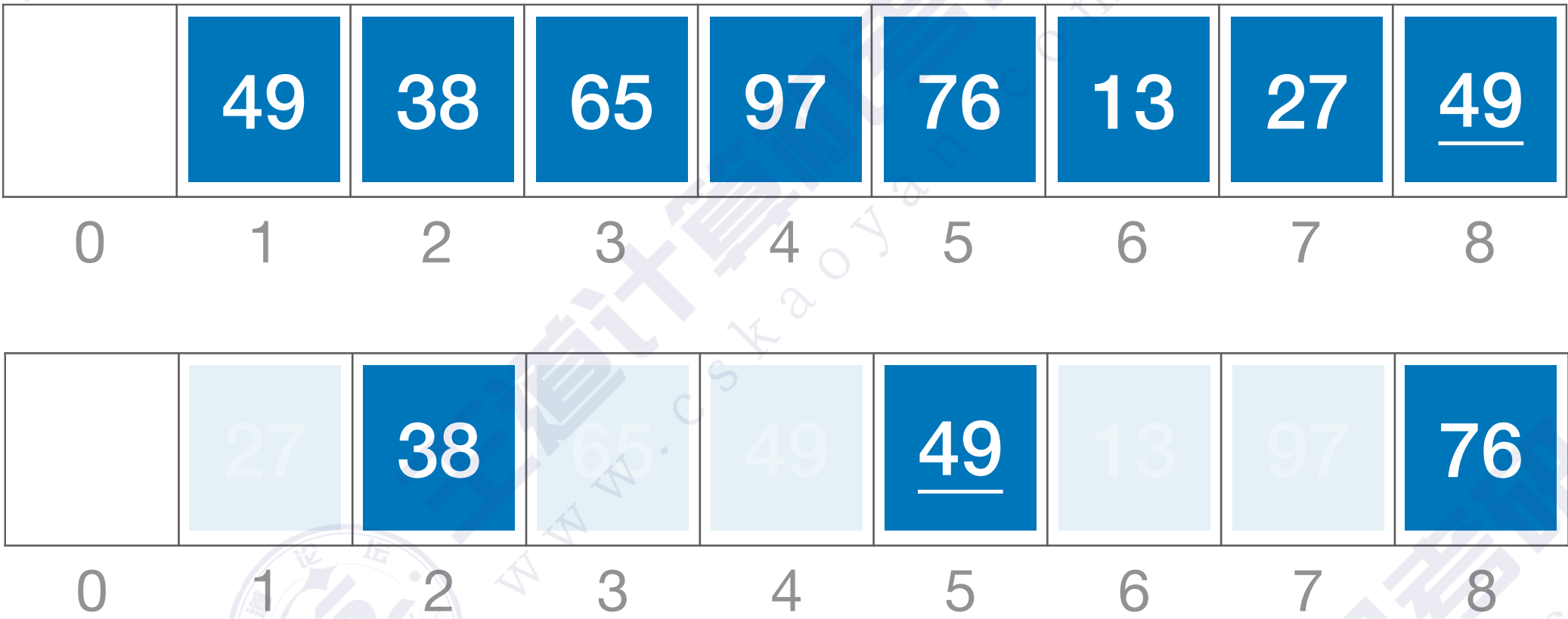
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



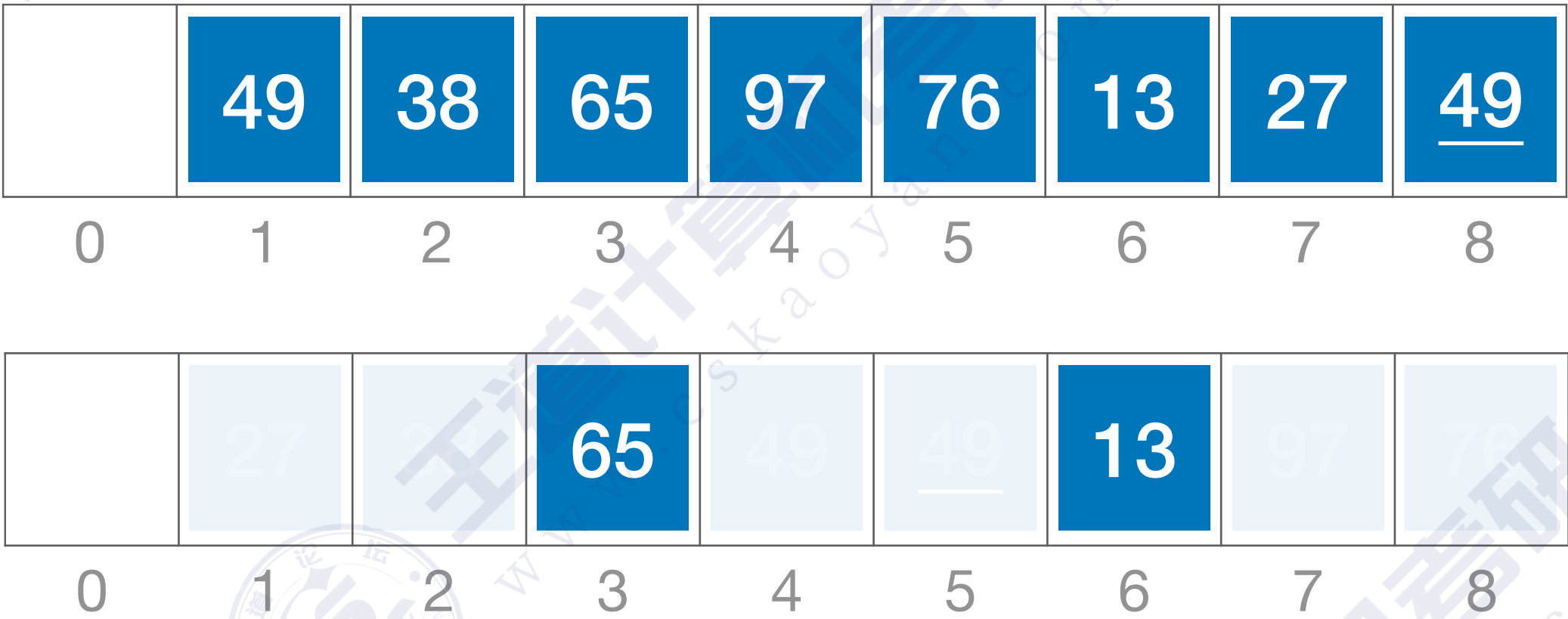
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



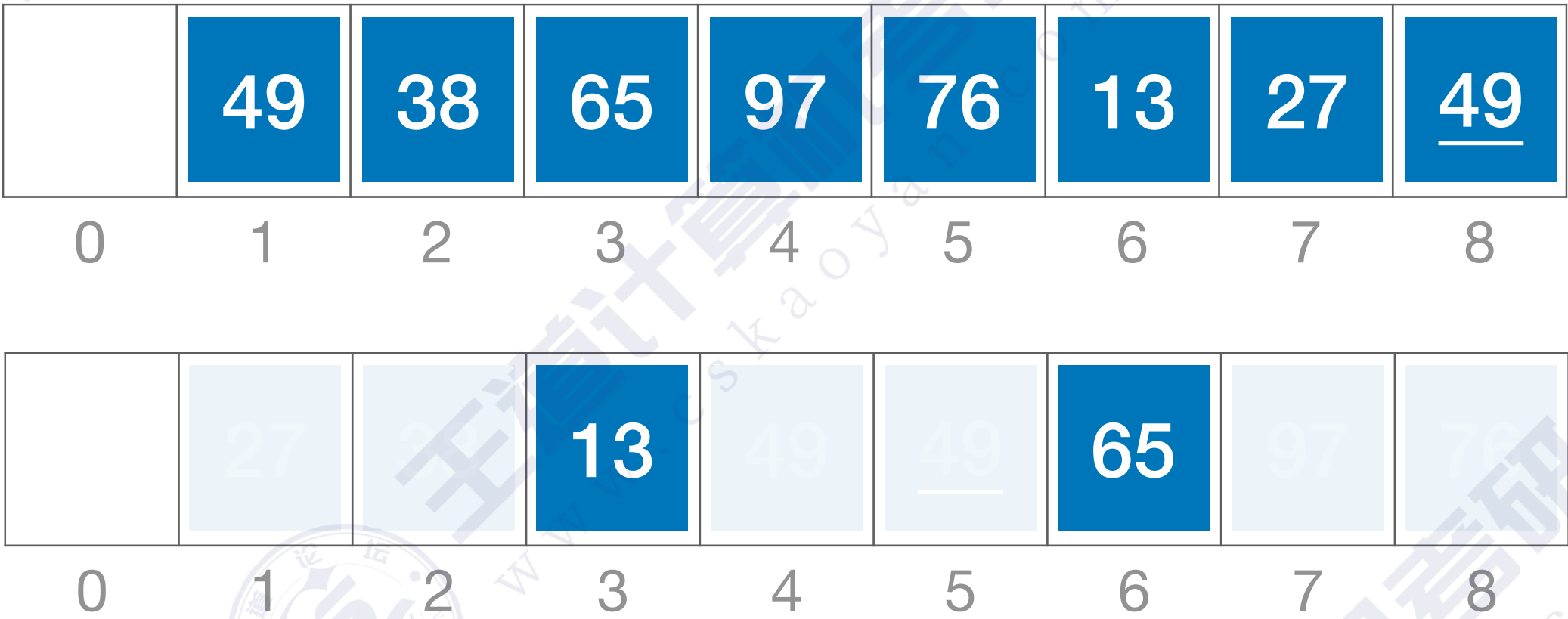
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



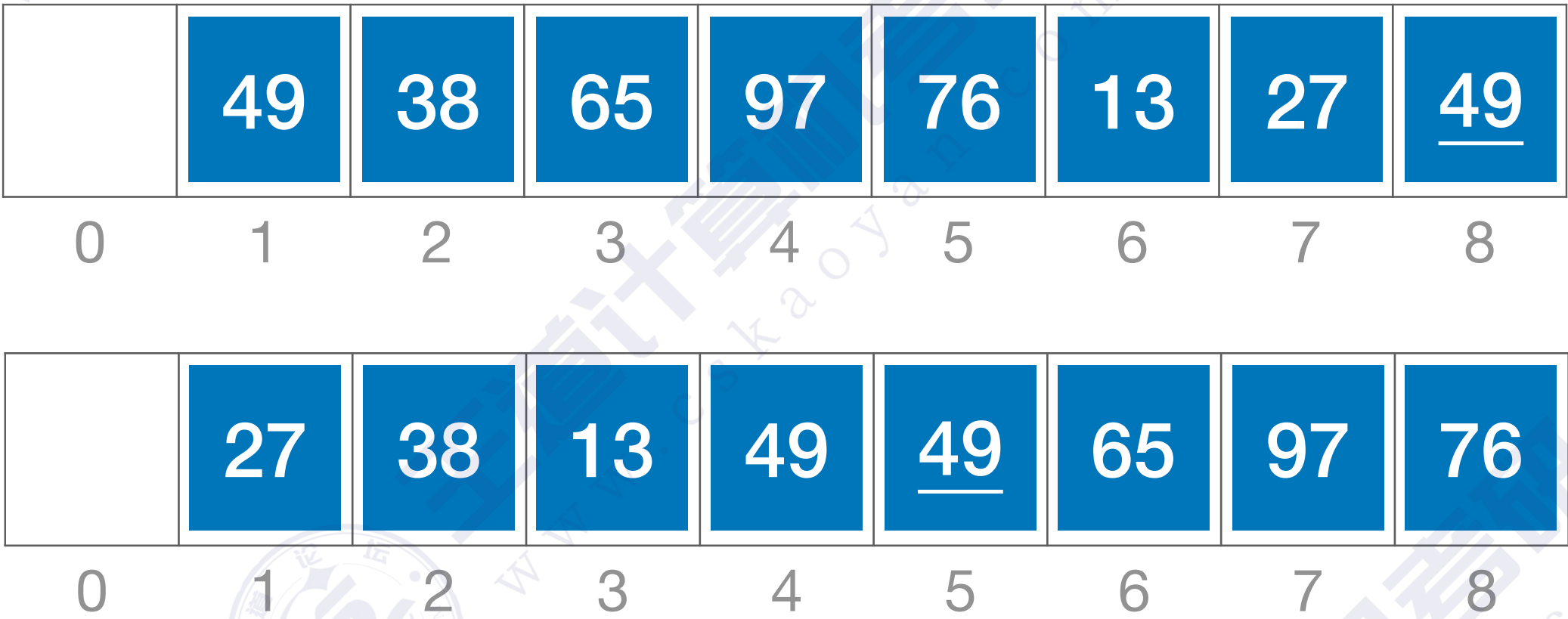
第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

第一趟： $d_1=3$



第二趟： $d_2=1$

希尔排序 (Shell Sort)



希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

考试中可能遇到各种增量

第一趟： $d_1=3$

	49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7	8

	27	38	13	49	<u>49</u>	65	97	76
0	1	2	3	4	5	6	7	8

第二趟： $d_2=1$

	13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7	8

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
    int d, i, j;
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
    for(d= n/2; d>=1; d=d/2)    //步长变化
        for(i=d+1; i<=n; ++i)
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
                A[0]=A[i];    //暂存在A[0]
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
                    A[j+d]=A[j];    //记录后移，查找插入的位置
                A[j+d]=A[0];    //插入
            }//if
    }
```

第一趟：d=n/2=4

	49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7	8

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

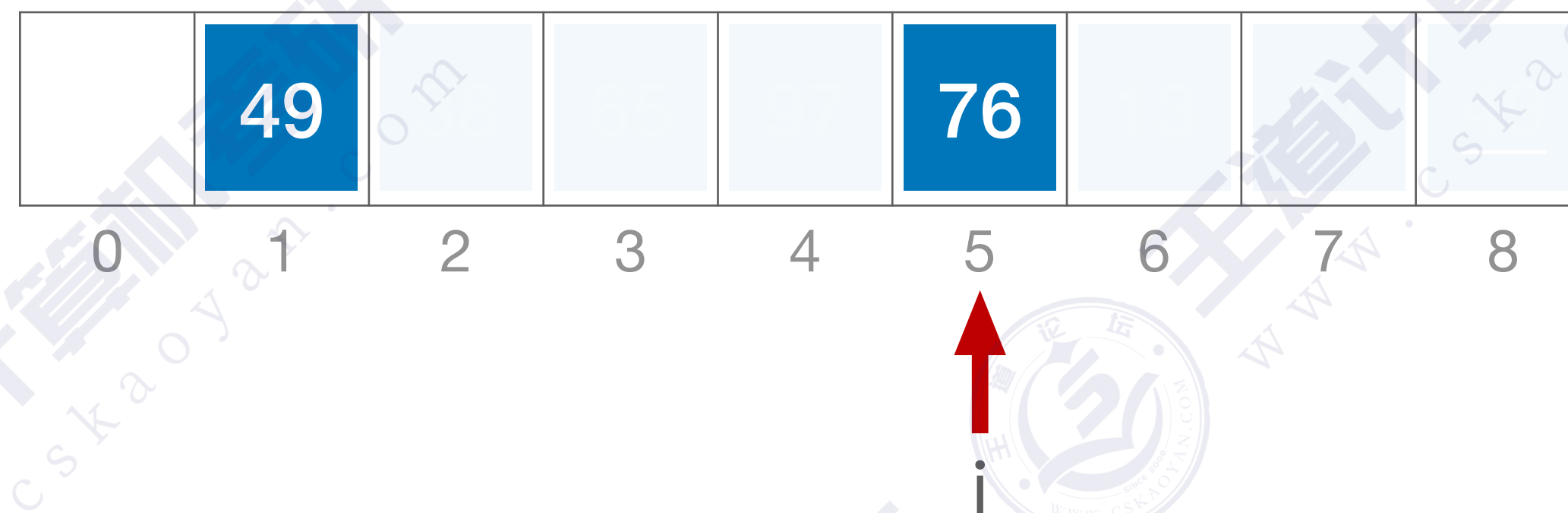
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

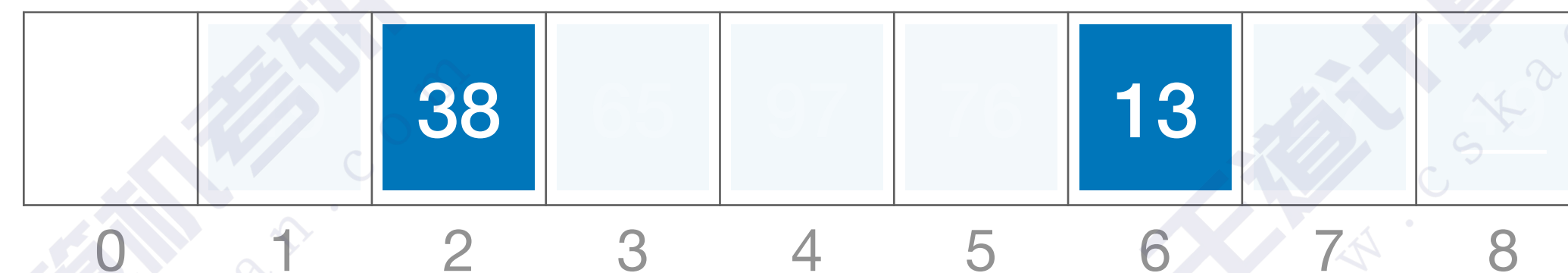
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

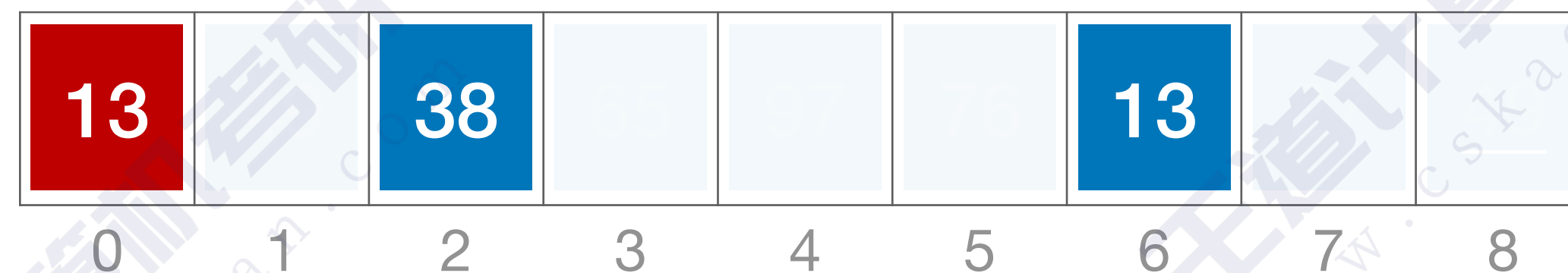
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

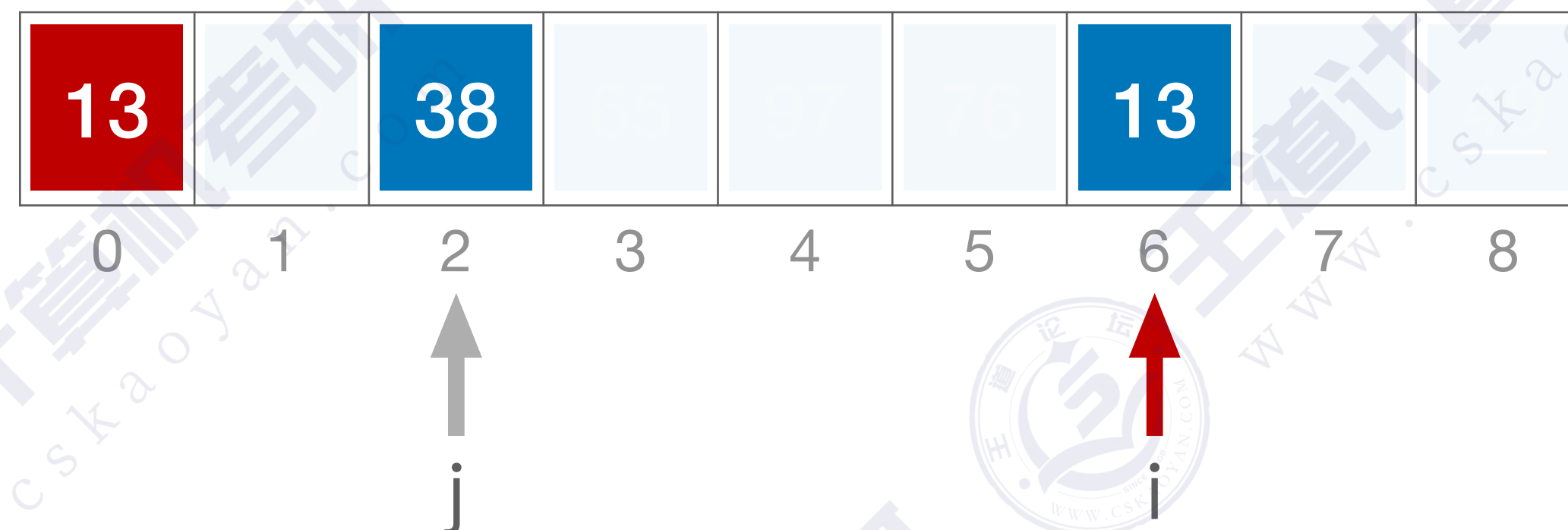
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

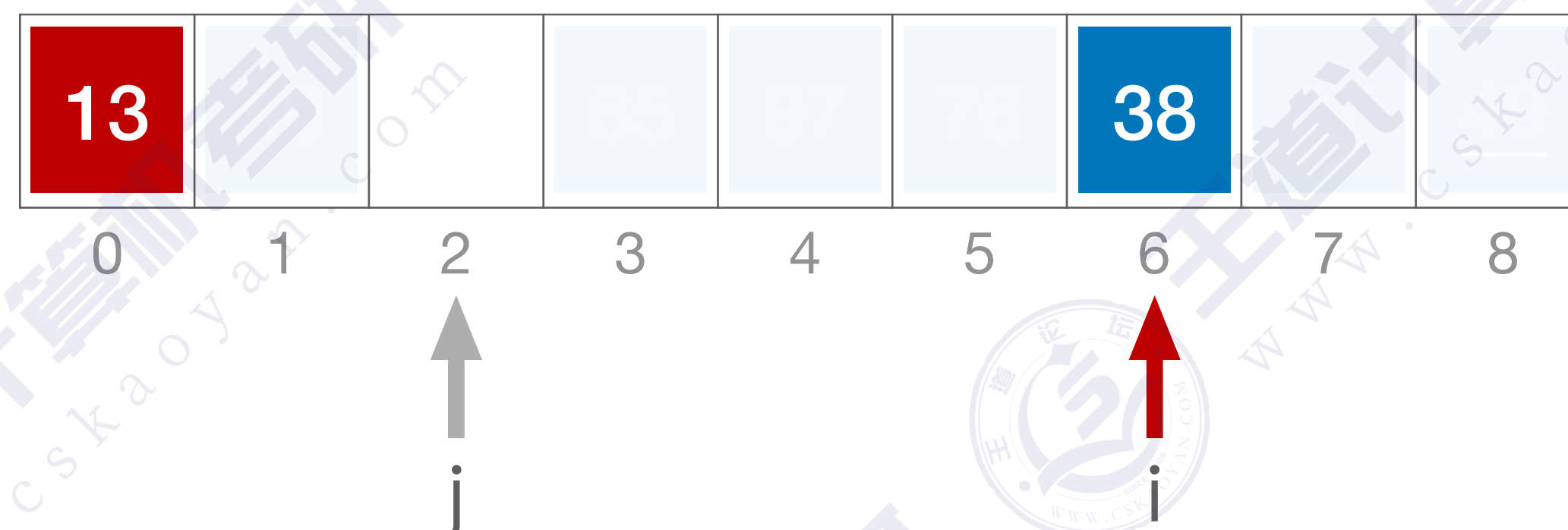
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

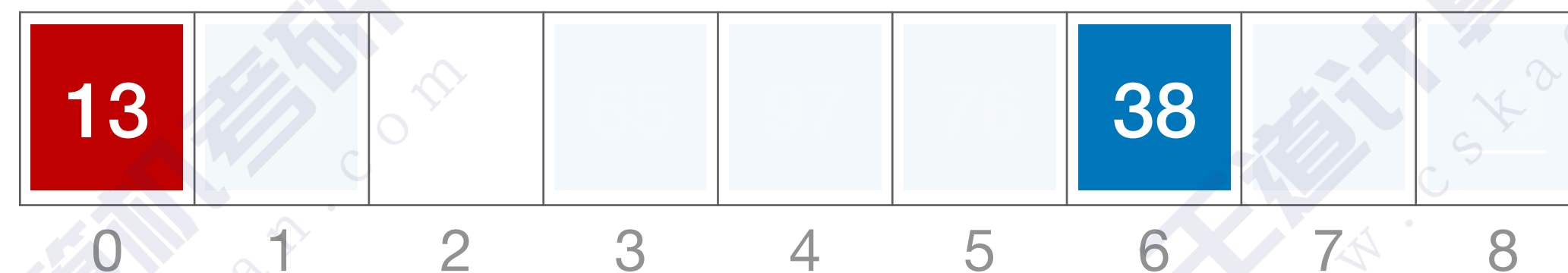
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第一趟：d=n/2=4



↑
j=-2

↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

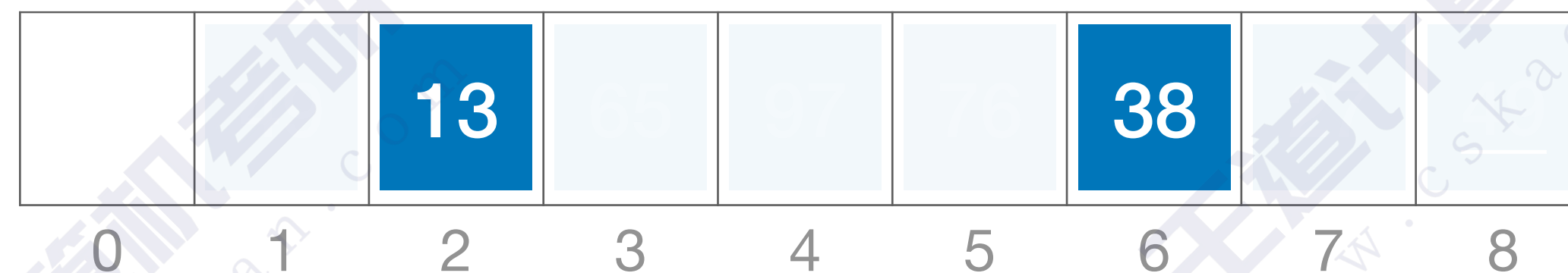
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

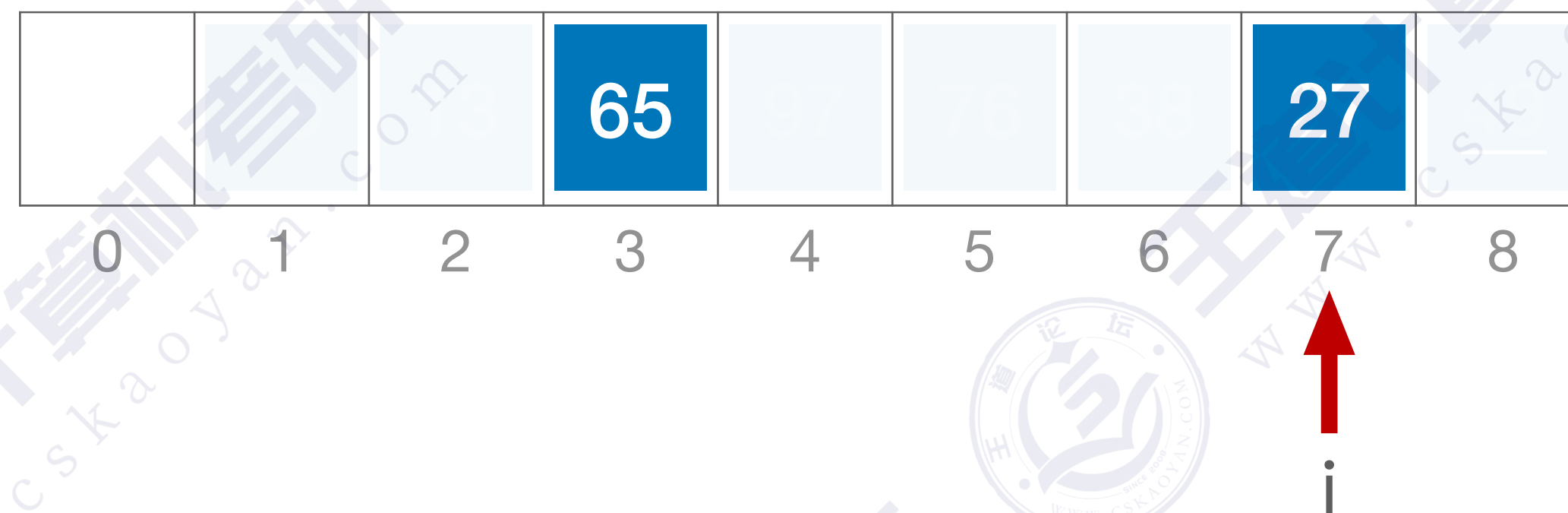
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
int d, i, j;
```

//A[0]只是暂存单元，不是哨兵，当 $j \leq 0$ 时，插入位置已到

```
for(d= n/2; d>=1; d=d/2) //步长变化
```

```
for(i=d+1; i<=n; ++i)
```

```
if(A[i]<A[i-d]){ //需将A[i]插入有序增量子表
```

```
A[0]=A[i];
```

```
//暫存在A[0]
```

```
for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
A[j+d]=A[j];
```

//记录后移，查找插入的位置

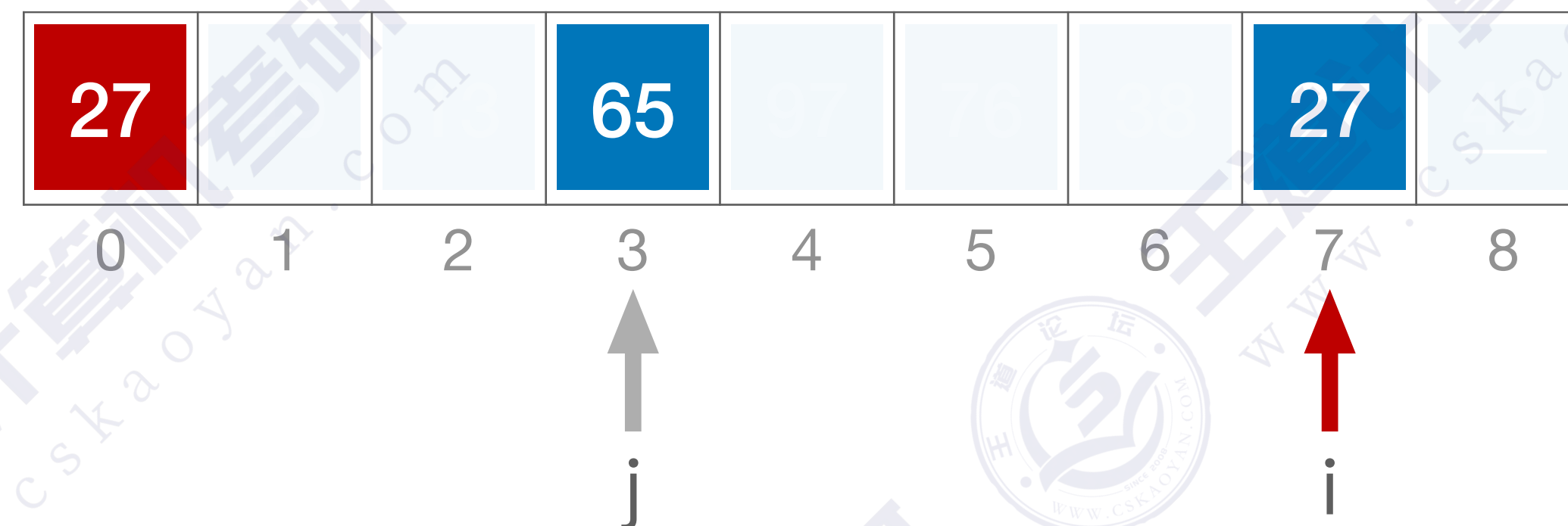
```
A[j+d]=A[0];
```

//插入

```
}//if
```

}

第一趟: $d=n/2=4$



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

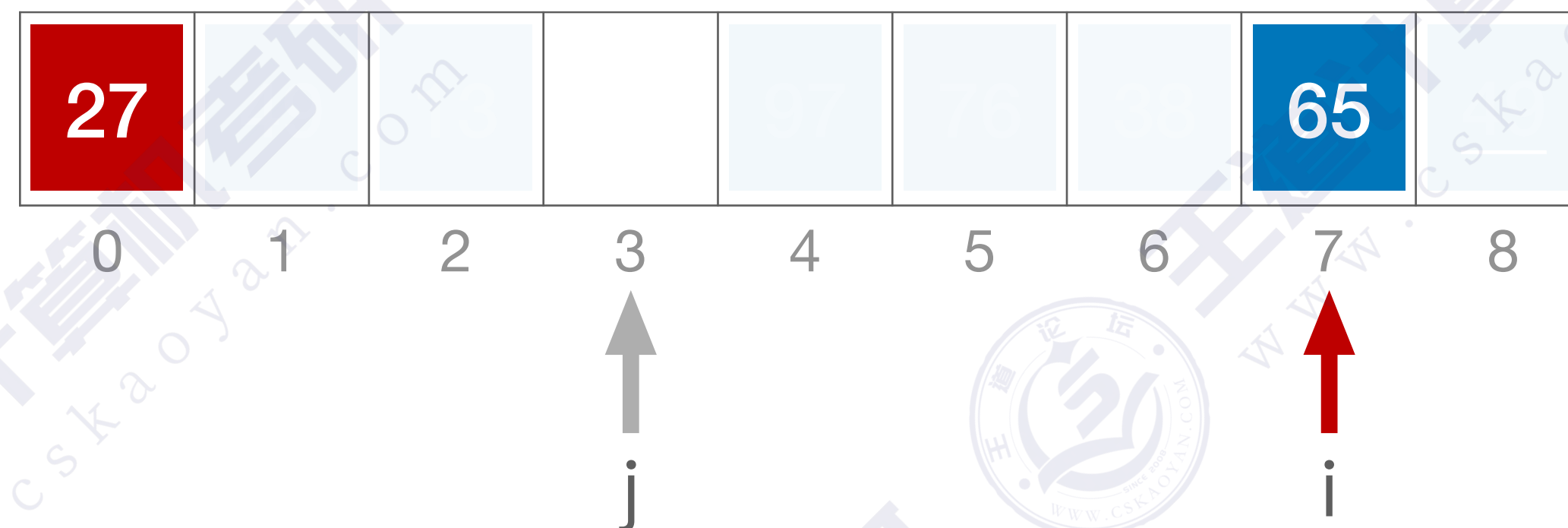
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

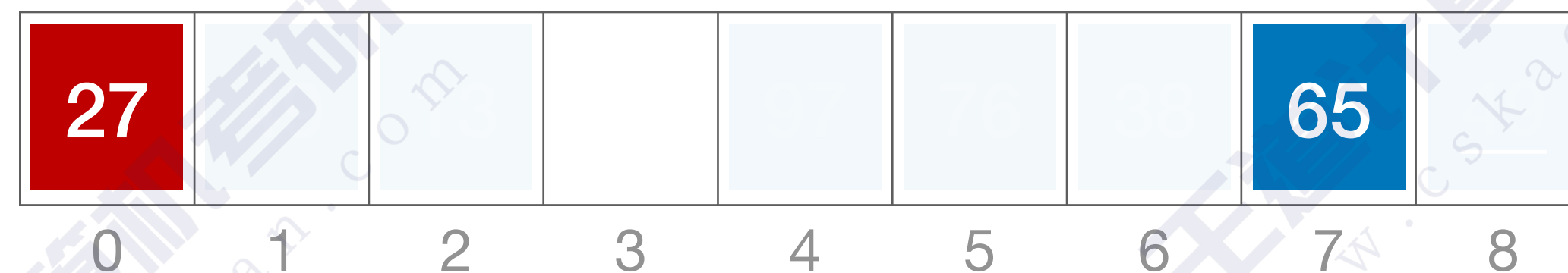
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



↑
j=-1

↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

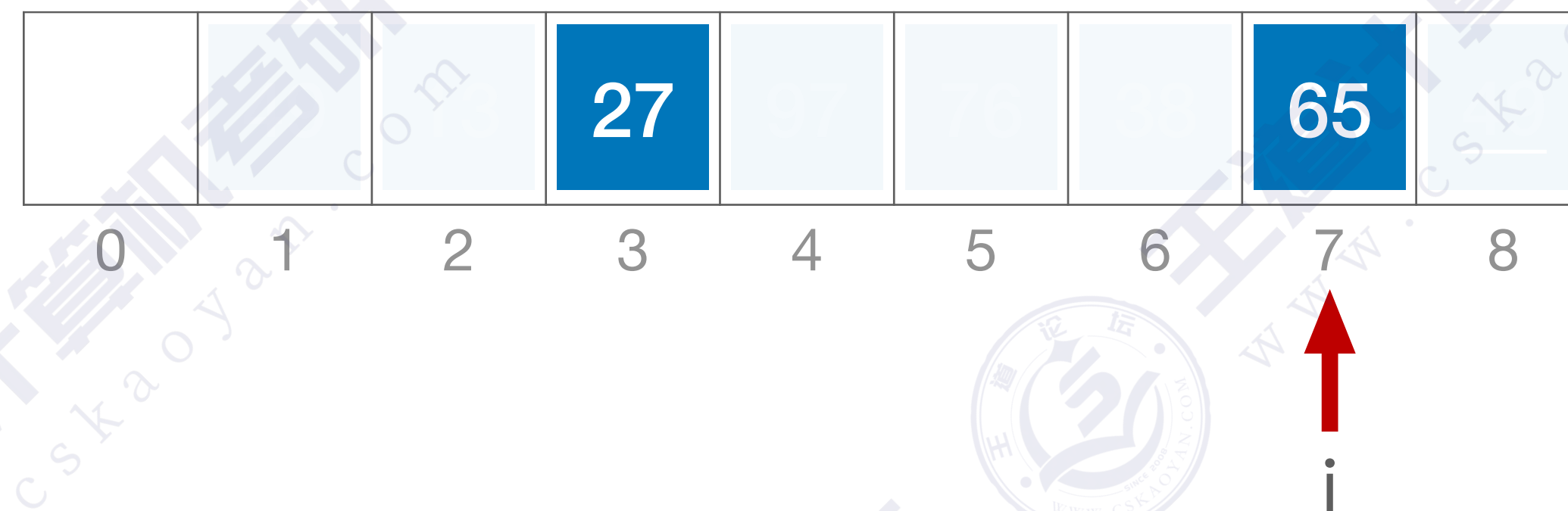
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

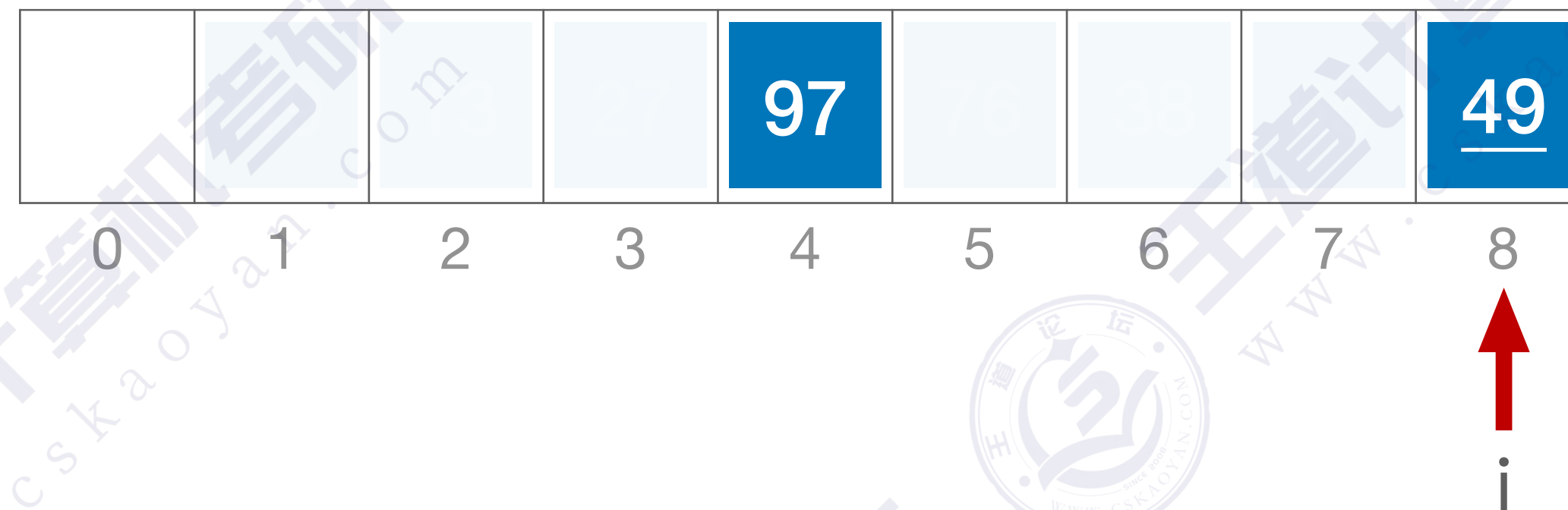
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

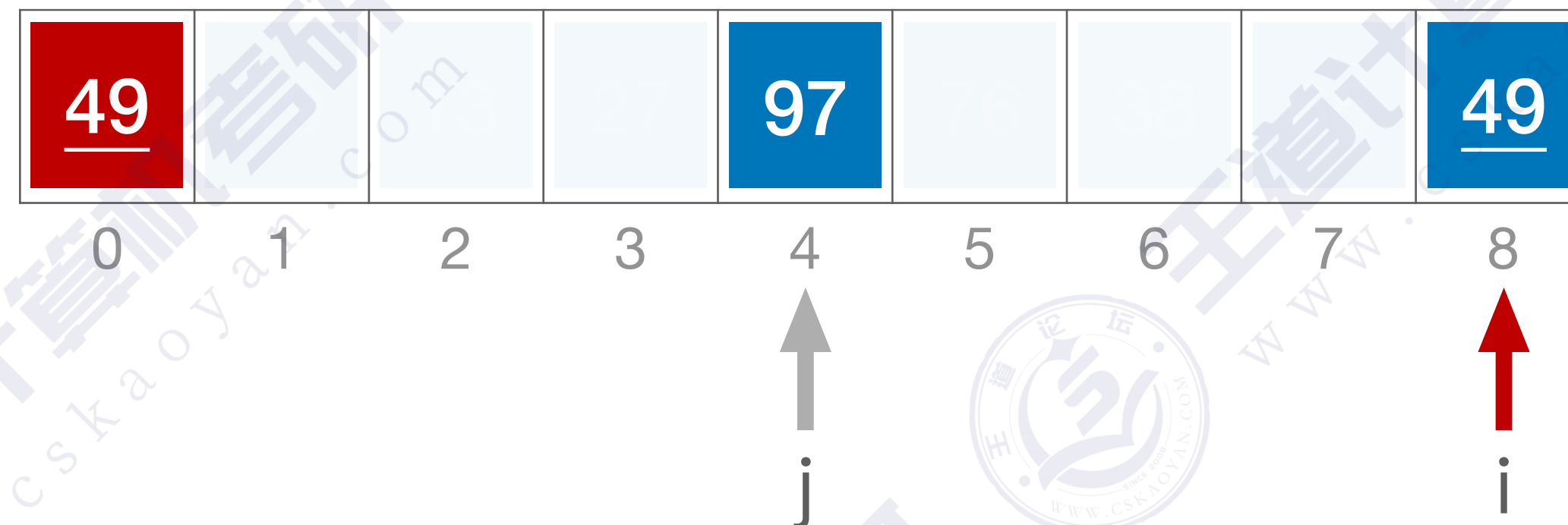
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

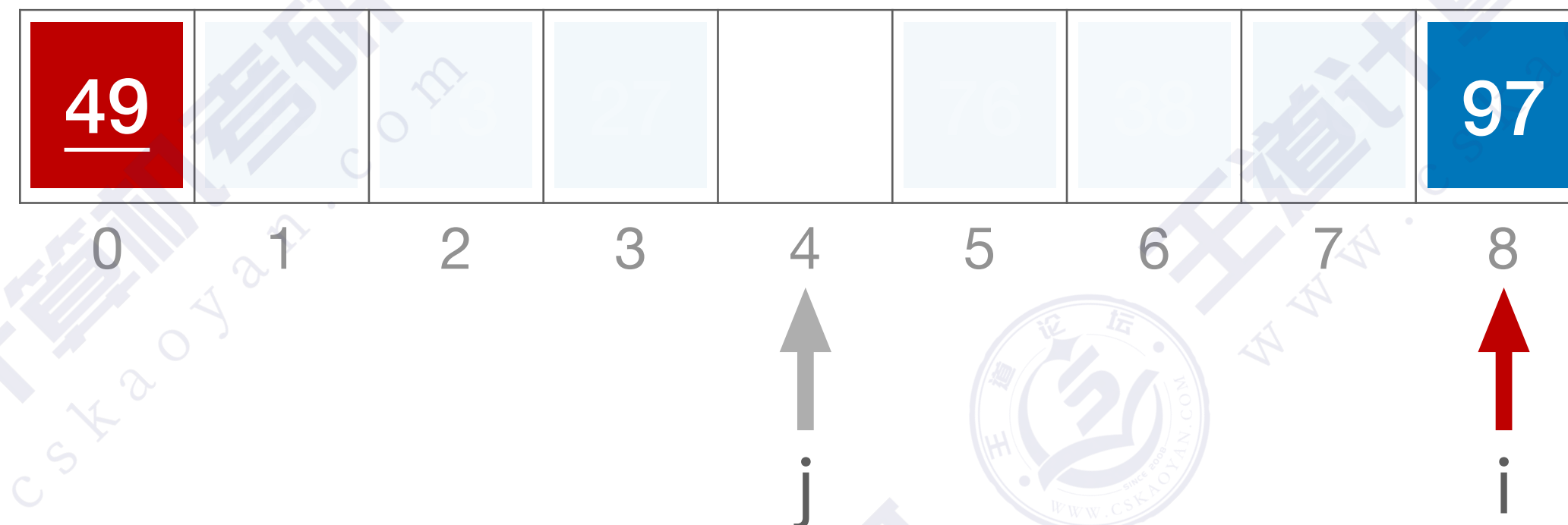
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

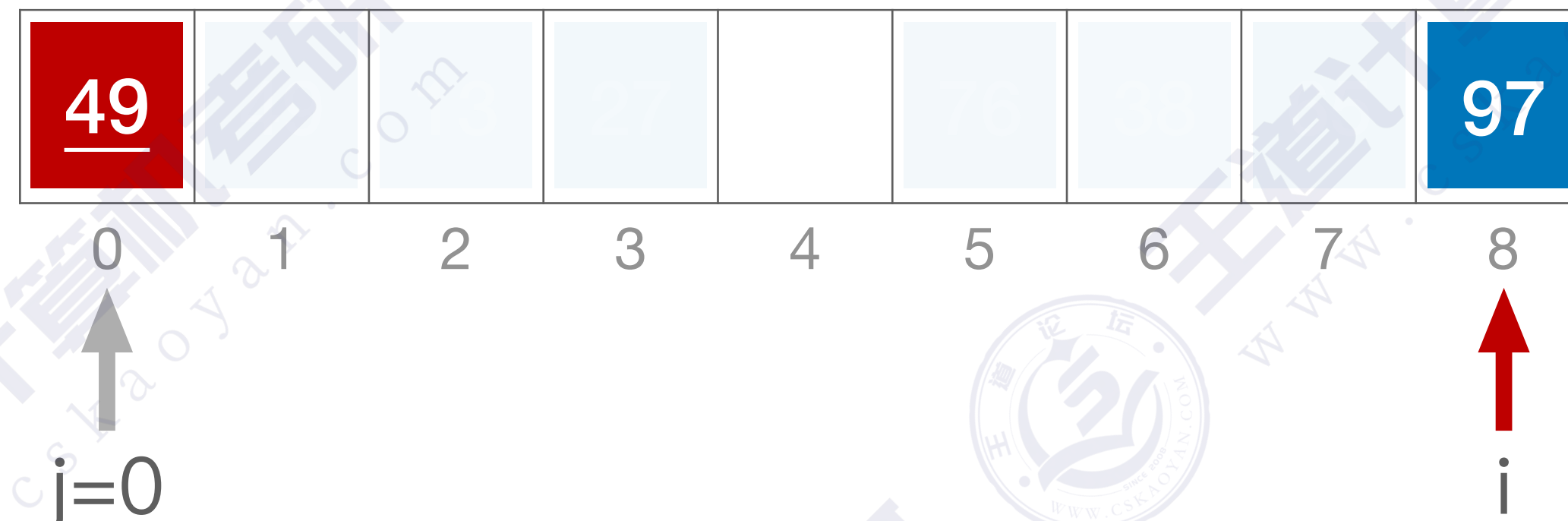
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

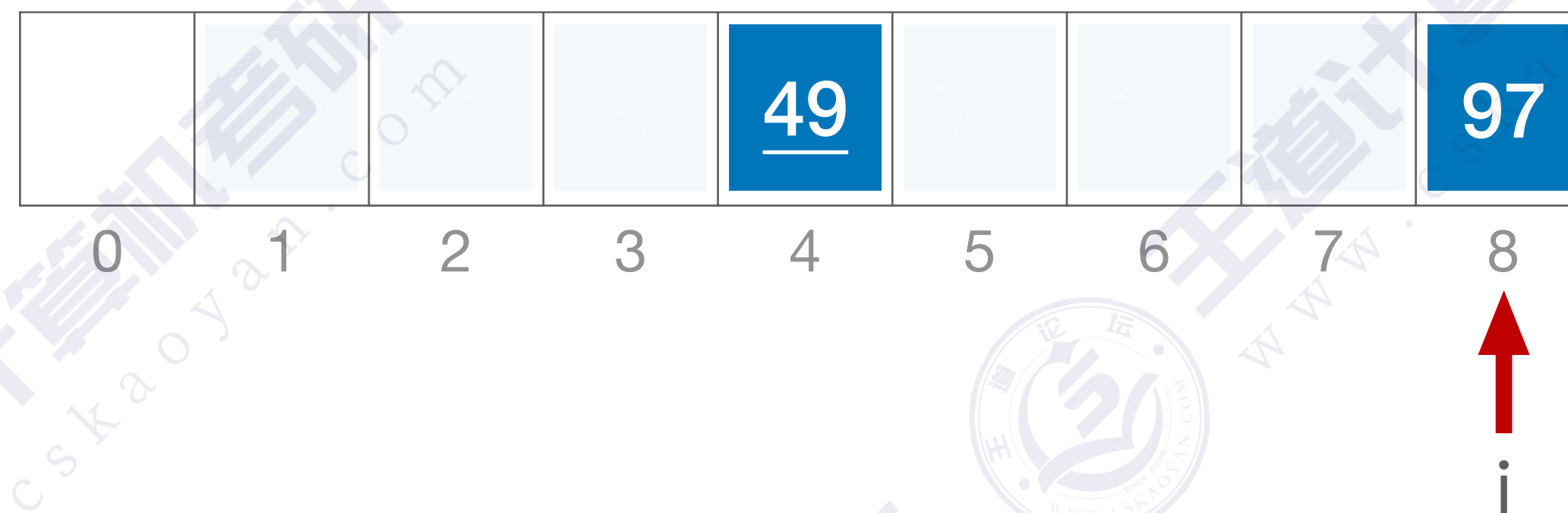
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第一趟：d=n/2=4



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第一趟：d=n/2=4



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2

	49	13	27	<u>49</u>	76	38	65	97
0	1	2	3	4	5	6	7	8

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

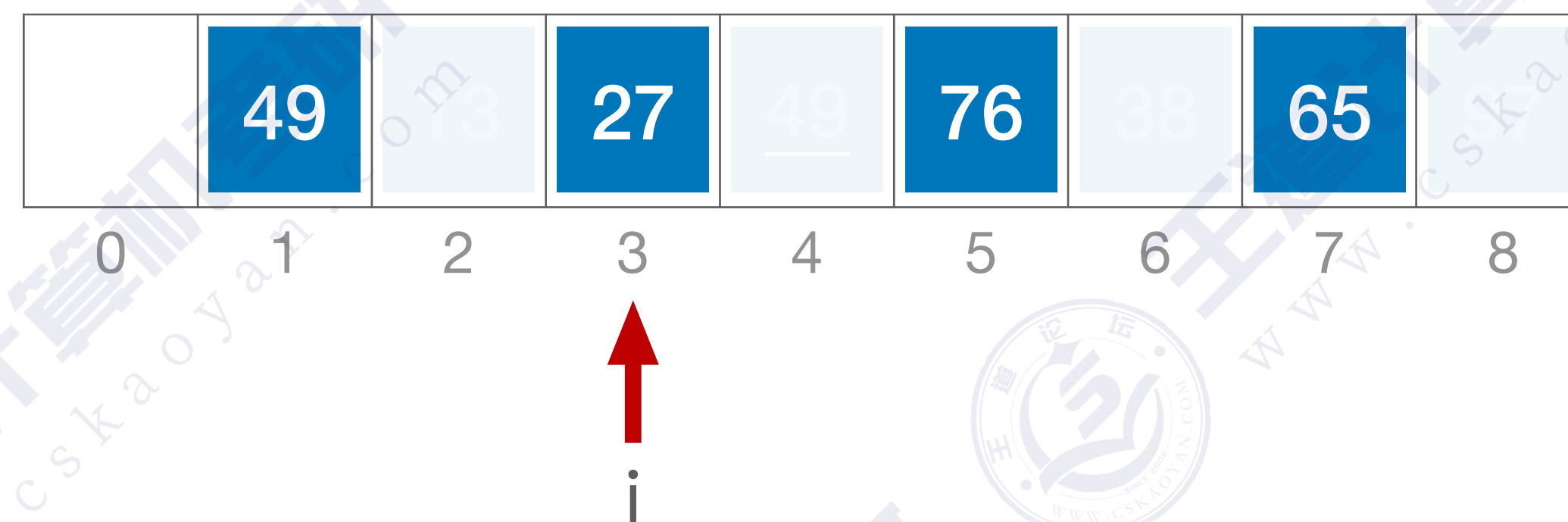
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

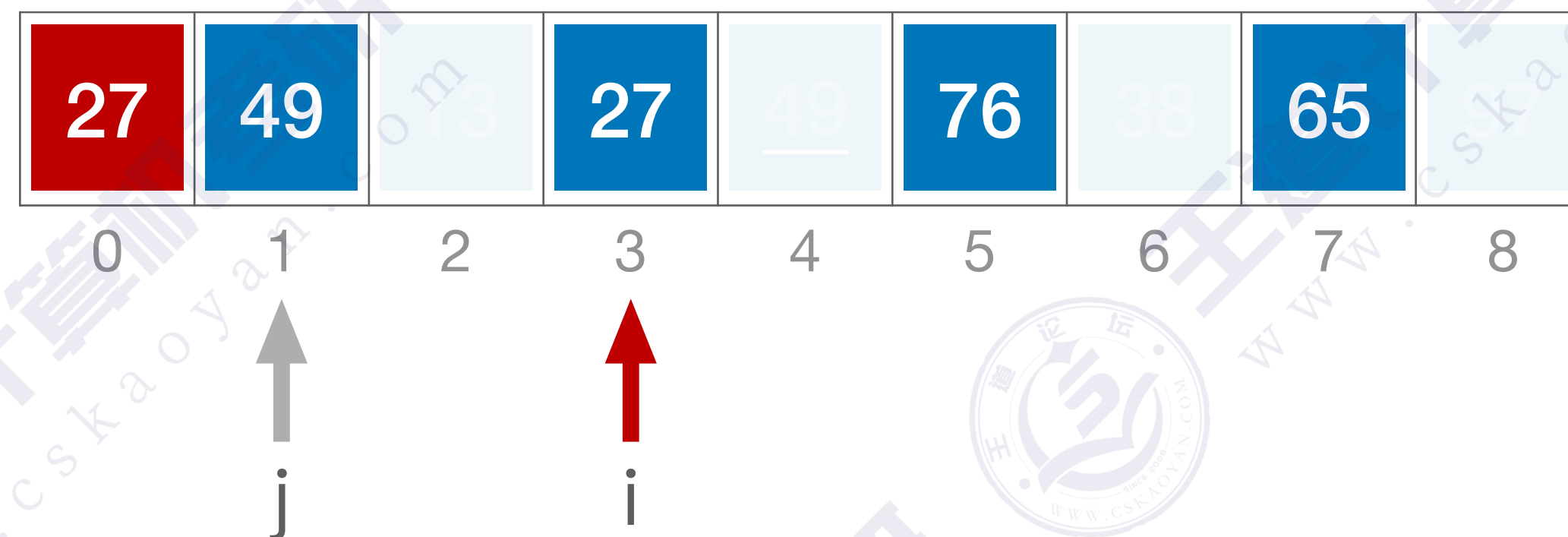
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

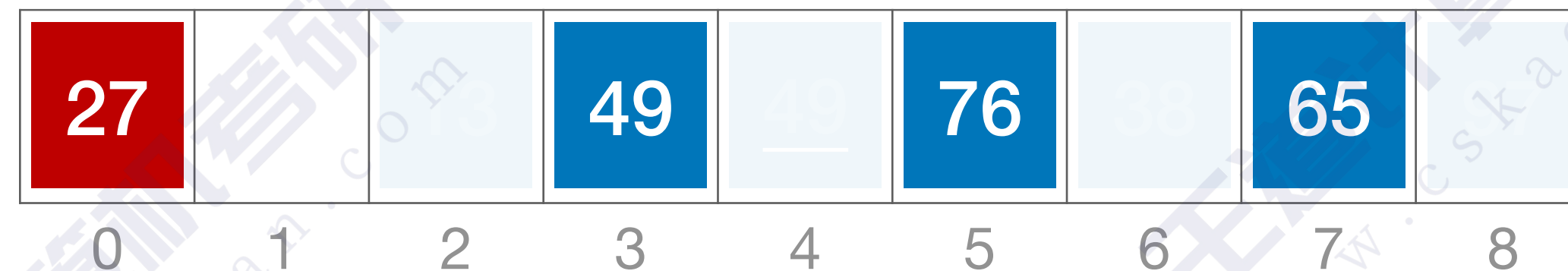
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



↑
j=-1

↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

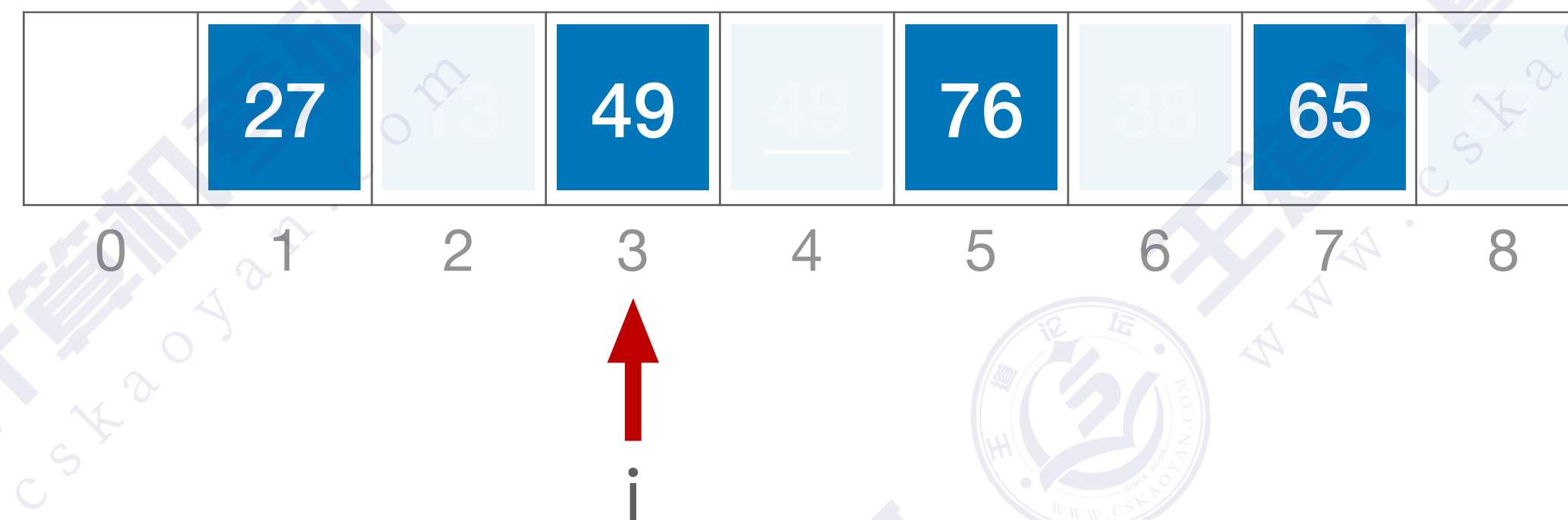
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

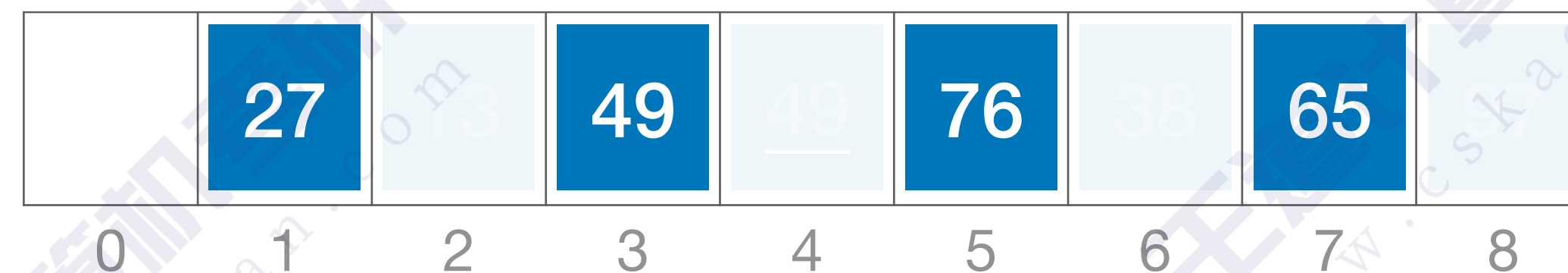
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

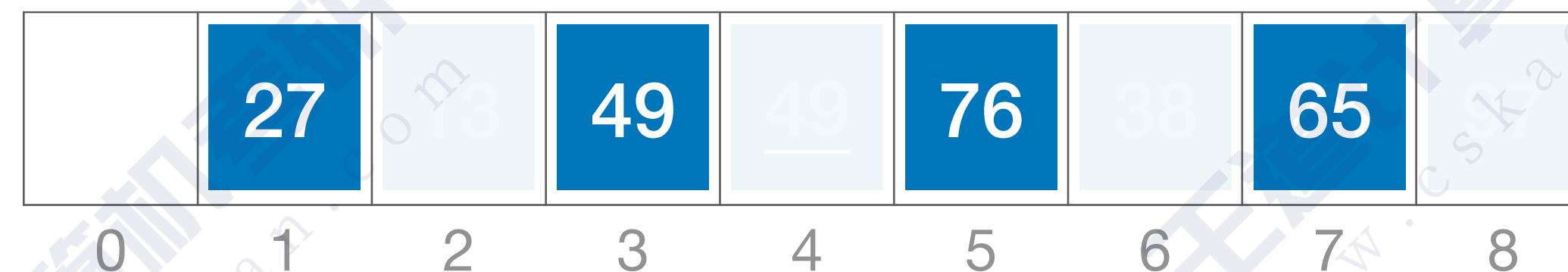
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

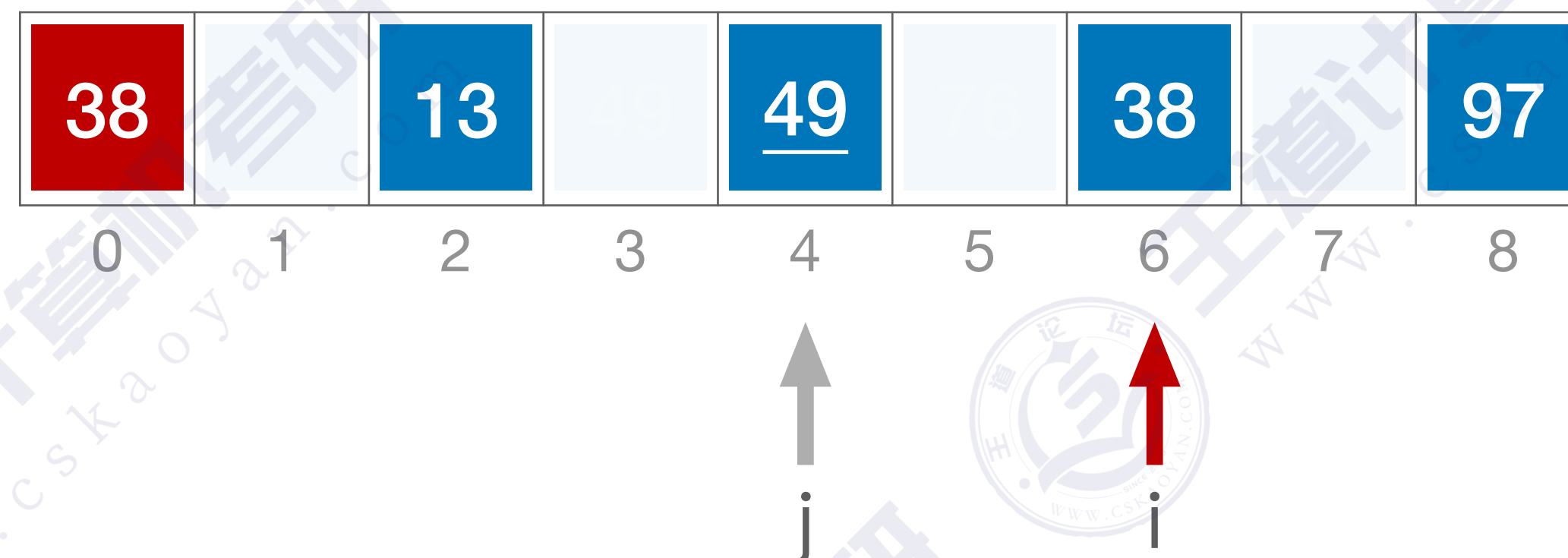
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

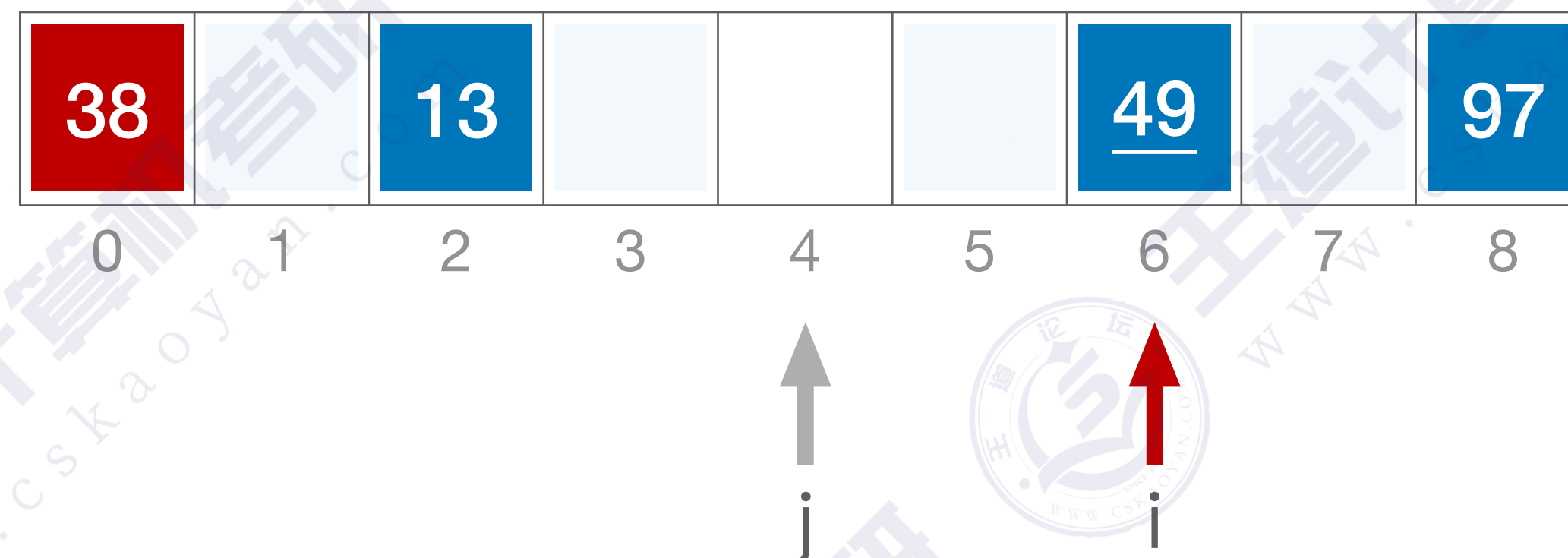
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

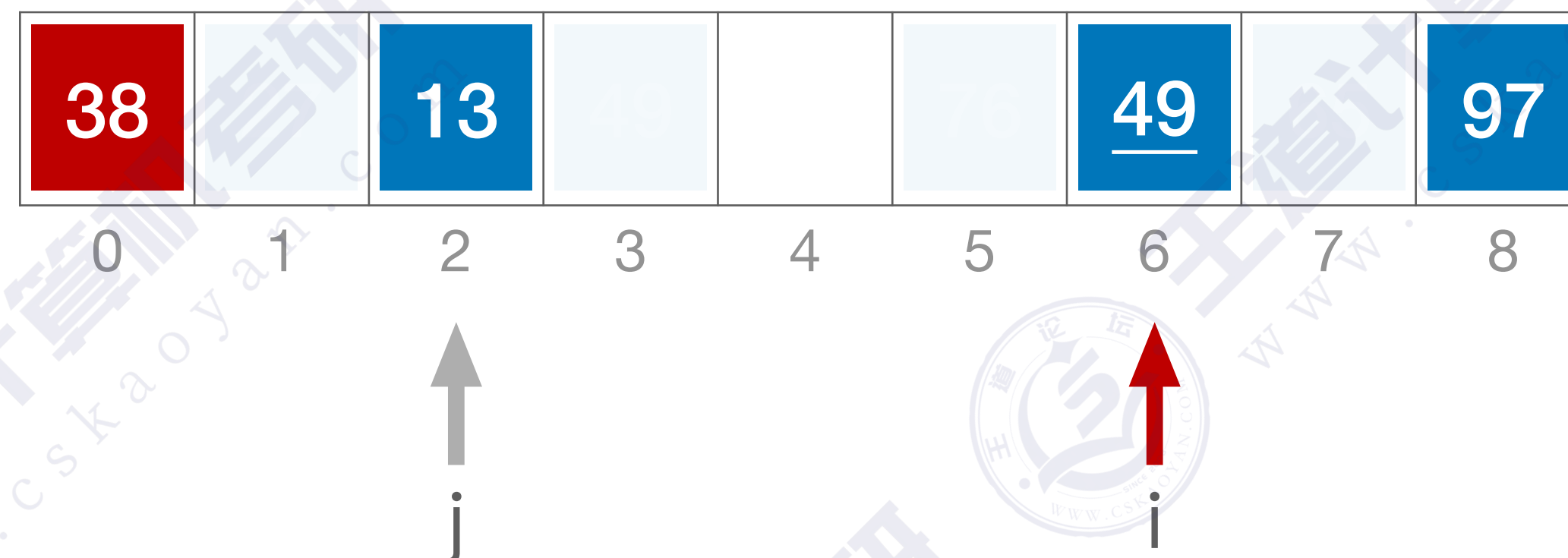
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

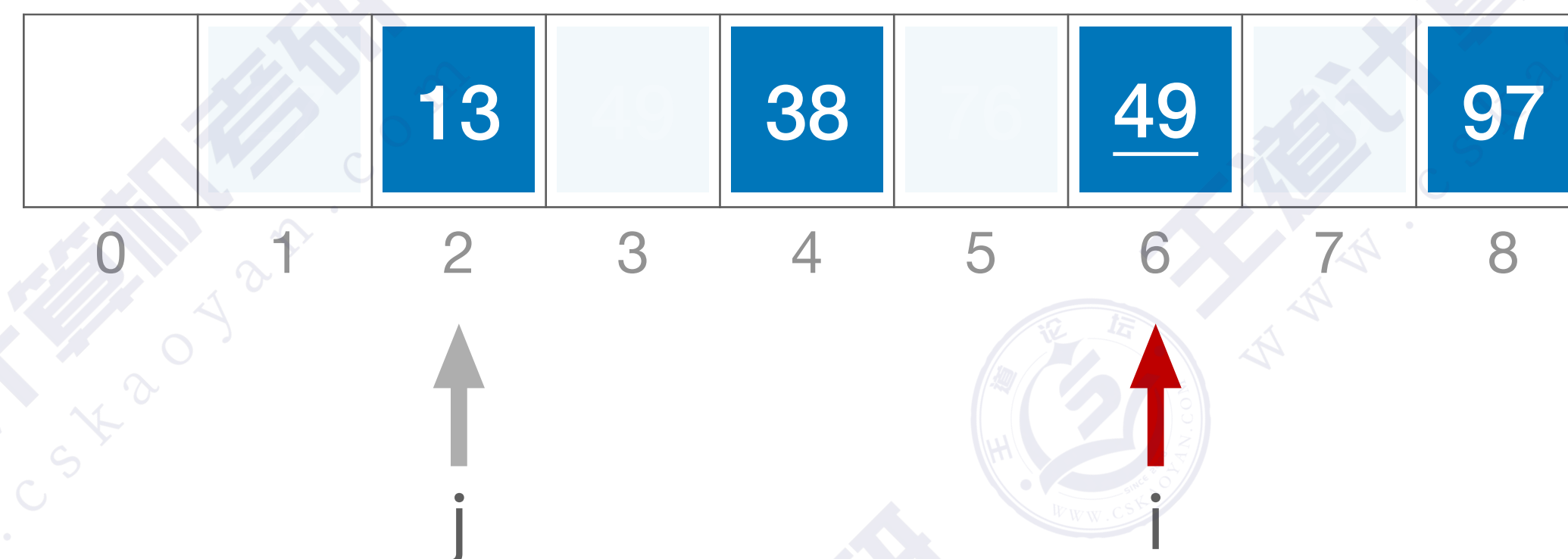
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

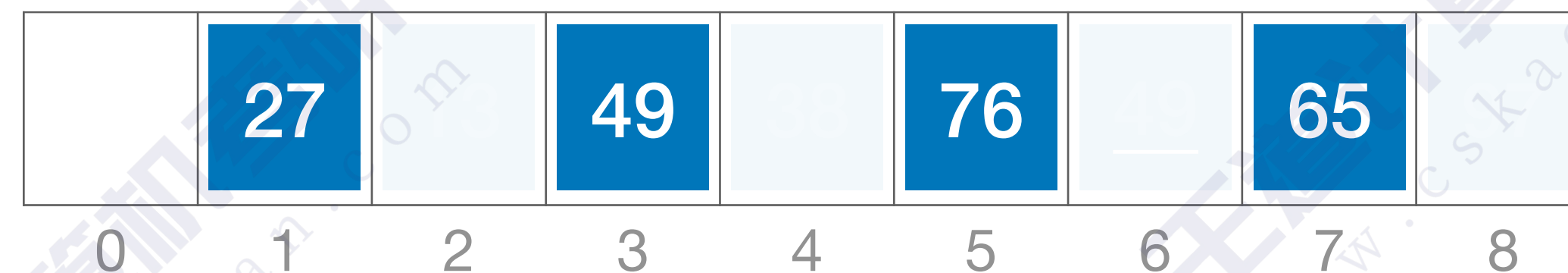
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

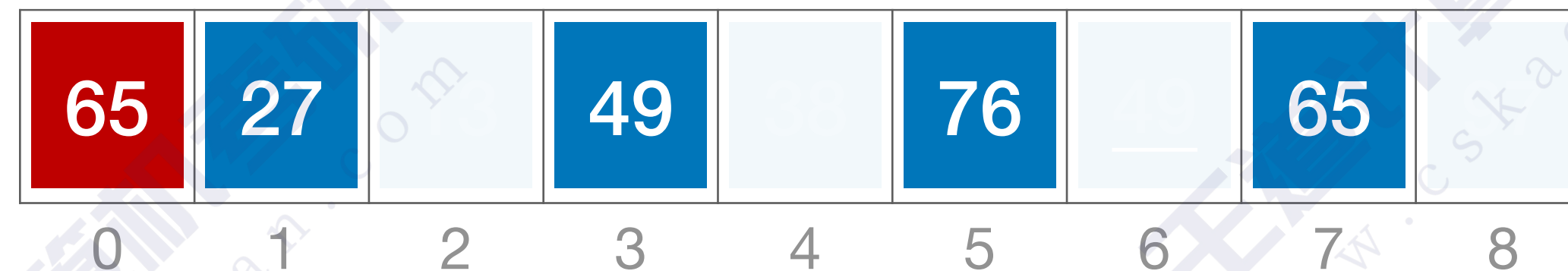
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



↑
j

↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

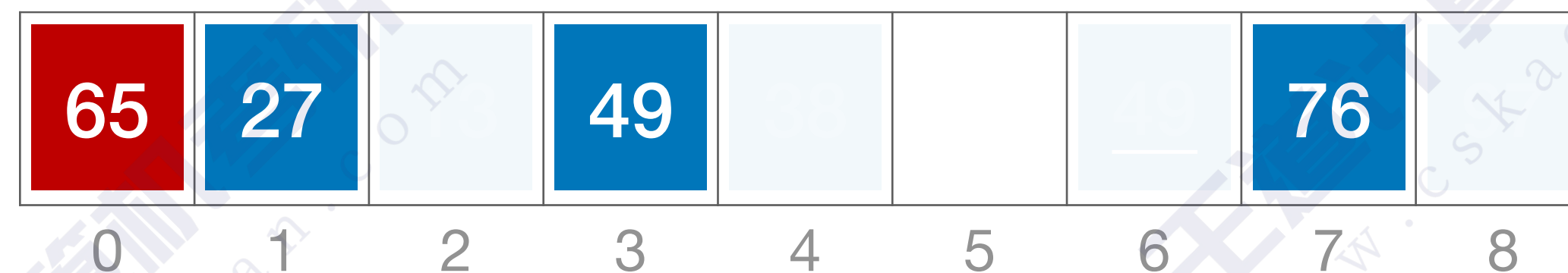
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

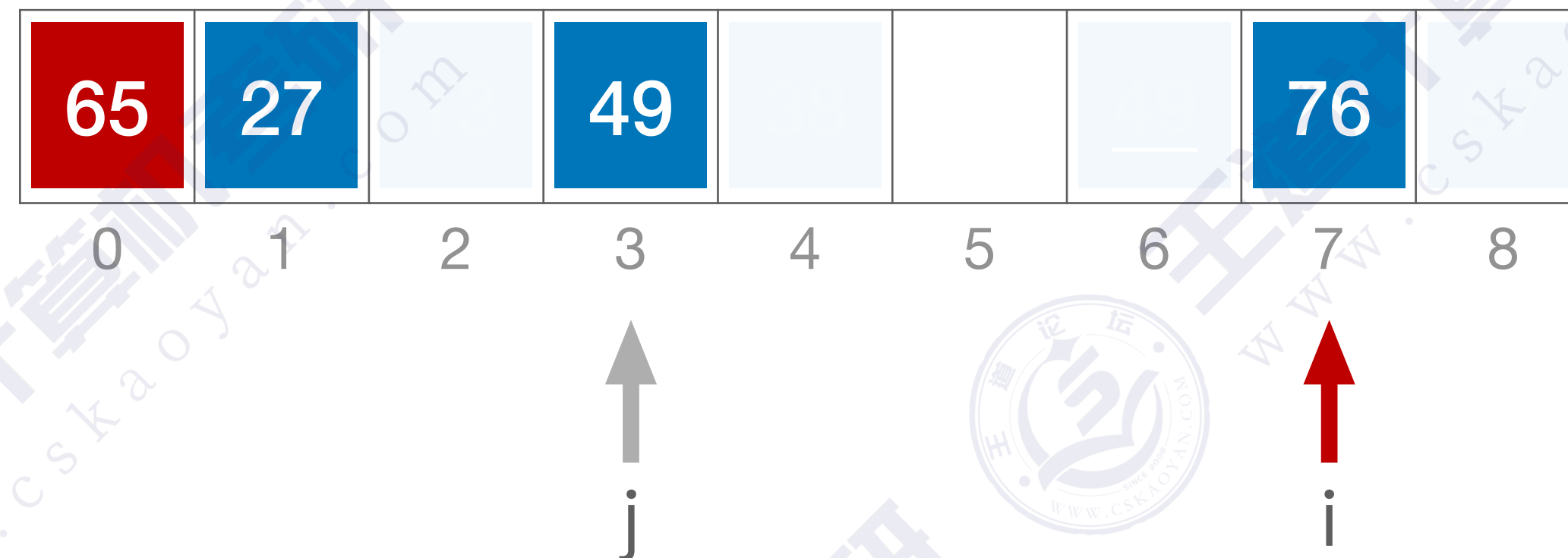
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

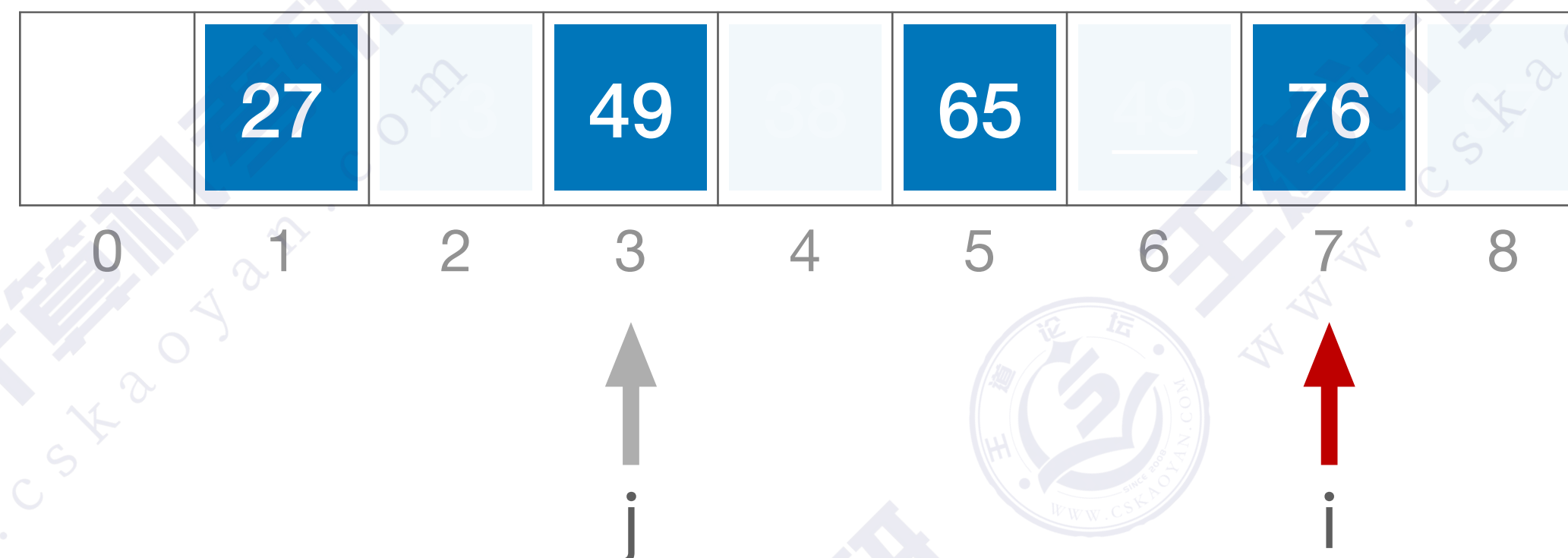
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
    }
```

第二趟：d=2



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第二趟：d=2



↑
i

算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

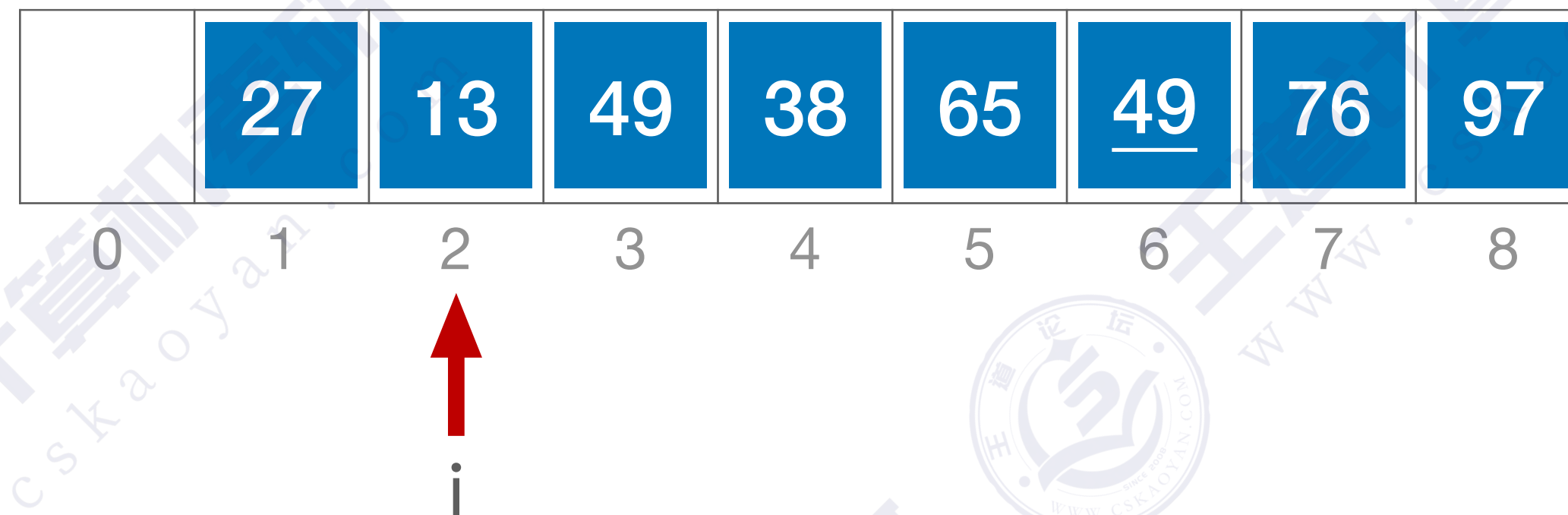
```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第三趟：d=1



算法实现

//希尔排序

```
void ShellSort(int A[],int n){
```

```
    int d, i, j;
```

```
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
```

```
    for(d= n/2; d>=1; d=d/2)    //步长变化
```

```
        for(i=d+1; i<=n; ++i)
```

```
            if(A[i]<A[i-d]){    //需将A[i]插入有序增量子表
```

```
                A[0]=A[i];    //暂存在A[0]
```

```
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
```

```
                    A[j+d]=A[j];    //记录后移，查找插入的位置
```

```
                A[j+d]=A[0];    //插入
```

```
            }//if
```

```
}
```

第三趟：d=1

	13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7	8

算法性能分析

空间复杂度: $O(1)$

	49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7	8

第一趟: $d_1=n/2=4$

49	13	27	<u>49</u>	76	38	65	97
----	----	----	-----------	----	----	----	----

第二趟: $d_2=d_1/2=2$

27	13	49	38	65	<u>49</u>	76	97
----	----	----	----	----	-----------	----	----

第三趟: $d_3=d_2/2=1$

13	27	38	49	<u>49</u>	65	76	97
----	----	----	----	-----------	----	----	----

第一趟: $d_1=3$

27	38	13	49	<u>49</u>	65	97	76
----	----	----	----	-----------	----	----	----

第二趟: $d_2=1$

13	27	38	49	<u>49</u>	65	76	97
----	----	----	----	-----------	----	----	----



时间复杂度: 和增量序列 $d_1, d_2, d_3...$ 的选择有关, 目前无法用数学手段证明确切的时间复杂度
最坏时间复杂度为 $O(n^2)$, 当 n 在某个范围内时, 可达 $O(n^{1.3})$

算法性能分析

原始序列:

65 49 49

第一趟: $d=2$

49 49 65

第二趟: $d=1$

49 49 65

稳定性: 不稳定!

适用性: 仅适用于顺序表, 不适用于链表

知识回顾与重要考点

先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。

希尔排序

性能

空间复杂度



$O(1)$

时间复杂度



未知，但优于直接插入排序

稳定性



不稳定

适用性



仅可用于顺序表

高频题型：给出增量序列，分析每一趟排序后的状态